

A PROJECT REPORT

MACROVA: A WEB-BASED DIET AND MACRONUTRIENT TRACKING APPLICATION WITH RULE-BASED ENERGY ESTIMATION AND AI-ASSISTED MEAL SUGGESTIONS

A project report submitted to
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
in partial fulfillment of the requirements for the award of the degree of
BACHELOR OF COMPUTER APPLICATIONS

BY
KARTHICK S
REG. NO. <DC2431006010XXX>

UNDER THE GUIDANCE OF
Dr. A. Thirumurthi Raja M.C.A., M.E., Ph.D.,
ASSISTANT PROFESSOR, SRMIST-DDE

DEPARTMENT OF COMPUTER APPLICATIONS
DIRECTORATE OF DISTANCE EDUCATION
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be university u/s 3 of UGC Act, 1956)
Kattankulathur – 603 203, Chennai, Tamilnadu

NOVEMBER 2026

BONAFIDE CERTIFICATE

This is to certify that the project report titled “MACROVA: A WEB-BASED DIET AND MACRONUTRIENT TRACKING APPLICATION WITH RULE-BASED ENERGY ESTIMATION AND AI-ASSISTED MEAL SUGGESTIONS” is a bonafide record of the independent project work carried out by KARTHICK S (Register Number DC2431006010XXX) in the Department of Computer Applications, Directorate of Distance Education, SRM Institute of Science and Technology, in partial fulfillment of the requirements for the award of the degree of Bachelor of Computer Applications, under my guidance and supervision.

The work described in this report has been verified against the accompanying software repository. Chapters 5 to 10 of the report describe the modules, the data model and the algorithms exactly as they are implemented in that repository, and Chapter 11 distinguishes the behaviour that was verified from the behaviour that is only inferred from the source.

To the best of my knowledge, the work reported herein is original, and it has not been submitted earlier, either in part or in full, for the award of any other degree or diploma of this or any other institution.

PROJECT GUIDE

Dr. A. Thirumurthi Raja
Assistant Professor,
SRMIST-DDE

HEAD OF THE DEPARTMENT

Dr. A. Thirumurthi Raja
Programme Coordinator,
Department of Computer Applications,
Directorate of Distance Education
(DDE),
SRMIST

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

I declare that this project report is my own work.

I, KARTHICK S (Register Number DC2431006010XXX), hereby declare that the project report entitled “MACROVA: A WEB-BASED DIET AND MACRONUTRIENT TRACKING APPLICATION WITH RULE-BASED ENERGY ESTIMATION AND AI-ASSISTED MEAL SUGGESTIONS” submitted to SRM Institute of Science and Technology in partial fulfillment of the requirements for the award of the degree of Bachelor of Computer Applications is a record of original software engineering work carried out by me under the supervision and guidance of Dr. A. Thirumurthi Raja, M.C.A., M.E., Ph.D., Assistant Professor and Programme Coordinator, Department of Computer Applications, Directorate of Distance Education.

I further declare that the source code submitted with this report is my own work, that the screens reproduced in Chapter 9 are captures of that code running on my own machine, and that the figures and tables were prepared by me specifically for this report. This project report, or any part thereof, has not been submitted earlier to any other university or institution for the award of any degree, diploma or fellowship.

Signature of the Candidate

Place: Chennai
Date:

ACKNOWLEDGEMENT

I am grateful to Dr. T. R. Paarivendhar, Chancellor, SRM Institute of Science and Technology, and to the Director of the Directorate of Distance Education, for the institutional support that made this project possible.

I record my sincere thanks to my guide, Dr. A. Thirumurthi Raja, Assistant Professor and Programme Coordinator, for his sustained attention to this work. His insistence that a claim in a report must be traceable to a line of code is the single instruction that shaped the verification chapter of this document more than any other.

I thank the faculty members of the Department of Computer Applications who reviewed the project at successive stages and whose comments changed the design of the tracking module in particular.

I acknowledge the maintainers of the open-source projects on which Macrova is built, in particular the Spring and Angular communities, whose documentation made the security and reactivity design of this application tractable for an undergraduate project.

Finally, I thank my parents and my friends for their patience and encouragement throughout the period of this work.

KARTHICK S

ABSTRACT

Dietary adherence, rather than the selection of a particular diet, is the variable that determines the outcome of most weight-management programmes, and adherence in turn depends on feedback that the user can understand and trust. The tools available to an individual dieter usually sit at one of two extremes: a spreadsheet in which every figure must be recalculated by hand, or a commercial application that prescribes a plan without exposing the arithmetic that produced it. Macrova is a web application that occupies the space between the two. Every kilocalorie and every gram of macronutrient shown to the user is produced by a documented rule and is stored against the plan that generated it, so that the numbers can be audited after the fact.

The system is implemented as a three-tier application. The presentation tier is a single-page client written in Angular 19 with standalone components, signals and functional HTTP interceptors. The application tier is a Spring Boot 3.4.4 service on Java 21 organised into packages for identity, diet planning, food suggestion, shared value objects, exception handling and stateless token security. The data tier is MongoDB, reached through Spring Data MongoDB, with one collection per aggregate: users, diet_plans and diet_plan_tracks, together with a fourth collection for suggestions that the delivered build does not write to. The tiers communicate only through a JSON interface whose twenty-six endpoints are enumerated and cross-checked against the controller classes in this report.

Energy requirements are computed with the Mifflin–St Jeor basal metabolic rate equation and multiplied by one of five activity factors to obtain total daily energy expenditure. Weight-loss targets apply a deficit of fifteen, twenty or twenty-five per cent, weight-gain targets apply a surplus of the same nominal intensity moderated by ten percentage points, and both results are rounded to a value ending in fifty or one hundred. Four macronutrient presets translate the energy target into gram targets, and a fixed split of thirty, thirty, fifteen and twenty-five per cent across breakfast, lunch, snack and dinner produces the per-meal budget against which the user logs intake.

A four-step wizard persists each step, so that the construction of a plan is resumable and backward navigation can be served from the stored document rather than from a client-side draft. Activating a plan creates the first day's tracking document, which is then updated whenever the user records intake or a new body weight. Logged macronutrients are converted back into energy with the Atwater factors, the percentage of the daily target achieved is recomputed on every update, and the weekly

weight observations are rendered as a trend chart. History screens expose every past plan and its day-by-day records through projections that avoid loading the embedded meal tracks.

An optional feature integrates a large language model through a chat-completions endpoint. The prompt is assembled from four filters and from the calorie allowance that the plan computes for the selected meal, and the JSON reply is parsed into suggestion cards. The report is explicit about the limits of that path: the nutritional values are generated and are not validated against a food-composition database, the parse depends on the model honouring an instructed schema, and the persistence method that would cache suggestions exists but is never invoked.

Security is implemented with stateless JSON Web Tokens: BCrypt password hashing, a custom authentication manager, a sign-in filter that issues an HMAC-512 signed refresh token as an HttpOnly cookie, a bearer verification filter and an exception-translating filter. Every plan and tracking operation is scoped to the authenticated principal inside the service layer, so guessing an identifier does not expose another user's data. The report also records the concrete weaknesses of the present configuration, among them the absence of refresh-token rotation and revocation, a cross-origin allow-list pinned to the development origin and input ranges that are enforced in the browser only.

The delivered system comprises 71 Java source files (2,779 lines), 60 TypeScript files (1,915 lines), 20 component templates (1,238 lines) and 19 stylesheets (2,381 lines), together with the twenty-six endpoints, three collections written in normal operation and a shared model of value objects and enumerations. The report closes with a verification chapter that maps every documented behaviour to the artefact that implements it, a summary of the limitations that remain, and an ordered list of enhancements that the present architecture can accommodate.

Keywords: macronutrient tracking, Mifflin–St Jeor equation, total daily energy expenditure, Angular 19 standalone components, Spring Boot 3.4.4, Spring Security, JSON Web Token, MongoDB, Spring Data, Chart.js, large language model, diet plan wizard, software verification.

TABLE OF CONTENTS

Title Page i
Bonafide Certificate ii
Declaration iii
Acknowledgement iv
Abstract v
Table of Contents vii
List of Figures xi
List of Tables xii
Abbreviations and Acronyms xiii

CHAPTER 1: INTRODUCTION 1
1.1 Overview of the Project 1
1.2 Background of the Study 1
1.3 Problem Statement 2
1.4 Objectives of the Project 3
1.5 Scope of the Project 3
1.6 Project Motivation 4
1.7 Assumptions and Constraints 5
1.8 Organisation of the Report 5
CHAPTER 2: LITERATURE REVIEW 7
2.1 Introduction 7
2.2 Review of Related Work 7
2.2.1 Predictive equations for resting energy expenditure 7
2.2.2 Re-evaluation of predictive equations in clinical practice 7
2.2.3 Physical activity levels and total energy expenditure 8
2.2.4 Self-monitoring as the active ingredient of weight management 8
2.2.5 The effect of goal setting and feedback on adherence 8
2.2.6 Standalone components and signals in Angular 9
2.2.7 Functional HTTP interceptors for cross-cutting concerns 9
2.2.8 Stateless authentication with JSON Web Tokens 9
2.2.9 Adaptive password hashing with BCrypt 10
2.2.10 Spring Data repositories and projection queries 10
2.2.11 Document databases for aggregate-oriented persistence 11
2.2.12 Large language models as constrained generators 11
2.2.13 Nutritional misinformation and the limits of generated advice 11
2.2.14 Chart libraries and accessible data visualisation 12
2.3 Comparative Study of the Reviewed Work 12
2.4 Research Gaps Identified 13
2.5 Summary of the Literature Review 13
CHAPTER 3: SYSTEM ANALYSIS 15
3.1 Existing System Analysis 15
3.2 Limitations of the Existing System 15
3.3 Proposed System 16
3.4 Advantages of the Proposed System 17

3.5 Feasibility Study 18
3.5.1 Technical Feasibility 18
3.5.2 Economic Feasibility 18
3.5.3 Operational Feasibility 19
3.5.4 Schedule and Effort Feasibility 19
3.6 Software Requirements Specification 19
CHAPTER 4: SYSTEM DESIGN 22
4.1 Design Overview and Principles 22
4.2 System Architecture 23
4.3 Data Flow Diagrams 25
4.3.1 Level 0 Data Flow Diagram (Context Diagram) 25
4.3.2 Level 1 Data Flow Diagram (Functional Decomposition) 25
4.3.3 Level 2 Data Flow Diagram (Macro Tracking Refinement) 26
4.4 Use Case Diagram and Actor Specification 27
4.5 Object-Oriented and Behavioural Models 28
4.5.1 Class Diagram 28
4.5.2 Sequence Diagram 28
4.5.3 Activity Diagram 30
4.5.4 State Machine Diagram 31
4.6 Database Design 32
4.7 User Interface Design System 33
4.8 Security Design 33
CHAPTER 5: TECHNOLOGIES AND TOOLS USED 35
5.1 Software Requirements 35
5.2 Hardware Requirements 36
5.3 Programming Languages Used 36
5.3.1 Java 21 for the Application Tier 36
5.3.2 TypeScript 5.7 for the Presentation Tier 37
5.3.3 HTML5, CSS3 and JSON 37
5.4 Frameworks and Libraries 37
5.5 Development and Runtime Environment 38
5.6 Build and Run Workflow 39
CHAPTER 6: MODULES AND IMPLEMENTATION OVERVIEW 40
6.1 Module 1: Identity and Session Management 40
6.2 Module 2: Diet Plan Construction 41
6.3 Module 3: Daily Tracking and Progress 42
6.4 Module 4: AI-Assisted Food Suggestion 43
6.5 Module 5: Cross-Cutting Infrastructure 44
6.6 Module 6: Application Shell, Routing and Session 44
6.7 Module 7: Plan and Tracking Screens 45
6.8 Module 8: History, Suggestions and Fallback Screens 46
CHAPTER 7: DATABASE DESIGN AND DATA DICTIONARY 47
7.1 Design Approach 47
7.2 The users Collection 48
7.3 The diet_plans Collection 48
7.4 The diet_plan_tracks Collection 50

7.5 The ai_food_suggests Collection 51
7.6 Indexing, Consistency and Integrity 52
CHAPTER 8: REST INTERFACE AND CLIENT SERVICE LAYER 53
8.1 Conventions 53
8.2 Endpoint Catalogue 53
8.3 Request and Response Examples 54
8.4 Error Responses 56
8.5 Client Service Layer 57
8.6 Interceptors and Cross-Cutting Client Concerns 58
CHAPTER 9: USER INTERFACE DESIGN AND APPLICATION SCREENS 59
9.1 Design Objectives 59
9.2 Registration and Authentication Screens 59
9.2.1 Create Account Screen 59
9.2.2 Log In Screen 60
9.3 Empty State and Plan Initiation 60
9.3.1 No Active Diet Plan Screen 60
9.4 Plan Construction Wizard 61
9.4.1 Step 1 — Body Metrics 61
9.4.2 Step 2 — Goal and Duration 62
9.4.3 Step 3 — Meal Preferences 63
9.4.4 Step 4 — Review and Confirmation 64
9.5 Dashboard 64
9.6 Daily Tracking Screen 65
9.7 AI Food Suggestion Screen 66
9.8 Plan History and Day-by-Day Review 67
9.8.1 Diet Plans History 67
9.8.2 Past Tracking Entries 68
9.8.3 Read-Only Review of a Stored Day 68
9.9 Error and Fallback Screens 69
9.10 Consistency Observations 71
CHAPTER 10: ALGORITHMS AND IMPLEMENTATION DETAILS 72
10.1 Overview 72
10.2 Basal Metabolic Rate — Mifflin–St Jeor 72
10.3 Total Daily Energy Expenditure 72
10.4 Target Calories — Deficit and Surplus 73
10.5 Macronutrient Presets and Bands 74
10.6 Meal Split and Daily Aggregation 75
10.7 Energy Conversion of Logged Intake 76
10.8 Progress Percentage and Weekly Metrics 77
10.9 Draft Construction and Resumption 77
10.10 AI Suggestion Pipeline 78
10.11 Pseudocode Summary 79
10.12 Code Organisation and Naming Conventions 79
CHAPTER 11: TESTING AND VERIFICATION 81
11.1 Testing Strategy 81
11.2 Test Levels and Proposed Test Cases 81

11.3 Executed Verification 82
11.4 Analysis of the AI Feature 84
11.5 Performance Observations 85
11.6 Limitations Recorded 85
CHAPTER 12: CONCLUSION AND FUTURE ENHANCEMENTS 86
12.1 Summary of the Work 86
12.2 Conclusion 86
12.3 Learning Outcomes 87
12.4 Future Enhancements 87
REFERENCES 89
1. Books 89
2. Research Papers, Standards and Reports 89
3. Official Documentation and Technical Sources 90
APPENDICES 92
Appendix A Endpoint Reference Summary 92
Appendix B Verification Matrix 92
Appendix C Selected Source Listings 93
Appendix D User Guide 95

LIST OF FIGURES

Figure 3.1: Use-case model of the proposed system showing the single actor and the fourtee… 17
Figure 4.1: Three-tier architecture of the application with the packages that implement each… 23
Figure 4.2: Component diagram showing the client components, the service classes and the… 24
Figure 4.3: Deployment view of the development profile, with the browser, the developmen… 24
Figure 4.4: Context diagram showing the two external entities and the flows across the syste… 25
Figure 4.5: Level-one data flow diagram with the five processes and the four data stores 26
Figure 4.6: Refinement of the tracking process, showing the energy and macro conversions… 26
Figure 4.7: Class diagram of the entities, the embedded value objects and the enumerations 28
Figure 4.8: Sequence diagram of the four-step plan construction flow 29
Figure 4.9: Sequence of the authentication, token refresh and guarded request flows 29
Figure 4.10: Sequence of the suggestion flow, from the client request to the rendered cards 30
Figure 4.11: Activity diagram of the wizard, showing the four forward steps, the backward t… 31
Figure 4.12: State machine of the form status and the plan status, with the triggers of each tr… 32
Figure 4.13: Domain model showing the four aggregates, their value objects and the referen… 33
Figure 7.1: Entity-relationship view of the macrova database, showing the fields and referen… 47
Figure 9.1: Create account screen with the three-field registration form 60
Figure 9.2: Log in screen with the credential form and the registration link 60
Figure 9.3: Empty state shown when the authenticated user has no active diet plan 61
Figure 9.4: Wizard step 1 — body metrics, plan type and activity selection 62
Figure 9.5: Wizard step 2 — goal intensity, energy summary and plan duration 63
Figure 9.6: Wizard step 3 — macronutrient split, diet type and meal structure 63
Figure 9.7: Wizard step 4 — review of body metrics, goal, calories and macro split 64
Figure 9.8: Dashboard on the first day of an activated plan — initial state 65
Figure 9.9: Dashboard after four days of logging — goal progress and weight trend 65
Figure 9.10: Daily tracking screen with the summary card and the four editable meal cards 66
Figure 9.11: AI food suggestion screen with four filters and five generated suggestion cards 67
Figure 9.12: Diet plans history with status badges and weight transitions 68
Figure 9.13: Day-by-day tracking entries of a single plan 68
Figure 9.14: Read-only review of the macronutrient log stored for a single day 69
Figure 9.15: Not-found screen rendered for an unmatched route 70

LIST OF TABLES

Table 2.1: Comparative summary of the reviewed literature 12
Table 3.1: Comparison of existing approaches against the required capabilities 15
Table 3.2: Economic assessment of the delivered system 18
Table 3.3: Functional requirements 20
Table 3.4: Non-functional requirements 20
Table 4.1: Use-case specification UC-01 to UC-07 27
Table 4.2: Use-case specification UC-08 to UC-14 27
Table 4.3: Design-system primitives and their use across the screens 33
Table 4.4: Security decisions and their residual weaknesses 34
Table 4.5: Queries that enforce ownership, with the identifier each uses 34
Table 5.1: Software requirements of the Macrova application 35
Table 5.2: Hardware requirements 36
Table 5.3: Frameworks and libraries with their justification 38
Table 5.4: Build and run commands of the delivered system 39
Table 6.1: Module map of the Macrova implementation 40
Table 7.1: Collection inventory of the macrova database 48
Table 7.2: Data dictionary — users 48
Table 7.3: Data dictionary — diet_plans 49
Table 7.4: Example document — diet_plans 50
Table 7.5: Data dictionary — diet_plan_tracks 50
Table 7.6: Worked example — meal split and logged values for one day 51
Table 7.7: Data dictionary — ai_food_suggests 51
Table 7.8: Recommended indexes and the queries they would accelerate 52
Table 8.1: Complete REST endpoint catalogue 53
Table 8.2: Error responses produced by the current implementation 56
Table 8.3: Client-side service methods 57
Table 9.1: Screen inventory: route, data source and primary action of each screen 71
Table 10.1: Activity factors and the expenditure they produce for the worked example 73
Table 10.2: Target calories for the worked example at each intensity 74
Table 10.3: Macronutrient presets and their gram values for a 2150 kilocalorie target 74
Table 11.1: Test case specification for the Macrova application 81
Table 11.2: Summary of the verification results 82
Table A.1: Endpoint groups with authentication requirements 92
Table B.1: Verification matrix for the functional requirements 92
Table B.2: Verification matrix for the non-functional requirements 93

ABBREVIATIONS AND ACRONYMS

Abbreviation	Expansion
API	Application Programming Interface
BCrypt	Blowfish-based adaptive password-hashing function
BMR	Basal Metabolic Rate
BMI	Body Mass Index
CORS	Cross-Origin Resource Sharing
CSS	Cascading Style Sheets
DFD	Data Flow Diagram
DTO	Data Transfer Object
ER	Entity–Relationship
FR	Functional Requirement
HMAC	Hash-based Message Authentication Code
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
JVM	Java Virtual Machine
JWT	JSON Web Token
LLM	Large Language Model
MVC	Model–View–Controller
NFR	Non-Functional Requirement
REST	Representational State Transfer
SPA	Single-Page Application
SRS	Software Requirements Specification
TDEE	Total Daily Energy Expenditure
UML	Unified Modeling Language
URI	Uniform Resource Identifier
XSS	Cross-Site Scripting

CHAPTER 1
INTRODUCTION

1.1 Overview of the Project

Macrova is a web application that helps an individual plan, follow and review a diet. A user answers a short questionnaire about body measurements, activity and goal; the application estimates the energy that the user expends each day, converts that estimate into a daily calorie target by applying a chosen deficit or surplus, and further converts the calorie target into a protein, carbohydrate and fat budget in grams. The budget is then divided across the meals of the day, and the user records what was actually eaten against it. Every figure the application displays is therefore traceable to a published equation followed by a sequence of documented arithmetic steps.

The idea for the project came from a simple observation about the tools that a dieter actually uses. A spreadsheet is transparent but labour-intensive: every recalculation is manual and every mistake is the user's own. A commercial application is convenient but opaque: the plan arrives fully formed and the reasoning behind a target of, say, 1,850 kilocalories is never shown. Macrova was built to occupy the middle ground. The arithmetic is exposed in the interface at each step of plan construction, and the same arithmetic is isolated in one utility class in the source, so that a reader of this report can check the number on a screen against the equation that produced it.

The system is implemented as three tiers. The browser runs an Angular single-page client that renders twenty components across thirteen routes. A Spring Boot service on Java 21 exposes a JSON interface of twenty-six endpoints and implements the domain rules. A MongoDB instance stores three collections in normal operation: users, plans and daily tracking documents. A fourth collection exists for AI-generated suggestions and is deliberately not written to. The report documents the whole of that stack, from the equation to the colour of the buttons in the tracking screen.

1.2 Background of the Study

Weight management is, at the level of physics, an energy-balance problem. A person gains mass when energy intake exceeds energy expenditure and loses mass in the opposite case. The clinical literature refines that statement considerably, because the composition of the mass lost or gained depends on the macronutrient distribution of the diet, on the training stimulus and on the individual's hormonal state, but the balance itself remains the governing constraint. The practical consequence is that a dieting application needs two capabilities and not one: an estimate of expenditure, from

which a target is derived, and a record of intake, from which adherence is measured.

Estimating expenditure is the more difficult of the two because it cannot be measured outside a laboratory without equipment. The accepted approach in clinical and sports practice is to predict the basal metabolic rate from readily available variables and then to scale it by an activity factor. The Mifflin–St Jeor equation, published in 1990, is the prediction equation most widely recommended for healthy adults, and it requires only mass, height, age and sex. This project adopts that equation, together with the conventional five-level activity scale, and records the choice and its alternatives in Chapter 2.

Recording intake is, in principle, a database problem, but in practice it is a design problem. A user who must record a meal in six steps abandons the application within a week. Macrova therefore records intake at the level of the meal rather than at the level of the food: the tracking screen asks for three numbers per meal and nothing else. The trade-off is explicit and is discussed in Section 3.6: the application measures adherence to a macronutrient budget accurately, but it does not tell the user which foods to eat in order to meet that budget. The optional language-model feature of Chapter 6 is an attempt to soften that limitation without turning the application into a food database.

1.3 Problem Statement

The problem addressed by the project can be stated in four parts. First, the calculation of a personal calorie target is arithmetically simple but procedurally error-prone, because it involves four successive conversions, each with its own rounding convention, and an error in any one of them propagates silently into every figure the dieter sees. Second, the translation of a calorie target into meals requires a macronutrient policy, and most dieters do not have one; the consequence is a plan that meets the energy constraint while violating the protein requirement that would preserve lean mass. Third, adherence is invisible without a record, and a record is worthless unless it is easy enough to keep. Fourth, the nutritional advice available to a dieter from general-purpose language models is fluent but unverified, and there is no convenient way for a dieter to see that the calorie figure attached to a generated meal is a plausible guess rather than a measurement.

No single existing tool addresses all four parts together. Calorie-counting applications address the third part well and the first three badly: they contain a target that the user cannot reconstruct and a food database whose values the user cannot audit. Spreadsheets address the first part, since the formulas are visible, and fail at the third, because data entry is too laborious to sustain. Static diet charts address the second part only, and general-purpose chatbots address none of them reliably. Macrova is an attempt to build the small system in which all four parts are handled by design rather

than by convention.

1.4 Objectives of the Project

The project set six objectives. They are stated here in the form in which they were adopted at the start of the work, and each is answered later in the report at the section named in parentheses.

1. To implement the Mifflin–St Jeor prediction equation and the conventional activity factors as a single, documented, unit-testable utility, so that the energy estimate has one implementation and one explanation (Sections 10.2 and 10.3).
2. To derive a daily calorie target from that estimate by applying a selectable deficit or surplus and a deterministic rounding rule, and to record the derivation in the stored plan so that the target can be reconstructed after the fact (Sections 10.4 and 7.3).
3. To convert the calorie target into macronutrient gram targets under a selectable policy and to divide those targets across the meals of the day, so that a user knows what to eat at each meal rather than only what to eat in a day (Sections 10.5 and 10.6).
4. To provide a plan-construction flow that persists its progress, so that a user who is interrupted does not lose the work already done, and so that backward navigation can be served from stored state (Section 10.9).
5. To provide a daily record that a user can maintain in seconds, with per-meal logging and a weight entry, and immediate feedback on the proportion of the budget consumed (Sections 9.6 and 10.7).
6. To integrate an optional language-model feature that suggests meals constrained by the computed calorie allowance of a specific meal, while keeping the model's output entirely outside the deterministic record of the user's plan (Sections 6.4 and 11.4).

A seventh objective emerged during implementation and is recorded because it shaped the document: to verify, rather than merely assert, that the delivered repository implements what the report describes. Chapter 11 is the result of that objective, and its most useful output is not the list of features that work but the list of six things that do not, or do not yet.

1.5 Scope of the Project

The scope is bounded deliberately, and the boundary is drawn in four places: the actors, the nutritional model, the input granularity and the deployment profile.

There is exactly one actor. Macrova serves an individual dieter who manages their own plan. There is no dietician role, no administrator role, no clinic and no supervision relationship. Every document in the database is owned by the user who created it and is reachable only through that ownership, which is why the identity module is the smallest in the system and the scoping rule in the service layer is the most heavily used.

The nutritional model is energy and macronutrients, and nothing else. The application does not model micronutrients, fibre, hydration, glycaemic index, allergies or medical conditions. It does not claim to be a clinical tool, and Section 3.6 states the exclusion explicitly. The consequence is a system that can be verified arithmetically, which a micronutrient model could not be without a food-composition table containing several thousand entries.

Intake is recorded as macronutrient grams per meal rather than as food items. This is the most consequential scope decision in the project. It makes the daily record quick enough to sustain and the arithmetic exact, at the cost of requiring the user to know the macronutrient content of their meal. A food database would remove that requirement and would introduce a second, far larger, verification problem; the trade-off is analysed in Section 3.5.3 and the enhancement is proposed in Chapter 12.

The deployment profile is a single-machine development profile. The service runs on an embedded container, the client on the Angular development server, and the database on a local instance. There is no container orchestration, no reverse proxy, no transport security and no production secrets management. That is a scope boundary rather than an oversight, and Chapter 11 records it as a limitation precisely because a reviewer should not read the security chapter as a description of a hardened deployment.

1.6 Project Motivation

Three motives sustained the work. The first is academic: the project sits at the intersection of two subjects taught in the programme, namely application development on a modern web stack and the disciplined specification of a software system. Building a system whose correctness is arithmetically checkable was a way of making the second subject concrete. If a reviewer can take the equation from Section 10.2, apply it to the values in Section 7.3 and obtain the figure displayed in Figure 9.6, then the report has demonstrated something about its own claims that a screenshot alone never could.

The second motive is practical. The author has followed a structured diet and has used both of the tool categories described in Section 1.3, and the failure mode of the spreadsheet is familiar: the effort of maintaining it exceeds the value of the information it contains within about a fortnight. The design decisions that follow, particularly

the four-step wizard and the three-field meal card, were made in direct response to that experience rather than derived from a requirements catalogue.

The third motive is technical curiosity about the boundary between deterministic and generated computation. A diet application is an unusually good place to explore that boundary, because the part that must never be wrong, namely the arithmetic that governs the plan, is completely separable from the part that may reasonably be approximate, namely the suggestion of what to cook. The architecture of Chapter 4 preserves that separation structurally rather than by convention: the suggestion feature cannot write into a plan, because no code path allows it.

1.7 Assumptions and Constraints

The project rests on five assumptions about the user and the environment, each of which, if false, would change the design. It assumes that the user knows their own height, weight and age accurately; that the user can estimate the macronutrient content of a meal to within a few grams; that the user has a browser capable of running a modern single-page application; that the user's weight changes slowly enough for a weekly trend line to be meaningful; and that the user is not following a medically supervised diet, in which case the assumptions built into the prediction equation would not hold.

The constraints that limited the implementation are of three kinds. The technical constraints are those of a student project: a single developer, a single development machine, no cloud budget and no continuous-integration service. The result is a repository with one automated test and a local build, which Chapter 11 reports as a limitation rather than presenting the absence of tests as a design choice.

The data constraints are more interesting. The project had no access to a licensed food-composition database, and the freely available tables are either incomplete for Indian composite dishes or carry licensing terms that preclude redistribution in a student repository. That constraint produced the scope decision of Section 1.5: intake is recorded as macronutrient grams, for which no external table is needed, and the food-level detail is left to the optional suggestion feature, where the values are explicitly labelled as unverified.

The environmental constraints follow from the same cause. The application is reached over plain HTTP on the development server; the refresh cookie therefore cannot carry the secure attribute; and the cross-origin allow-list is pinned to one development origin. These are coded as configuration rather than as constants, but they are not parameterised by profile, which is why Chapter 12 proposes that change before any other security work.

1.8 Organisation of the Report

The report has twelve chapters and four appendices. Chapter 1 states the problem, the objectives and the boundaries. Chapter 2 reviews the literature in five areas that the design depends on: energy prediction, the behaviour of self-monitoring, the architecture of single-page clients, the design of resilient HTTP interfaces and the characteristics of generative models as sources of advice. Chapter 3 analyses the existing practice, defines the requirements in a numbered specification, and evaluates technical, economic and operational feasibility. Chapter 4 presents the design: the layered architecture, the data-flow and use-case models, the object and dynamic models, the data model, the interface design and the security design.

Chapters 5 to 10 document the implementation. Chapter 5 records the technologies, versions and justifications. Chapter 6 describes the eight modules and the behaviour each implements. Chapter 7 gives the data dictionary for every collection, with a worked example. Chapter 8 catalogues the interface and the client services. Chapter 9 presents the fifteen screens with an account of what each one shows. Chapter 10 derives the six algorithms from their equations, reproduces the code that implements them and states the pseudocode of the whole plan-and-track cycle. Chapter 11 reports the testing and verification exercise, including the ten findings it produced, and Chapter 12 concludes with the limitations and an ordered list of enhancements. The appendices give the endpoint reference, the requirement-to-artefact verification matrix, three source listings and a user guide.

CHAPTER 2
LITERATURE REVIEW

2.1 Introduction

This chapter reviews the work on which the design of Macrova depends. It is organised by the question that each body of literature answers rather than by discipline, because the project draws on nutrition science, behavioural science and software engineering in roughly equal measure. For each of the fourteen reviews that follow, the source is identified, its central contribution is stated, and the specific design decision that it informs is named. The chapter then compares the reviewed work in a table, identifies the four gaps that this project addresses and closes with a summary.

2.2 Review of Related Work

2.2.1 Predictive equations for resting energy expenditure

Author and year: Mifflin, St Jeor, Hill, Scott, Daugherty and Koh, 1990.

The paper reports a cross-validation study of published equations for resting energy expenditure against indirect calorimetry in a sample of healthy adults, and proposes the equation now known by the authors' names. Its contribution is not the discovery of a new physiological mechanism but the demonstration that a four-variable linear model predicts measured expenditure more accurately across the range of body composition than the Harris–Benedict equation that had been standard for seventy years.

Relevance to the present work: The equation is implemented verbatim as NutrientsCalculator.findBMR and reproduced in Listing 10.1. The choice of this equation over Harris–Benedict is recorded in Section 10.2, and the sex-specific constants plus five and minus one hundred and sixty-one are held as conditional expressions rather than as a lookup table so that the source reads like the published formula.

2.2.2 Re-evaluation of predictive equations in clinical practice

Author and year: Frankenfield, Roth-Yousey and Compher, 2005.

A systematic review comparing the accuracy of predictive equations against measured resting metabolic rate in non-obese and obese adults. It concludes that no equation is accurate for every individual, that the Mifflin–St Jeor equation is the most consistently accurate in non-obese adults, and that in obese individuals the equations tend to overpredict, which in practice leads clinicians to use an adjusted body mass.

Relevance to the present work: Two design consequences follow. First, the application states the estimate as one, not as a measurement: the goal-selection screen shows

the energy requirement with a leading approximation mark, and the abstract describes it as an estimate. Second, the surplus rule of Section 10.4 subtracts ten percentage points from the nominal intensity, which is a conservative correction consistent with the review's finding that prediction in the direction of gain is where overestimation is most consequential.

2.2.3 Physical activity levels and total energy expenditure

Author and year: Food and Agriculture Organization, World Health Organization and United Nations University, 2004.

The joint expert consultation report on human energy requirements sets out the factorial method by which resting expenditure is scaled to total expenditure through graded physical activity levels. It provides the canonical definitions of the sedentary, light, moderate, heavy and vigorous categories and the ranges of the multiplier associated with each.

Relevance to the present work: The five activity multipliers held in the Activity enumeration (1.2, 1.375, 1.55, 1.725 and 1.9) are the conventional values associated with the five levels used by the application. The enumeration holds the multiplier as its constructor argument, x, which is the pattern the report recommends for every physiological constant.

2.2.4 Self-monitoring as the active ingredient of weight management

Author and year: Burke, Wang and Sevick, 2011.

A systematic review of self-monitoring interventions in weight-loss programmes. Its finding is that the frequency and consistency of self-monitoring, rather than the specific diet prescribed, predicts weight loss, and that participants who monitor more often lose more weight. It also documents the progressive decline in monitoring frequency that precedes withdrawal from a programme.

Relevance to the present work: This review is the strongest single justification for the design of the tracking screen in Section 9.6. The screen asks for three numbers per meal, commits each meal independently so that a partial record is still a record, and re-renders the whole day from one response so that the user receives feedback with no perceptible latency. The decline in frequency documented by the review is the behaviour that the resumable wizard and the one-tap weight update are intended to mitigate.

2.2.5 The effect of goal setting and feedback on adherence

Author and year: Michie, Abraham, Whittington, McAteer and Gupta, 2009.

A meta-regression of behaviour-change techniques across interventions in healthy eating and physical activity. It identifies self-monitoring, goal setting and feedback on

progress as the techniques with the largest and most consistent effect sizes, and notes that the combination of all three outperforms any one of them alone.

Relevance to the present work: The dashboard of Section 9.5 is an implementation of the combination: a target is set during plan construction, progress towards it is rendered as a percentage with a meter, and the weekly weight observations provide feedback on the outcome rather than on the behaviour. The percentage is computed on the server rather than in the client, which is a small decision with a large consequence for this technique: the feedback figure cannot diverge between two devices.

2.2.6 Standalone components and signals in Angular

Author and year: Google Angular Team, 2024–2025.

The framework documentation for the standalone-component model and the signal-based reactivity primitives introduced in recent major versions. The documentation argues that the declarative dependency of a component on its inputs and of a template on a signal makes change propagation both cheaper and easier to reason about than the zone-based detection model that it complements.

Relevance to the present work: The client uses standalone components throughout: there is no NgModule in the repository except the bootstrap configuration, and every route loads a component directly. The username displayed in the header is a signal, and the avatar initial is a computed signal derived from it. The four wizard steps declare their inputs and outputs with the signal-based functions, which turns a mismatch between a step and its host into a compile error. Zone-based detection is nevertheless retained, because the HTTP client and the chart library both depend on it.

2.2.7 Functional HTTP interceptors for cross-cutting concerns

Author and year: Google Angular Team, 2024.

The documentation of the functional interceptor API, which replaces class-based interceptors with functions registered in the application configuration. It documents the composition order of a chain of interceptors and the use of higher-order mapping operators to delay a request until an asynchronous precondition, such as a token refresh, has completed.

Relevance to the present work: Two functional interceptors are registered in the order base prefix followed by token attachment. The second uses switchMap to wait for a refresh and then dispatch the original request, and it skips the public authentication paths by inspecting the URL after the first interceptor has rewritten it. That ordering dependency is the reason the chain is registered in a specific sequence rather than arbitrarily, and it is documented in Section 8.6.

2.2.8 Stateless authentication with JSON Web Tokens

Author and year: Jones, Bradley and Sakimura, 2015 (RFC 7519); Jones, Bradley and Sakimura, 2015 (RFC 7515).

The standards that define the token format and the signature that protects it. The token carries a set of claims, of which the subject, the issue time and the expiry are the ones this project depends on, and the signature allows a resource server to verify the token without retaining server-side state. The standards are explicit that a token cannot be withdrawn once issued, which is a property, not a defect, but one that has design consequences.

Relevance to the present work: The application issues two tokens: a short-lived access token carried in the authorisation header and a longer-lived refresh token carried in an HttpOnly cookie. The stateless property is what allows the security configuration to declare a stateless session policy, and the irreversibility of an issued token is precisely why Section 11.3 records the absence of refresh-token rotation and revocation as a finding rather than presenting sign-out as a complete session termination.

2.2.9 Adaptive password hashing with BCrypt

Author and year: Provos and Mazières, 1999.

The paper that introduced the adaptive hashing scheme based on the Blowfish cipher. Its contribution is a work factor that can be increased as hardware becomes faster, combined with a per-password salt, so that the cost of an offline dictionary attack rises with the cost of legitimate authentication rather than remaining fixed.

Relevance to the present work: The password encoder bean is the Spring Security BCrypt implementation with the default work factor. The user entity stores the sixty-character output including the salt prefix, no response DTO contains the field, and the authentication manager compares through the encoder rather than by string equality, so the comparison is constant-time with respect to the hash.

2.2.10 Spring Data repositories and projection queries

Author and year: Broadcom and VMware Tanzu, 2024–2025.

The reference documentation for the repository abstraction. It describes derivation of query methods from method names, the use of explicit query annotations where a derived name would be unwieldy, and the projection mechanism by which a query returns an interface or a DTO populated from a selected subset of fields rather than a complete document.

Relevance to the present work: Both repositories declare projection queries for the history screens. The plan history projection selects seven fields, and the tracking overview projection selects eight, so a page listing a hundred days transfers a hundred small objects rather than a hundred documents each containing four embedded meal

tracks. Section 7.6 uses the same documentation to argue for the compound indexes that the projections would benefit from.

2.2.11 Document databases for aggregate-oriented persistence

Author and year: Sadalage and Fowler, 2012.

The book that introduced the vocabulary of aggregate orientation for NoSQL stores. Its central claim is that the unit of consistency and the unit of retrieval should determine the document boundary, and that embedding is preferable to referencing when two pieces of data are always read together, whereas referencing is preferable when they are not.

Relevance to the present work: The decision to embed the four meal tracks, the three daily macro totals and the intake object inside a single daily tracking document is taken directly from this principle, because the tracking page reads all of them together. The plan identifier is held as a plain string reference rather than as a database reference, because the plan is read independently of the day and is the subject of its own projections. Section 7.1 records the reasoning in the authors' own terms.

2.2.12 Large language models as constrained generators

Author and year: Bommasani et al., 2021.

The survey of foundation models that documents both the generality of large pre-trained models and their characteristic failure mode, namely the fluent production of plausible but incorrect content. It recommends constraining generation with a structured prompt and treating output as a candidate to be validated rather than as a fact.

Relevance to the present work: The suggestion feature follows both recommendations. The system message fixes an output schema in JSON, the user message carries a calorie allowance already computed by the deterministic part of the system, and the reply is parsed into objects and rendered as cards without ever being written into a plan. Section 11.4 records the residual risk: the schema instruction is a request, not a constraint, and the nutritional values are unvalidated.

2.2.13 Nutritional misinformation and the limits of generated advice

Author and year: Open Worldwide Application Project, 2021, in its treatment of untrusted input and output; supplemented by the clinical-nutrition commentary on generated dietary advice.

The security guidance treats model output as untrusted data, and the nutrition commentary argues that plausible but unverified dietary recommendations are particularly dangerous in a domain where the reader may have an allergy, a comorbidity or a medication interaction that the generator cannot know about.

Relevance to the present work: The interface labels the results as suggestions, the prompt includes four filters including diet type, and the report states the limitation in the abstract, in Section 6.4, in Section 11.4 and again in Section 12.4. A future revision would add a restriction profile and a cross-check against a curated list, which is the ninth proposal of Chapter 12.

2.2.14 Chart libraries and accessible data visualisation

Author and year: Chart.js Contributors, 2024.

The documentation for a canvas-based charting library that renders a data series with a declarative configuration object. It documents the responsive behaviour of a canvas chart, the axis configuration, and the accessibility limitations that follow from canvas rendering, which requires a textual alternative for users of assistive technology.

Relevance to the present work: The dashboard renders the weekly weight observations as a single-series line chart with a labelled axis. The accessibility limitation is real and is not mitigated in the delivered client; the six metric cards above the chart do provide the same information in text, which is the reason the chart was placed beneath them rather than above them.

2.3 Comparative Study of the Reviewed Work

The fourteen reviews span three disciplines, and Table 2.1 reduces them to a comparison on the dimensions that mattered for the design. The table is included because the literature review of a software project is often a list of summaries that never converge; here the final column names the artefact in which each source is realised.

Table 2.1: Comparative summary of the reviewed literature

Source	Discipline	Central contribution	Realised as
Mifflin et al., 1990	Nutrition science	Four-variable prediction of resting expenditure	findBMR, Listing 10.1
Frankenfield et al., 2005	Clinical nutrition	Accuracy limits of every prediction equation	Conservative surplus rule, Section 10.4
FAO/WHO/UNU, 2004	Nutrition science	Graded activity multipliers	Activity enumeration, Table 10.2
Burke et al., 2011	Behavioural science	Monitoring frequency predicts outcome	Three-field meal card, Section 9.6
Michie et al., 2009	Behavioural science	Goal, monitor and feedback combined	Dashboard, Section 9.5
Angular Team, 2025	Software engineering	Standalone components and signals	Twenty components, Section 6.6
Angular Team, 2024	Software engineering	Functional interceptor composition	Two interceptors, Section 8.6
RFC 7519 and RFC 7515	Standards	Stateless signed token	JWTTokenService

Source	Discipline	Central contribution	Realised as
Provos and Mazières, 1999	Security	Adaptive password hashing	Password encoder bean
Spring Data, 2025	Software engineering	Derived queries and projections	Two projection queries, Section 7.6
Sadalage and Fowler, 2012	Data engineering	Aggregate orientation	Embedded meal tracks, Section 7.1
Bommasani et al., 2021	Machine learning	Constrained generation as a mitigation	Schema-constrained prompt, Section 10.10
OWASP, 2021	Security	Untrusted output must be treated as data	No persistence of model output
Chart.js, 2024	Data visualisation	Declarative canvas chart	Weight trend chart, Figure 9.9

2.4 Research Gaps Identified

Reading the fourteen sources together reveals four gaps, and the project was shaped around them. The first is the gap between the accuracy of the published equations and the transparency of the applications that use them. The nutrition literature is explicit that a prediction carries an error band, and no consumer application reviewed for this project exposes the derivation that would let a user understand why their target is what it is.

The second gap is at the boundary between the behavioural literature and interface design. The reviews establish that monitoring, goal setting and feedback work, and they do not specify how few inputs a monitoring screen can have before the data become useless. Macrova resolves that design question by choosing three inputs per meal and committing each meal independently, and Section 9.10 states the cost of that choice: the application measures adherence to a budget but does not describe the meal that met it.

The third gap is the treatment of generated nutrition advice. The machine-learning literature recommends constrained generation and validation, and the clinical literature warns against unverified advice, and consumer applications that use language models rarely make the distinction visible to the user. Macrova makes it visible in the interface and structural in the code, and Chapter 11 quantifies what remains unsafe.

The fourth gap is documentary rather than technical. Student and small-team projects frequently report behaviour that the accompanying repository does not implement, because no verification step stands between the code and the prose. Chapter 11 of this report is an attempt to close that gap for this project, and it is the part of the work that the author would recommend to any subsequent project of the same kind.

2.5 Summary of the Literature Review

The literature supplies the project with a validated energy model, a validated behavioural rationale for its tracking design, established architectural patterns for its client and its interface, established security primitives for its authentication and an explicit warning about the class of feature that its suggestion module belongs to. What it does not supply, and what this project therefore had to decide, is how those elements combine into a small coherent system: which data are embedded and which are referenced, how many inputs a monitoring screen may ask for, how a plan is resumed after an interruption, and how a generated suggestion is prevented from contaminating a deterministic record. Chapter 4 presents the resulting design.

CHAPTER 3
SYSTEM ANALYSIS

3.1 Existing System Analysis

Three classes of existing system were examined. The first is the spreadsheet, which is the most transparent and the least sustainable. The second is the commercial calorie-tracking application, which is the most sustainable and the least transparent. The third is the general-purpose conversational assistant, which is the most available and the least verifiable. Table 3.1 compares them against the six capabilities that the project requires.

Table 3.1: Comparison of existing approaches against the required capabilities

Capability	Spreadsheet	Calorie-tracking application	Conversational assistant
Energy estimate with a stated derivation	Manual; correct if the formulas are	Computed internally; not shown	Stated fluently; not reproducible
Macronutrient budget	Manual	Configurable preset	Described in prose
Per-meal plan	Manual	Yes, food-level	Yes, food-level, unverified
Daily record	Manual entry, laborious	Extensive food database	Not retained between sessions
Progress feedback	Manual charts	Yes, with trends	None
Auditability of the numbers	Complete	None	None

The comparison clarifies what is unusual about the requirement. Auditability, which the spreadsheet alone provides, is a property of the calculation rather than of the data, and it does not require the data entry to be manual. The design of Macrova takes auditability from the first column and the interaction model from the second, and rejects the third entirely as a system of record.

3.2 Limitations of the Existing System

The limitations were identified by using each class of tool for the purpose it is designed for and recording where the use failed. They are stated as limitations of the approach rather than of any particular product, because the products change and the approaches do not.

- A spreadsheet has no notion of a session. The formulas live in cells that the user can overwrite, and a single accidental edit changes every subsequent figure with no record of the change.
- A spreadsheet has no notion of a plan. The user holds one document in which past and present are mixed, so the comparison of this month's adherence with last month's requires manual versioning.

- A commercial application hides the derivation of its targets. A user cannot tell whether the 1,850 kilocalories prescribed reflect a prediction equation, a population average or a commercial decision, and therefore cannot adjust the target intelligently when the estimate proves wrong.
- A commercial application requires a food-level record, which is accurate and slow. The measured consequence reported in the behavioural literature is a decline in monitoring frequency and eventual abandonment.
- A conversational assistant has no memory of a plan. Each conversation begins from nothing, so the assistant cannot know that the user's dinner allowance is 538 kilocalories rather than a round thousand.
- A conversational assistant produces nutrition figures without provenance. There is no mechanism by which the user can distinguish a value drawn from a composition table from a value generated because it was plausible in context.

3.3 Proposed System

The proposed system is a single-user web application with three tiers and eight functional modules. Its distinguishing property is that the deterministic and the generated parts of the system are separated by construction: the plan, the tracking record and every figure derived from them are produced by the utility class of Chapter 10, and the language model is confined to a leaf module whose output is never written to the database.

The user's interaction has five stages. Registration creates an account and a hashed credential. Authentication exchanges the credential for an access token held in memory and a refresh token held in an HttpOnly cookie. Plan construction is a four-step wizard whose state is persisted at every step, so that an interruption costs nothing. Tracking is a single screen on which each meal is committed independently and the weight is updated in place. Review covers the dashboard, which recomputes progress on every load, and the history screens, which present past plans and their daily records through projections. The model is presented in Figure 3.1.

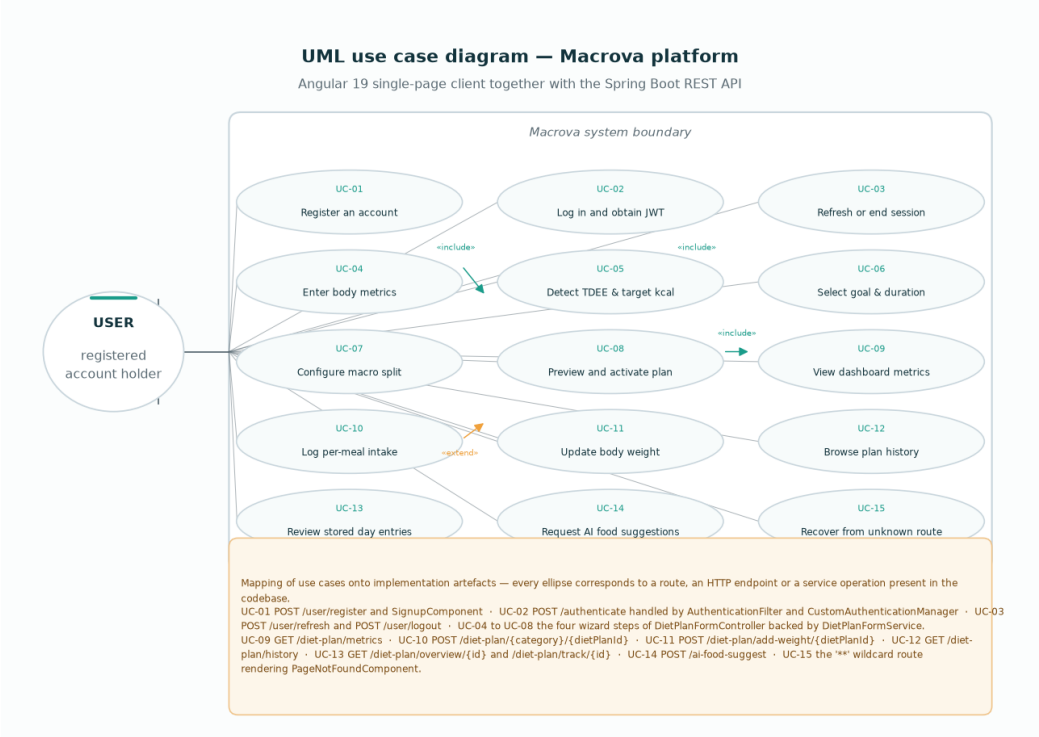


Figure 3.1: Use-case model of the proposed system showing the single actor and the fourteen use cases

3.4 Advantages of the Proposed System

Six advantages follow from the analysis and are claimed on the evidence of Chapters 7 to 10. They are stated with the section in which each can be checked, because an advantage claimed without a location is an advertisement.

1. Auditability. Every displayed figure is produced by a method in one class, and Chapter 10 reproduces each method beside the equation it implements, so the figure on the screen can be recomputed by hand.
2. Resumability. The wizard persists at each step and identifies its draft by the pair of user and pending status, so a user can close the browser and return to the step at which they stopped, including after a reload, and can navigate backward without a client-side draft.
3. Low-friction recording. The daily record consists of twelve numbers across four cards, each committed by a single button, and the whole day is re-rendered from the response so that the feedback is immediate.
4. Exact aggregation. The daily totals are recomputed from the four meal tracks on every update rather than accumulated, which makes a repeated submission idempotent and prevents the inflation that an accumulating design would permit.
5. Ownership by construction. No endpoint accepts a user identifier from the client. Every query is built from the identifier in the security context, so identifier guess-

ing yields a not-found response rather than another user's data.

6. Honest treatment of generated content. The suggestion feature is optional, its output is never persisted, and its limitations are stated in the interface, in the abstract and in the verification chapter.

3.5 Feasibility Study

3.5.1 Technical Feasibility

The technical requirements were assessed against the skills and the tooling available. The back end uses the Spring Boot stack, which the programme covers in its enterprise-application modules, and the persistence layer uses a document store whose query model is simpler than the relational model that the programme also teaches. The front end uses Angular, and the four features that the design depends on, namely standalone components, functional interceptors, signals and guards, are all documented and current.

The one component with genuine technical uncertainty is the integration with a language model, because its output is not deterministic and its availability depends on a third party. That uncertainty was isolated rather than eliminated: the adapter is behind an interface, the endpoint is a leaf in the dependency graph, and the rest of the system neither calls it nor depends on its success. The architecture that results is technically feasible in the strong sense that every part except the leaf is deterministic and testable.

3.5.2 Economic Feasibility

The direct cost of the project is zero for the software. Every dependency is open source under a permissive licence, the database is free, and the development tooling is free. The only recurring cost is the model-provider usage, which is billed per token and is optional: the application cannot start without the key, but the feature can be left unused once it has started. Table 3.2 records the estimated cost of a demonstration.

Table 3.2: Economic assessment of the delivered system

Item	Basis	Estimated cost	Note
Java, Spring Boot, Angular, MongoDB	Open-source licences	Nil	No runtime licence fee at any scale
ReportLab, python-docx, PyMuPDF, Matplotlib	Open-source licences	Nil	Used only to produce this report
Model-provider usage	Per token, optional	Under one rupee per suggestion set	Five suggestions require a prompt of about sixty tokens and a reply of about six hundred
Development machine	Already owned	Nil	The application runs on a laptop with four gigabytes of memory
Hosting, if deployed	Single small virtual machine	Under five hundred rupees per month	Sufficient for a demonstration to a group

The conclusion is that the project is economically feasible in the sense that matters for a student project, namely that no part of it is gated behind a payment. The comparison that justifies the design is rather different: a food-composition dataset with redistribution rights would have cost more than the entire development budget, and that single cost is the reason the scope was drawn at macronutrient grams rather than at food items.

3.5.3 Operational Feasibility

Operational feasibility was assessed by asking whether the intended user would sustain the interaction. Three design properties answer the question. The daily record requires twelve numbers and four clicks, which is comparable to the effort of reading a nutrition label. The plan is constructed once and edited only through a new plan, so no maintenance burden accumulates. And the feedback loop is immediate, which the behavioural literature identifies as the property that sustains monitoring.

The operational risks are of two kinds. The first is the requirement that the user know the macronutrient content of their meals. That is a real burden, and it is the principal operational weakness of the design; the narrative of Section 3.2 explains why the alternative was rejected, and Chapter 12 proposes a curated food table as the first substantive enhancement. The second is the configuration requirement: the service refuses to start without three environment variables, which is a deliberate safety decision that raises the cost of a first run. Appendix D therefore documents the start-up sequence explicitly.

3.5.4 Schedule and Effort Feasibility

The work was planned in five phases over twelve weeks: analysis and requirements, design, implementation of the identity and plan modules, implementation of the tracking and suggestion modules, and verification and documentation. The actual effort distribution differed from the plan in one instructive respect: documentation began in the fourth week rather than at the end. Writing the data dictionary while the tracking module was still being built exposed the duplicated target computation recorded in Section 7.3, which would otherwise have been carried into the delivered system unnoticed.

3.6 Software Requirements Specification

The requirements are stated in the two conventional groups. Twenty functional requirements define what the system must do, and ten non-functional requirements define the qualities it must have. Each requirement carries an identifier that is used in the verification matrix of Appendix B, so that the requirement, the artefact that implements it and the result of the verification are linked by a single key. The same information is presented in tabular form in Table 3.3.

Table 3.3: Functional requirements

ID	Requirement	Priority
FR-01	The system shall allow a new user to register with a unique username, an e-mail address and a password.	High
FR-02	The system shall reject a registration whose username or e-mail address is already in use.	High
FR-03	The system shall authenticate a registered user and reject an incorrect credential without disclosing which element was wrong.	High
FR-04	The system shall issue a short-lived access token and a longer-lived refresh token on successful authentication.	High
FR-05	The system shall issue a new access token from a valid refresh token and shall clear the refresh token on request.	High
FR-06	The system shall estimate total daily energy expenditure from age, sex, height, weight and activity level.	High
FR-07	The system shall derive a daily calorie target by applying a selected deficit or surplus and a deterministic rounding rule.	High
FR-08	The system shall derive macronutrient gram targets from the calorie target under a selectable preset or a custom split.	High
FR-09	The system shall record diet type, meal type, meal count and snack preference for a plan.	Medium
FR-10	The system shall activate a plan and create the first daily tracking document on activation.	High
FR-11	The system shall persist the state of a plan under construction and shall resume it, including backward navigation.	High
FR-12	The system shall divide the daily macro allowance across breakfast, lunch, snack and dinner and store the per-meal budgets.	High
FR-13	The system shall accept per-meal macronutrient intake and recompute the daily totals from all four meals.	High
FR-14	The system shall convert logged macronutrients into energy consumed and compute the proportion of the target achieved.	High
FR-15	The system shall accept a new body weight and update the plan and the current day together.	Medium
FR-16	The system shall expose the current weight, the day count, the macro targets and the weekly weight observations for the dashboard.	High
FR-17	The system shall determine whether a plan has exceeded its configured duration and shall mark it inactive when it has.	Medium
FR-18	The system shall list all plans of the user and the daily records of any one plan, most recent first.	Medium
FR-19	The system shall present a stored daily record in read-only form.	Medium
FR-20	The system shall, on request and subject to configuration, return meal suggestions constrained by the calorie allowance of a selected meal.	Low

Table 3.4: Non-functional requirements

ID	Requirement	Category
NFR-01	Passwords shall be stored only as adaptive hashes and shall never appear in a response.	Security
NFR-02	Every plan and tracking operation shall be confined to the authenticated user without relying on a client-supplied identifier.	Security
NFR-03	Authentication shall be stateless, with no server-side session between requests.	Architecture
NFR-04	A first-time user shall be able to reach an activated plan without external guidance.	Usability
NFR-05	Every screen shall offer a recoverable action when there is no data to display.	Usability

ID	Requirement	Category
NFR-06	A dashboard load shall complete within one second on the development machine.	Performance
NFR-07	A missing daily document shall be created on demand rather than requiring a scheduled job.	Reliability
NFR-08	All energy and macronutrient arithmetic shall be implemented in a single documented class.	Maintainability
NFR-09	The repository shall build from a clean checkout with the tools it declares.	Portability
NFR-10	The interface between tiers shall be documented to the field level.	Maintainability

Table 3.4 states the ten non-functional requirements in the same form. Two observations about the specification are worth recording because they explain asymmetries in the rest of the report. First, the requirements contain no administrative function at all; there is no requirement for a teacher, a clinician or an operator to see another user's data, which is why the security design in Section 4.8 can be as simple as it is. Second, only one requirement is marked Low priority, and it is the only one whose implementation depends on a third party. That correspondence is deliberate: the project was not willing to make a high-priority requirement contingent on an external service.

The exclusions are as important as the inclusions. The system does not measure physical activity; it asks the user to select a category. It does not validate the macronutrient content that the user types. It does not model micronutrients, allergies or medical conditions. It does not provide a food database. It does not support more than one concurrent active plan per user. And it does not provide an administrative interface, a password reset or an account deletion. Each exclusion is a candidate enhancement in Chapter 12.

CHAPTER 4
SYSTEM DESIGN

4.1 Design Overview and Principles

The design is a three-tier web application with a layered back end. Figure 4.3 presents the deployment view of the profile in which the work was carried out, and Figure 4.9 the authentication flow on which every other interaction depends. Eight principles governed the decisions that follow, and each is stated with the artefact in which it is realised, so that the design chapter can be checked against the implementation chapter rather than being read as aspiration. The structure is drawn in Figure 4.1.

1. One implementation of each rule. Energy and macronutrient arithmetic exists only in NutrientsCalculator, a final class with a private constructor and eighteen static methods (Section 10.1).
2. Ownership resolved from the context, never from the request. UserSession reads the principal from the security context and every service call uses it (Section 8.1).
3. State where it can be resumed. The plan under construction lives on the server, so the client holds no draft and a reload loses nothing (Section 10.9).
4. Aggregates embedded, subjects referenced. The four meal tracks are embedded in the daily document; the plan is referenced by identifier (Section 7.1).
5. Recompute rather than accumulate. Daily totals are derived from the meal tracks on every update, which makes the update idempotent (Section 10.6).
6. Progressive disclosure of arithmetic. Each wizard step shows the figure it computed before asking for the next input (Section 9.4).
7. Generated content is a leaf. The language model is reachable only from one controller and cannot write to any collection (Section 6.4).
8. Every state has a screen. Loading, empty, expired, error and unknown-route states all resolve to designed components (Section 9.9).

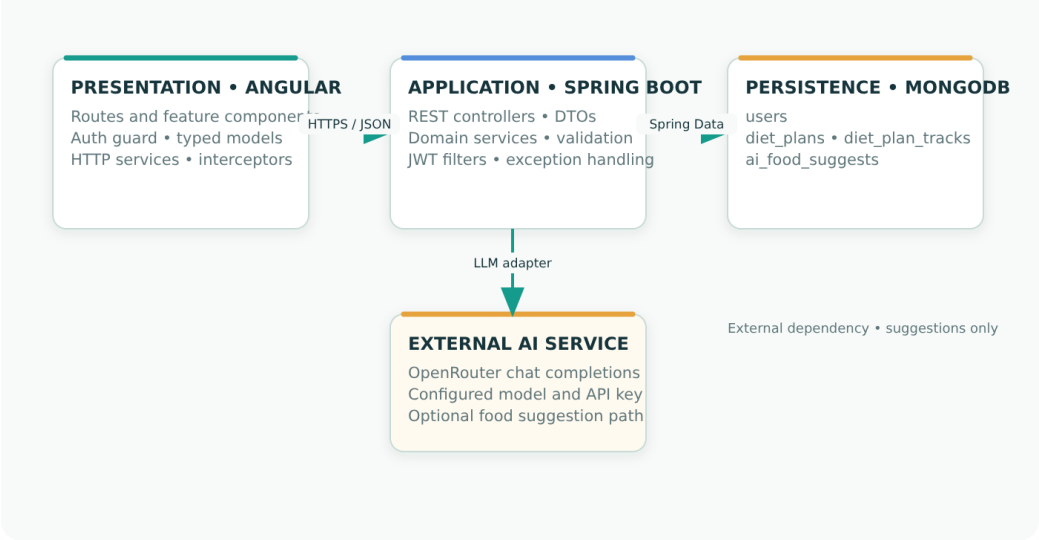


Figure 4.1: Three-tier architecture of the application with the packages that implement each tier

4.2 System Architecture

The presentation tier is a single-page client. It holds no persistent state other than the access token in memory, and it derives every displayed value either from a response or from a signal. Its four services mirror the four resource groups of the interface, and its two interceptors handle the two cross-cutting concerns that every request shares: prefixing the path and attaching the token.

The application tier is a Spring Boot service organised by feature. Each of the four features owns a controller, a service, a repository and its DTOs, and six sibling packages hold the cross-cutting code: configuration, exception handling, security, shared value objects, utilities and the shared model. The service layer is the only layer that contains domain logic, and the controllers are deliberately thin: each controller method resolves the principal where it is needed, delegates to one service method and returns the result.

The data tier is a single MongoDB database named macrova with four collections. There is no replica set, so multi-document transactions are unavailable, which the design tolerates because the only multi-document write is plan activation and the read path repairs a partial activation on the next load (Section 7.6). The model is presented in Figure 4.2.

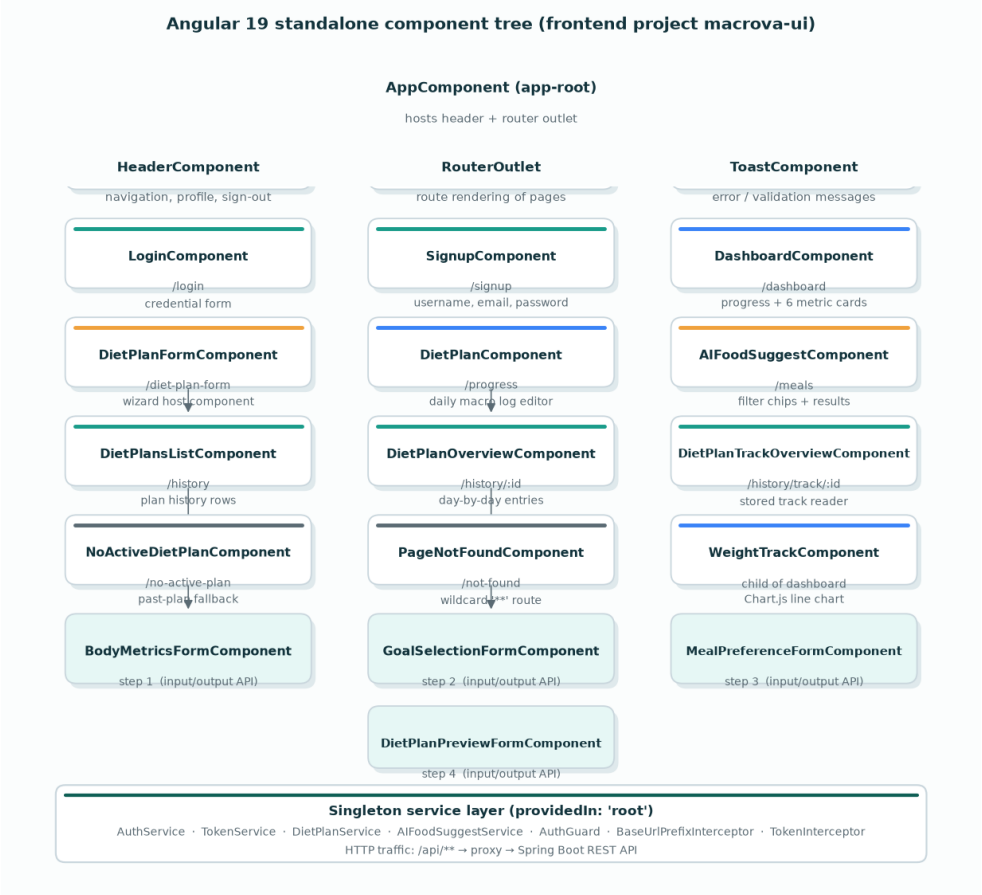


Figure 4.2: Component diagram showing the client components, the service classes and the shared infrastructure

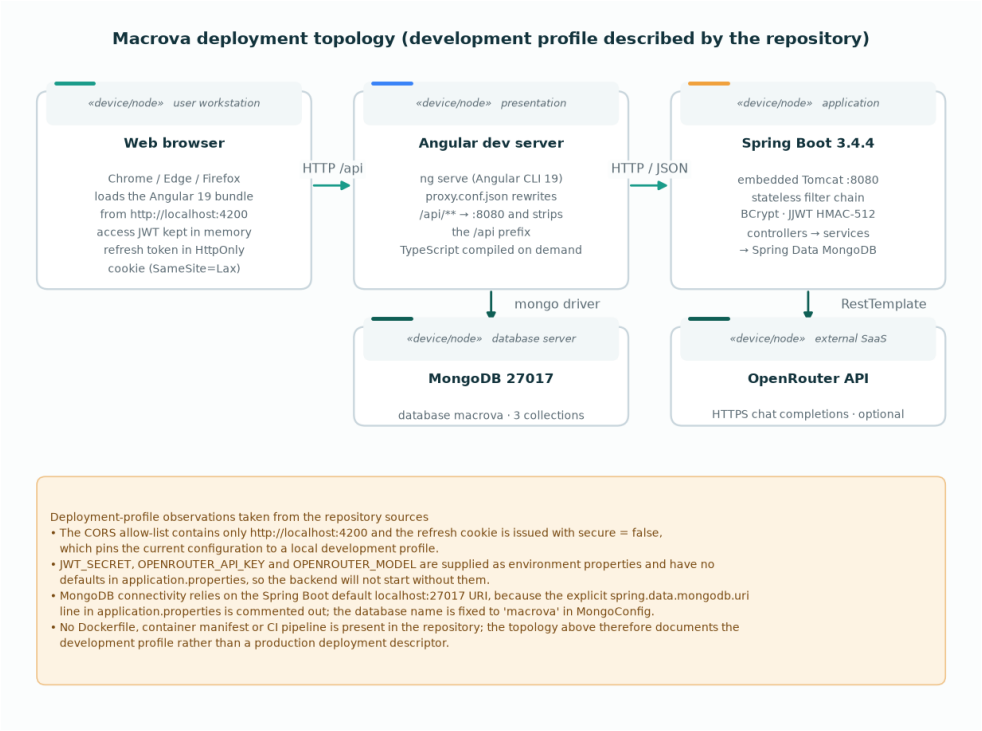


Figure 4.3: Deployment view of the development profile, with the browser, the development server and the two back-end processes

4.3 Data Flow Diagrams

The data-flow models are presented at three levels. The context diagram fixes the system boundary and the two external entities. The level-one diagram decomposes the system into five processes and four data stores. The level-two diagram expands the tracking process, which is the one whose internal flow is not obvious from the level-one diagram because it both reads and writes the same store within a single request.

4.3.1 Level 0 Data Flow Diagram (Context Diagram)

The context diagram shows the system as a single process with two external entities. The user supplies credentials, body measurements, goal selections, meal preferences, macronutrient intake and weights, and receives plans, tracking documents, metrics and suggestions. The model provider receives a prompt and returns a JSON array of suggestions, and is the only entity with which the system initiates communication. The arrangement described above is shown in Figure 4.4.

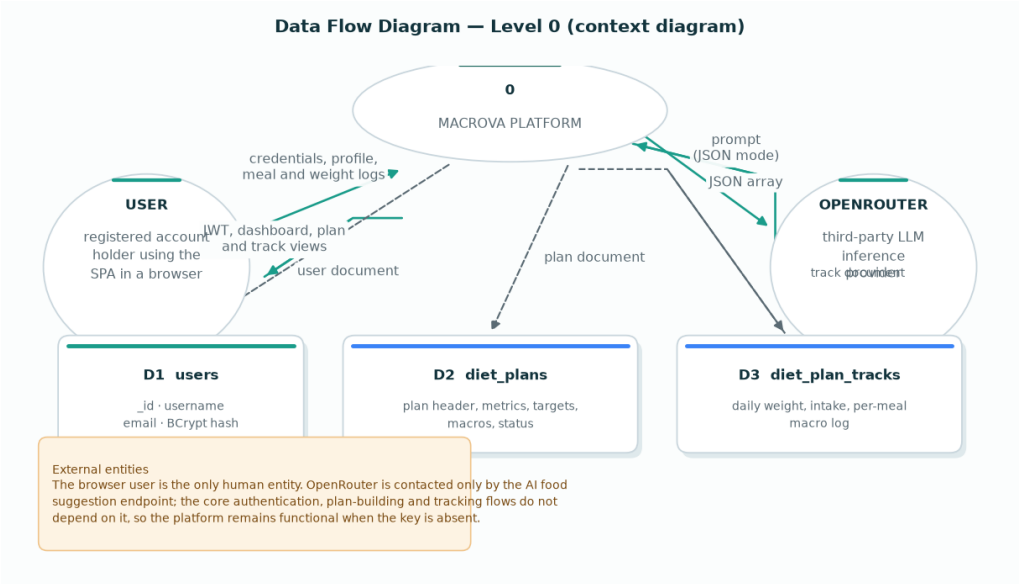


Figure 4.4: Context diagram showing the two external entities and the flows across the system boundary

4.3.2 Level 1 Data Flow Diagram (Functional Decomposition)

The first decomposition identifies five processes: identity, plan construction, tracking, reporting and suggestion. It identifies four data stores, of which three are written in normal operation. The flows between the processes are mediated by the stores rather than direct: plan construction writes the plan store, tracking reads it and writes the tracking store, and reporting reads both. The structure is drawn in Figure 4.5.

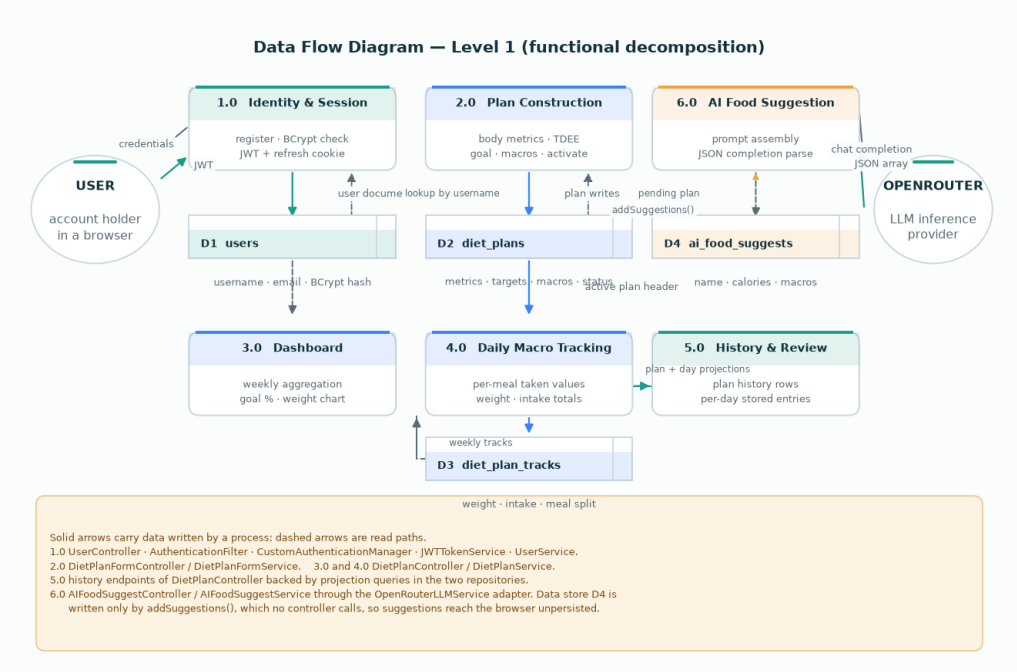


Figure 4.5: Level-one data flow diagram with the five processes and the four data stores

4.3.3 Level 2 Data Flow Diagram (Macro Tracking Refinement)

The refinement of the tracking process shows the three steps that a single update performs; the suggestion flow of Figure 4.10 follows the same contractual pattern one tier further out. The process first resolves the plan from the authenticated user and the active status, then resolves the current day, creating it if the calendar day has changed, and then writes the submitted macros into the selected meal track, recomputes the daily totals and returns the complete document. The refinement matters because it explains why the client can re-render an entire page from one response: the process returns the aggregate, not the delta. The model is presented in Figure 4.6.

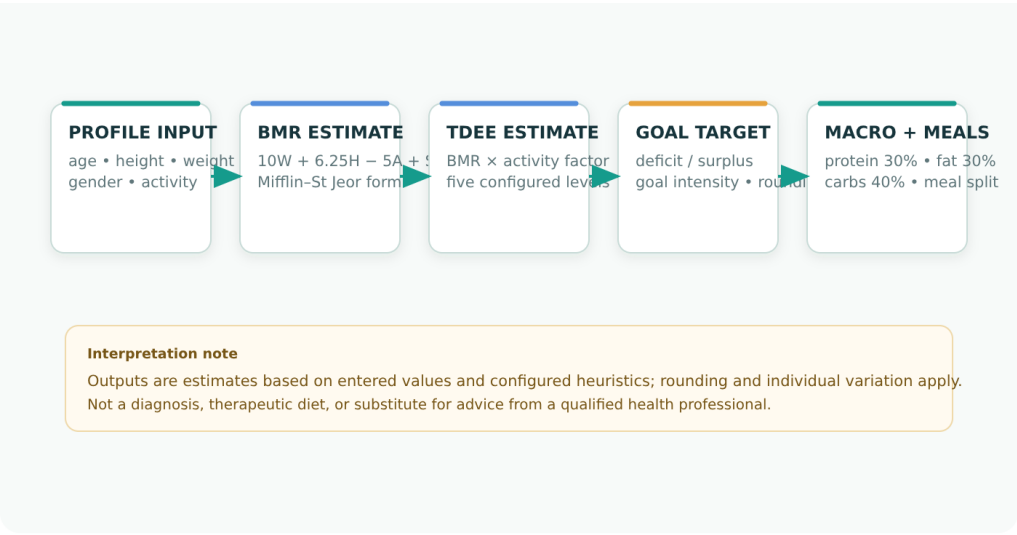


Figure 4.6: Refinement of the tracking process, showing the energy and macro conversions performed in one update

4.4 Use Case Diagram and Actor Specification

There is one actor, the dieter, who is either anonymous or authenticated. Two use cases are available to the anonymous actor, namely registration and authentication, and the remaining twelve require an authenticated session. Table 4.1 specifies the first seven in the form normally used for a use-case specification, with the preconditions, the main flow, the alternate flow and the postcondition; Table 4.2 specifies the remaining seven more briefly, because their flows are short.

Table 4.1: Use-case specification UC-01 to UC-07

ID	Use case	Actor	Precondition and main flow	Alternate and postcondition
UC-01	Register	Anonymous	None. The actor supplies a username, an e-mail address and a password; the system checks uniqueness, hashes the password, stores the user and returns the DTO.	A duplicate produces a 400 with a message; the account exists but is not signed in.
UC-02	Sign in	Anonymous	A registered account. The actor supplies credentials; the filter verifies them through the authentication manager, sets the refresh cookie and returns an empty 200.	A wrong credential yields a uniform 401; the session is not established.
UC-03	Refresh the access token	Anonymous with a cookie	A valid refresh cookie. The system validates the token and returns a new access token.	An absent or invalid cookie yields a 401; the client clears its state.
UC-04	Construct a plan	Authenticated	No pending plan. The actor completes four steps in order; each step persists and advances the status; the fourth activates the plan.	A step submitted with no pending plan yields a 400; the draft remains at the last committed step.
UC-05	Resume a plan draft	Authenticated	A pending plan exists. The client requests the status; the system returns the status and the payload of the current step.	If no draft exists the wizard renders the first step; the draft is unchanged.
UC-06	Navigate backward in the wizard	Authenticated	A pending plan at a later step. The client posts the target status; the system returns the reconstructed payload of that step.	A status ahead of the stored one is not served; the stored status is preserved.
UC-07	Log a meal	Authenticated	An active plan. The actor submits three gram values for a category; the system writes the meal track, recomputes the totals and returns the document.	An invalid category yields a 400; the document is unchanged.

Table 4.2: Use-case specification UC-08 to UC-14

ID	Use case	Precondition	Outcome
UC-08	Update body weight	An active plan and a current-day document	The plan's current weight and the day's weight are both updated
UC-09	View the dashboard	An authenticated session	The metrics are returned or the actor is routed to the empty state on a 404
UC-10	View the tracking screen	An active plan	The plan and today's document are returned; the document is created if absent
UC-11	Browse plan history	At least one plan	A projection of all plans, most recent first
UC-12	Browse the days of a plan	A plan identifier owned by the actor	A projection of the plan's daily documents, most recent first

ID	Use case	Precondition	Outcome
UC-13	Review a stored day	An authenticated session	The complete stored document, rendered read-only
UC-14	Request meal suggestions	An authenticated session	A list of suggestions, or an error routed to the error page

4.5 Object-Oriented and Behavioural Models

4.5.1 Class Diagram

The class model is organised around four aggregates. The user aggregate is a single entity. The plan aggregate is a plan entity with three embedded value objects and eight enumerated attributes. The tracking aggregate is a tracking entity with four value objects and six calorie objects nested two levels deep. The suggestion aggregate is a simple entity that is declared but not written. The shared package holds the value objects that cross aggregate boundaries, of which the macro and calorie pairs are the most used. The arrangement described above is shown in Figure 4.7.

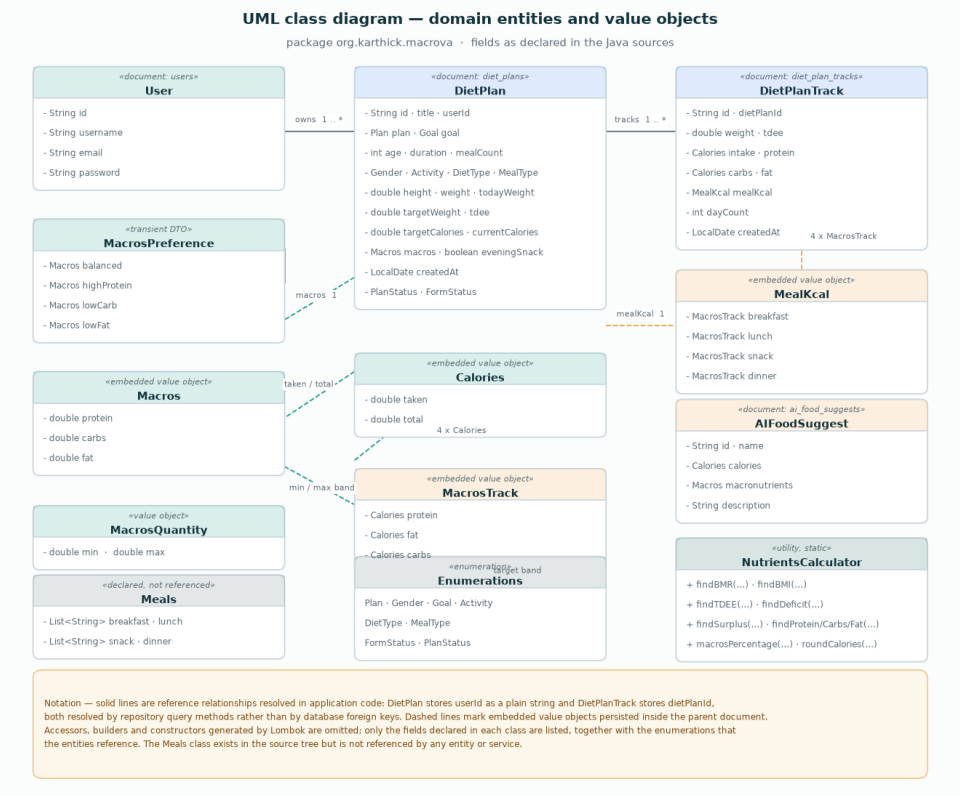


Figure 4.7: Class diagram of the entities, the embedded value objects and the enumerations

4.5.2 Sequence Diagram

The sequence diagram of plan construction shows the interaction across four steps. Each request is authenticated by the filter chain before it reaches the controller, the controller delegates to the form service, the service resolves the user, loads the pending plan, mutates it and saves it, and the response carries the payload that the next

step requires. The diagram makes visible the two properties that the design depends on: the plan is saved on every step, and the user identifier enters the flow from the security context rather than from the request body. The structure is drawn in Figure 4.8.

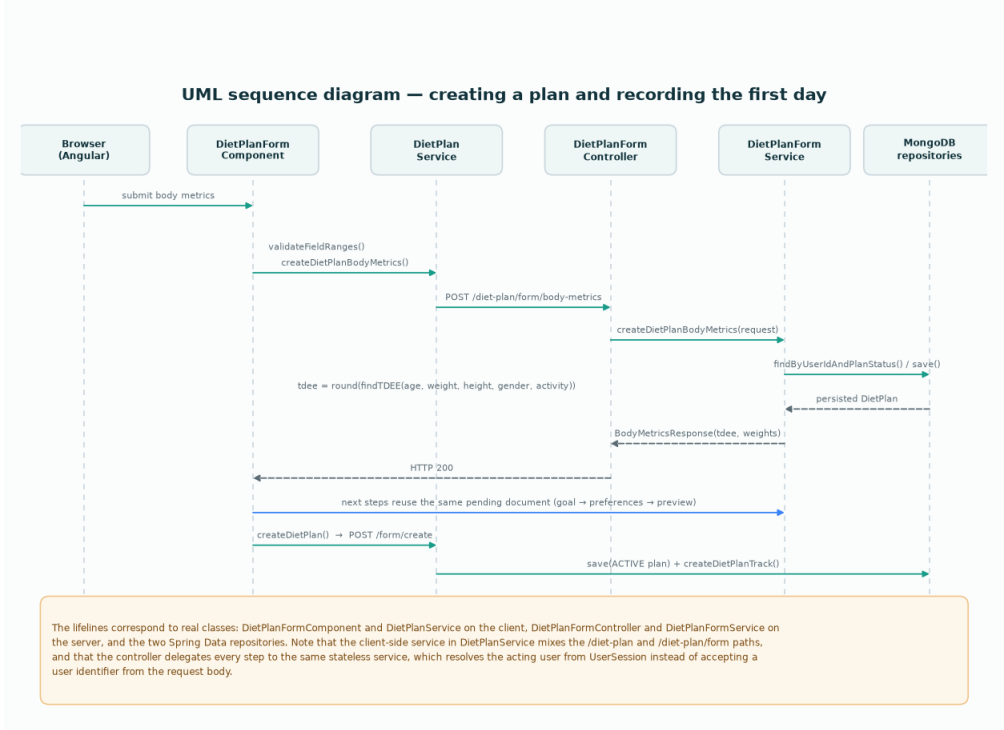


Figure 4.8: Sequence diagram of the four-step plan construction flow

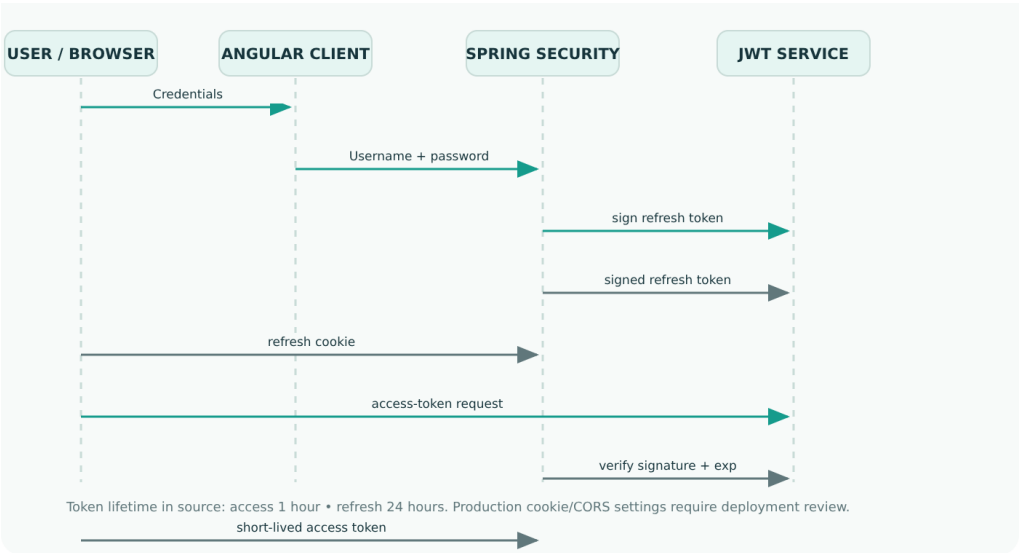


Figure 4.9: Sequence of the authentication, token refresh and guarded request flows

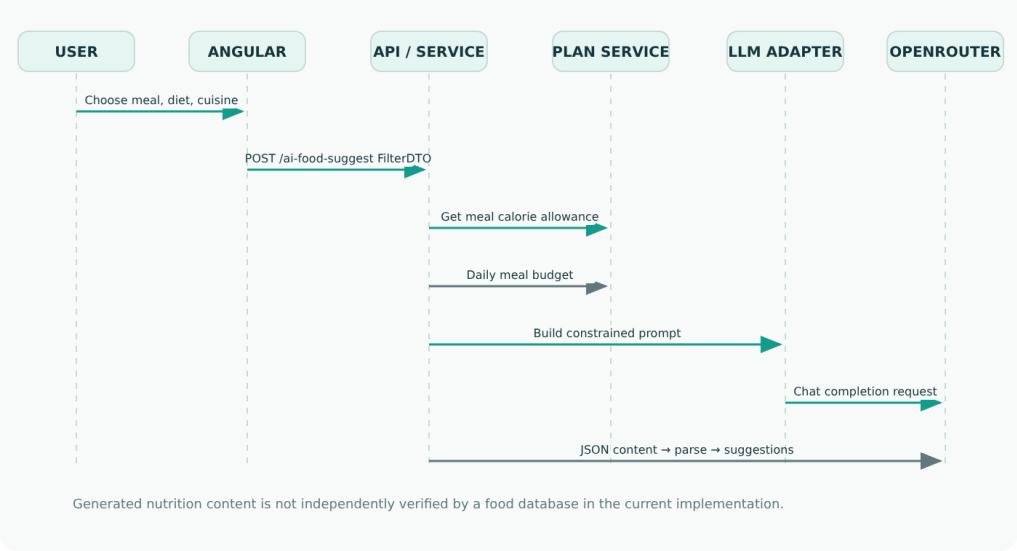


Figure 4.10: Sequence of the suggestion flow, from the client request to the rendered cards

4.5.3 Activity Diagram

The activity diagram of the tracking loop shows the decision that governs the whole interaction. On entering the tracking screen the client asks whether the plan has exceeded its duration; if it has, the flow branches to the empty state. Otherwise the plan is loaded, today's document is requested, and the user may iterate between logging a meal and updating the weight until they leave the screen. Every iteration returns to the same state, which is why the screen can be left and resumed without a save action. The model is presented in Figure 4.11.

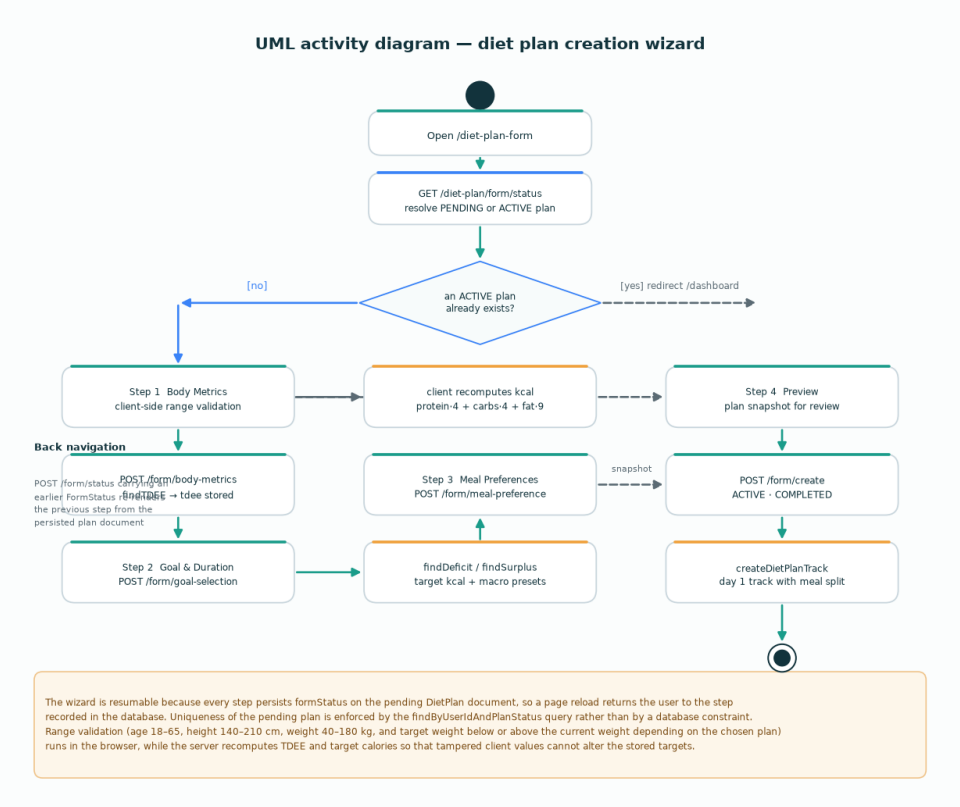


Figure 4.11: Activity diagram of the wizard, showing the four forward steps, the backward transitions and the reservation of the draft

4.5.4 State Machine Diagram

Two state machines are defined. The form status machine governs the wizard and has five states: body metrics, goal selection, meal preference, preview and completed. Transitions are forward only through the creation endpoints and backward through the status endpoint, and every step writes the new state, so the machine's state is never held only in the client.

The plan status machine governs the lifecycle and has three states. A plan is created pending, becomes active when the wizard completes, and becomes inactive when the elapsed day count exceeds the configured duration. The transition to inactive is evaluated when the client asks, not by a scheduler, which has two consequences that the report records: a plan with no visits is never marked inactive, and the transition happens as part of a read request, so a GET endpoint has a side effect. Both consequences follow from the decision to avoid a scheduler in a single-instance deployment. The arrangement described above is shown in Figure 4.12.

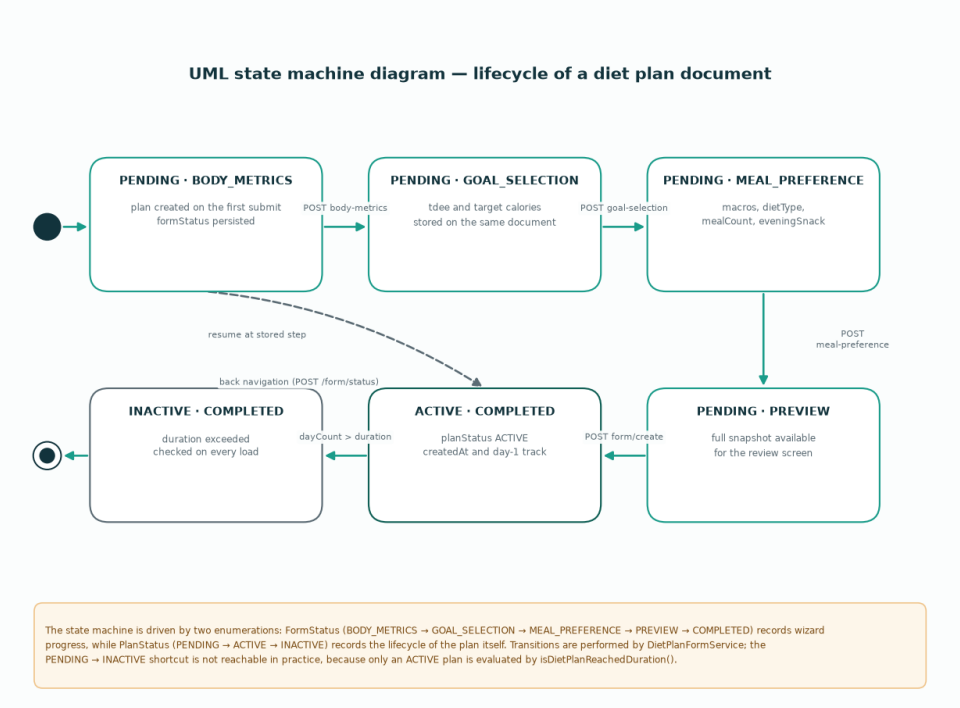


Figure 4.12: State machine of the form status and the plan status, with the triggers of each transition

4.6 Database Design

The data model is document-oriented and aggregate-first. Four collections are declared, of which three are written in normal operation. The document boundaries were drawn by asking which values are always read together, and the answer is recorded in Chapter 7 together with the data dictionary and a worked example of a stored document. This section states only the shape of the model and the reasoning that produced it.

A user document owns everything. A plan document embeds its own targets, macros and preferences, and refers to its owner by identifier. A tracking document embeds the day's totals, the four meal tracks and the day's energy figures, and refers to its plan by identifier. The reference direction is deliberate: a plan is a subject in its own right, because the history screens list plans without touching their days, whereas a day is meaningless without its plan and is therefore embedded in nothing but itself. The structure is drawn in Figure 4.13.

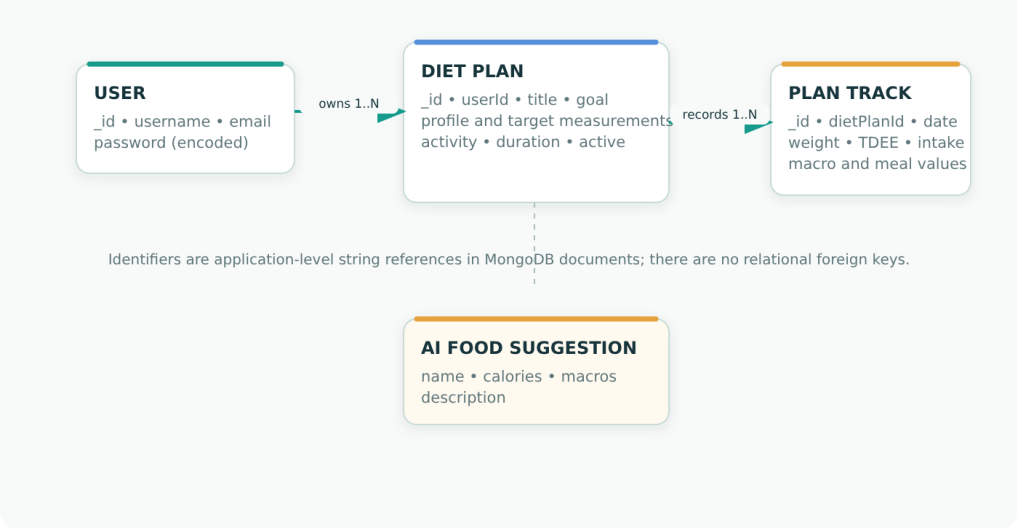


Figure 4.13: Domain model showing the four aggregates, their value objects and the reference between a day and its plan

4.7 User Interface Design System

The interface uses a small design system rather than a component library. Six primitives cover every screen: the card, the pill selector, the numeric field, the primary button, the metric card and the step indicator. The choice of a hand-built system over a library follows from the scope: twenty components and fifteen screens do not justify the weight of a full library, and a small system makes the visual language of Section 9.10 exactly as consistent as the report claims.

Table 4.3: Design-system primitives and their use across the screens

Primitive	Purpose	Appears on
Card	The single surface for grouped content; eight-pixel radius, one-pixel border, white fill on a tinted canvas	All eleven form and content screens
Pill selector	A single choice from two to five options, rendered as a row of rounded buttons	Wizard steps 1, 2, 3 and 4; suggestion filters
Numeric field	A labelled input with the required range stated in the label	Wizard step 1; tracking summary; meal cards
Primary button	The one concluding action of a screen, full width where it completes a form, in the brand colour	Every screen except the dashboard and the history drill-downs
Metric card	An uppercase label with a value and an inline unit	Dashboard; tracking summary
Step indicator	Four numbered circles joined by rules, filled for completed steps	All four wizard steps

4.8 Security Design

The security design has four elements: how a password is stored, how a session is represented, how a request is authenticated and how data is confined to its owner. Table 4.3 summarises the decisions and, for each one, the weakness that remains after it.

Table 4.4: Security decisions and their residual weaknesses

Concern	Decision	Residual weakness
Password storage	BCrypt with a per-password salt, compared through the encoder	No policy beyond an eight-character minimum enforced in the client
Credential verification	A custom authentication manager that translates an unknown username into a bad-credentials failure, so the response cannot enumerate accounts	No throttling or lockout on repeated failure
Access token	HMAC-512 signed, short-lived, held in memory by the client and sent in the authorisation header	Cannot be revoked before expiry; the client's copy is readable by any script on the page
Refresh token	HMAC-512 signed, longer-lived, delivered as an HttpOnly, SameSite Lax cookie and never read by client script	Not rotated and not revocable; the secure attribute is absent in the development profile
Sign-out	The refresh cookie is overwritten with an empty value and a zero maximum age	The access token in memory remains valid until it expires
Request authentication	A filter verifies the bearer token and populates the security context; an exception filter translates verification failures into 401 responses	A failure in the filter chain returns plain text rather than the JSON error shape
Data confinement	Every service resolves the user from the security context and builds the query from it	The rule is applied by convention in each method rather than by a central check
Cross-origin access	An explicit allow-list with credentials permitted	Pinned to the development origin
Route protection	A functional guard on eleven of the thirteen routes	Two history routes are unguarded; the subsequent request still fails with 401

The design maintains a single invariant that is worth isolating, because it is the property on which the whole security argument rests: *no controller accepts a user identifier from the client*. The identifier is obtained from the security context inside the service and used to build every query. Table 4.4 lists the queries that enforce the invariant; a reviewer can verify each one against the source, and Appendix B records the result of doing so. The complete set of values is given in Table 4.5.

Table 4.5: Queries that enforce ownership, with the identifier each uses

Service method	Query	Identifier source
findDietPlanByUserId	owner and status	Security context
getPendingDietPlan	owner and pending status	Security context
findAllByUserId	owner, projected	Security context
findPastDietPlan	owner, most recent, projected	Security context
findDietPlanTrackByDietPlanId	plan identifier resolved from a plan already scoped to the owner	Plan resolved from the context
findByDateRangeAndDietPlanId	plan identifier and date range	Plan resolved from the context
getUsername	identifier of the authenticated principal	Security context

CHAPTER 5
TECHNOLOGIES AND TOOLS USED

5.1 Software Requirements

The software environment required to build and run Macrova is listed in Table 5.1. Every version is taken from the build manifests in the repository, from pom.xml for the service and from package.json for the client, rather than from the author's machine, so the table describes the project rather than one particular development environment.

Table 5.1: Software requirements of the Macrova application

Category	Software	Version	Role in the project
Language (service)	Java, JDK	21	Backend source and runtime; the Maven compiler is configured for release 21
Language (client)	TypeScript	5.7	All Angular source, with strict template type checking
Markup and style	HTML5, CSS3	—	Twenty component templates, nineteen component stylesheets and one global stylesheet
Framework (service)	Spring Boot	3.4.4	Auto-configuration, embedded container, dependency management
Web layer	Spring Web MVC	6.x, managed	REST controllers and Jackson message conversion
Security	Spring Security	6.x, managed	Password encoding, filter chain, cross-origin policy, security context
Token library	Auth0 java-jwt	4.5.0	HMAC-512 signing and verification of both token types
Persistence	Spring Data MongoDB	4.x, managed	Repository derivation, projections and type converters
Object mapping	ModelMapper	3.2.2	Entity-to-DTO conversion in the shared mapper
Boilerplate	Lombok	1.18.38	Accessors, builders and constructors on entities, DTOs and value objects
Database	MongoDB Community Edition	6 or later	Document store holding four collections
Framework (client)	Angular	19.2	Standalone components, signals, functional interceptors and route guards
Reactivity	RxJS	7.8	HTTP observables and operator composition
Change detection	zone.js	0.15	Zone-based detection with event coalescing
Charting	Chart.js	4.4.9	Weight trend line chart on the dashboard
Token inspection	jwt-decode	4.0.0	Reading the expiry claim in the client
Build (service)	Apache Maven through the wrapper	3.9 wrapper	mvnw and mvnw.cmd are committed, so no global Maven installation is needed
Build (client)	Angular CLI with esbuild	19.2	ng serve, ng build and the development proxy
Testing	JUnit 5 via spring-boot-starter-test	5.x, managed	One context-load test in the delivered repository
Documentation	ReportLab, python-docx, PyMuPDF, Matplotlib	current releases	This report, its diagrams and the screenshot preparation

Category	Software	Version	Role in the project
Version control	Git	—	One repository holding the service, the client and the documentation

5.2 Hardware Requirements

Macrova is a light application by contemporary standards: the service comprises 71 classes and the client bundle sits well inside the default Angular budget of one megabyte. The figures in Table 5.2 describe the machine on which the project was built and the practical minimum for a demonstration to a reviewer.

Table 5.2: Hardware requirements

Component	Minimum for development	Recommended	Notes
Processor	Dual-core x86-64, 1.6 GHz	Quad-core, 2.4 GHz or faster	The Maven build and the Angular development server are the heaviest workloads
Memory	4 GB	8 GB or more	A JVM, a Node.js process and the database daemon run concurrently
Free disk	1.5 GB	3 GB	The Maven and npm dependency caches dominate; the application data is measured in kilobytes
Display	1280 × 720	1920 × 1080	The screenshots of Chapter 9 were captured in a window of 1921 × 978 pixels
Network	Not required	Broadband	Required for the first build and for the optional suggestion feature
Server, if deployed	1 vCPU, 1 GB RAM	2 vCPU, 2 GB RAM	Sufficient for a small group, with the database co-located

Storage deserves a specific comment because the persistence model determines it. A plan document with all of its embedded value objects occupies roughly seven hundred bytes of BSON, and a tracking document with four meal tracks roughly six hundred. A user who tracks daily for a year therefore consumes under a quarter of a megabyte. The dominant storage cost in the repository is not data at all but the Maven and npm caches, which is why the free-disk column of the table is an order of magnitude larger than the data it protects.

5.3 Programming Languages Used

5.3.1 Java 21 for the Application Tier

The service is written in Java 21, the version declared by the java.version property of pom.xml and passed to the compiler through the release flag. Three language features explain the shape of several classes. Switch expressions with arrow labels select the macronutrient preset and the meal track, and validate a meal category in the constants class. The var keyword appears where a long generic type would obscure a short

method body. And the functional style of the service layer shows in the use of Optional: every repository finder returns one, and it is consumed with ifPresent, orElseGet, map and isEmpty rather than with get, except in the two places where an explicit emptiness check throws the domain exception.

Two features are deliberately absent. Records are not used, because the DTOs are the targets of both ModelMapper and Jackson and those libraries are simpler to configure against beans with Lombok accessors. Text blocks are not used either, because the longest string constants in the codebase are prompt templates of two or three lines rather than documents. The code therefore reads as conservative Java with a modern switch, which is a fair description of the style the project adopted.

5.3.2 TypeScript 5.7 for the Presentation Tier

The client is written in TypeScript with the strictness settings generated by the Angular CLI. Three idioms recur. Types are imported with a type-only import where they are erased at compile time, which keeps the emitted JavaScript free of unused module references. Component contracts are declared with the signal-based input and output functions rather than with decorators: each wizard step declares a required input whose type parameter names the payload it renders and emits a typed output when it is submitted, so a mismatch between a step and its host is a compile error. And state is held according to its scope, in a component field for local state, in a signal on a singleton service for shared state, and on the server for state that must survive navigation.

The last of those three is the decision that matters architecturally. The wizard keeps no draft in the browser: the pending plan lives in the database, and the client re-renders whichever step the server reports as current. That is why a reload at the third step returns the user to the third step rather than to the first, and why the host component has no reconstruction logic at all.

5.3.3 HTML5, CSS3 and JSON

The templates use the built-in control-flow blocks of modern Angular rather than the structural directives of the previous generation, which is the current idiom and removes the need to import a directives module in every component. CSS is organised per component with the default emulated encapsulation, so a rule declared in one component cannot leak into another; the global stylesheet holds only the design tokens, the reset, the type scale and the few utility classes that are genuinely shared. JSON is the only wire format, both between the browser and the service and between the service and the model provider, which is why a single serialisation stack covers both directions.

5.4 Frameworks and Libraries

Table 5.3 records why each dependency is present. The criterion applied during the review was that a dependency must be used by at least one class or component, so the table doubles as a justification of the two manifests.

Table 5.3: Frameworks and libraries with their justification

Dependency	Tier	Used by	Justification
spring-boot-starter-web	Service	Four controllers	Servlet container, MVC dispatch and Jackson message conversion
spring-boot-starter-security	Service	Security configuration, three filters, the authentication manager	Filter chain, BCrypt encoder and security context
spring-boot-starter-data-mongodb	Service	Four repositories, database configuration	Repository derivation, projections and custom date converters
java-jwt 4.5.0	Service	Token service	HMAC-512 signing and verification
modelmapper 3.2.2	Service	Shared mapper, user service	Declarative entity-to-DTO mapping
lombok 1.18.38	Service	All entities, DTOs and models	Accessors, builders and constructors without boilerplate
@angular/core and @angular/common	Client	All twenty components	Component model, control flow, pipes and built-in directives
@angular/router	Client	Route table, guard, header links	Routes, guards and navigation events
@angular/forms	Client	Login, signup, four wizard steps, weight input	Two-way binding through ngModel
@angular/common/http	Client	Four services and two interceptors	Typed HTTP client and the interceptor chain
rxjs	Client	Guard, interceptors, services	Observable composition with switchMap, map, tap and catchError
chart.js	Client	Weight trend component	Canvas line chart of the week's weights
jwt-decode	Client	Token service	Client-side inspection of the expiry claim
zone.js	Client	Bootstrap	Change detection, as configured by the CLI

5.5 Development and Runtime Environment

The repository is a single Git working tree with three top-level directories. The backend directory holds the Maven project and a committed Maven wrapper. The frontend directory holds the Angular workspace. The docs directory holds the diagrams, the screenshots, the reference documents and this report. The service is started with the wrapper script and the client with the npm start script, which invokes the Angular development server with the proxy configuration. The database runs independently, and the service connects to it with the driver default of the local host and port, with the database name fixed to macrova by a configuration class.

Three properties are supplied through the environment rather than committed: the secret used to sign and verify tokens, the key used as the bearer credential for the model provider, and the identifier of the model to request. None of the three has a default value, and the placeholders are resolved at start-up, so the service refuses to start when

they are absent. That is a deliberate safety decision that costs convenience during a demonstration, and Section 3.5.3 records the cost.

The development proxy is what makes the two-process arrangement transparent to the browser. The proxy configuration maps the /api prefix to the service port, enables origin rewriting and removes the prefix, so a client request to /api/diet-plan/metrics reaches the service as /diet-plan/metrics. Because the browser sees only the development-server origin, the cross-origin allow-list of the service is never exercised during development and would matter only if the client were served from a different origin, which the delivered build does not do.

5.6 Build and Run Workflow

The development workflow consists of five commands, and Table 5.4 records them with the reason each one exists. The table is included because the repository is handed over as a working tree rather than as a deployed artefact, and a reviewer who attempts to run it will need exactly this information.

Table 5.4: Build and run commands of the delivered system

Step	Command	Directory	What it does
1	mongod --dbpath <path>	any	Starts the database on its default port; the service creates the macrova database on first use
2	export JWT_SECRET, OPENROUTER_API_KEY, OPENROUTER_MODEL	any	Supplies the three properties the service resolves at start-up
3	./mvnw spring-boot:run	backend	Compiles the 71 classes and starts the embedded container on port 8080
4	npm start	frontend	Starts the Angular development server with the proxy configuration
5	npm run build	frontend	Produces the production bundle in the dist directory, for a static deployment

Two properties of the workflow are worth noting. The Maven wrapper is committed, so the first command downloads the declared Maven version rather than requiring a global installation, which is what makes the repository reproducible on a machine that has only a JDK. And the proxy configuration is read by the development server rather than by the browser, so a change to the service port requires editing one JSON file and restarting the client, not rebuilding it.

CHAPTER 6
MODULES AND IMPLEMENTATION OVERVIEW

The implementation is organised into eight modules. The first five are service concerns and the last three are client concerns. For each module this chapter states its responsibility, enumerates its artefacts and explains the behaviour it implements; Chapter 10 then examines the arithmetic in depth and Chapter 8 the interface through which the modules communicate. The same information is presented in tabular form in Table 6.1.

Table 6.1: Module map of the Macrova implementation

Module	Tier	Principal artefacts	Primary responsibility
M1 Identity and session	Service	User controller and service, user repository and entity, token service, custom authentication manager, three filters, security configuration, session component	Registration, credential verification, token issuance and request authentication
M2 Plan construction	Service	Form controller and service, eight form DTOs, plan entity, plan repository	Four-step wizard, energy estimation, target derivation and activation
M3 Daily tracking	Service	Plan controller and service, tracking entity and repository, metrics and overview DTOs	Per-meal logging, weight updates, daily aggregation and weekly metrics
M4 AI suggestions	Service	Suggestion controller and service, filter DTO, suggestion entity, provider adapter and its interface, chat model	Prompt construction, provider call, reply parsing and card rendering
M5 Cross-cutting	Service	Exception advice and three exception types, error response, mapper, two configuration classes, calculator, constants, nine value objects and eight enumerations	Error translation, bean configuration, conversions, arithmetic and shared types
M6 Shell and routing	Client	Root component, route table, bootstrap configuration, header, toast, guard, authentication and token services, two interceptors	Bootstrap, navigation, session state, request decoration and route protection
M7 Plans and tracking	Client	Wizard host and four step components, tracking component, dashboard, weight chart, limit pipe	Wizard rendering, macro log editing, dashboard metrics and chart
M8 History and fallbacks	Client	History list, plan overview, day overview, suggestion screen, empty state, not-found screen	History navigation, suggestion filters and empty or error states

6.1 Module 1: Identity and Session Management

The identity module is the only module whose principal sign-in behaviour is not mediated by a controller. Registration, refresh and sign-out are controller methods, but the sign-in request itself is handled by a filter that extends the username-and-password authentication filter of Spring Security and is mounted at the authenticate path by the security configuration. The arrangement follows the convention of the framework and has a useful consequence: the password never reaches a controller method and therefore cannot be logged by accident.

Registration is implemented in the user service. The method checks the uniqueness of the username and of the e-mail address, encodes the password with the injected encoder, saves the entity and converts it to a DTO. The conversion matters twice over: the response never contains the password hash, and the DTO is produced by the shared mapper rather than by a hand-written method, which is the convention the module follows throughout. A duplicate username produces a bad-request failure and a duplicate address the corresponding message for the address; the advice translates both into a 400 response.

Sign-in proceeds in four steps. The filter deserialises the JSON body into a user instance, constructs an authentication token and delegates to the custom manager. The manager loads the user through the user service, translating an unknown username into a bad-credentials failure so that the response cannot be used to enumerate accounts, and compares the supplied password with the stored hash through the encoder. On success the filter's successful-authentication hook asks the token service for a refresh token and writes it into a response cookie that is HttpOnly, SameSite Lax, scoped to the whole path and given a maximum age equal to the refresh lifetime. The response body is empty; the client obtains its access token from a separate call, which keeps the token out of any JavaScript-readable store.

The refresh endpoint is a controller method because it needs the cookie. It reads the cookie, rejects the request with an unauthorised failure when it is absent, validates the token and issues a new access token wrapped in a DTO that prefixes the string with the bearer scheme so that the client can use it directly. Sign-out invalidates nothing on the server, because the tokens are stateless; it overwrites the cookie with an empty value and a zero maximum age, which removes it from the browser.

The final artefact of the module resolves the acting user from the security context and exposes the authenticated identifier and username. Every service in the next two modules calls the first of those rather than accepting an identifier from the request, which is the enforcement mechanism for the data-scoping invariant of Section 4.8.

6.2 Module 2: Diet Plan Construction

Module M2 implements the four-step wizard. Its controller exposes six endpoints rooted at the form path: a GET and a POST on the status resource, and one POST each for the body metrics, the goal selection, the meal preference and the activation. The POST on the status resource is the mechanism for backward navigation: the client sends the form status of the step to which it wishes to return, and the service responds with the payload appropriate to that step, reconstructed from the pending document.

The form service holds all of the module's logic, and its methods fall into three groups. The three creation methods each begin by locating the pending plan; the body-metrics

method is the exception, because it is the method that creates the pending plan in the first place, and it therefore tolerates the absence of one by querying for a pending plan, copying its identifier if it exists and writing the request over the document either way. The status method assembles an envelope object that can carry any of five payloads depending on which step is current, and the private conversion methods translate between entities and DTOs in both directions.

The envelope deserves examination because it illustrates a design trade-off. It declares optional fields for the plan, the body-metrics request, the body-metrics response, the calorie-estimation response and the two status enumerations, and only one or two of them are populated on any given call. A stricter design would have used five separate endpoint pairs; the single envelope reduces the number of endpoints the client must know and makes backward navigation a single request, at the cost of a wider contract than necessary. The report records the trade-off rather than hiding it.

Activation has a side effect that is easy to overlook. Before returning, the method calls the tracking service to create the day-one document. That document copies the plan's expenditure, calorie target and macronutrient totals, and computes the per-meal split from the macros. Because the split is stored at creation time, the first day's meal budgets are fixed even if the plan is later edited, which is the correct behaviour for a log that is meant to be comparable with the plan active on that day.

6.3 Module 3: Daily Tracking and Progress

Module M3 is the largest service in the application. Its sixteen public methods fall into four families: plan retrieval, which reads a plan by identifier, by user, by status or as a projection; tracking lifecycle, which finds or creates the document for a day; logging, which accepts a weight or a set of meal macros and recomputes the totals; and aggregation, which assembles the metrics payload, the calorie allowance for a meal category and the day-by-day overview.

The logging method is the heart of the module. It begins by resolving today's document, which the finder creates on demand when the calendar day has changed. It then selects the macro track for the requested category through a switch expression over the four names defined by the constants class, copies the taken values into that track, recomputes the three daily totals by summing the four categories, converts those totals into energy with the Atwater factors and stores the result in the intake object. The method returns the whole document, so the client can refresh every card from a single response without a second request.

The aggregation method assembles the dashboard payload. It determines the current week in the Asia/Kolkata zone, queries the documents between the start of the previous week and the end of today, projects each record into a weekday abbreviation and a

weight value, and builds the metrics object containing the plan title, the percentage of the target achieved, the duration, the current day number, the current weight, the three macro targets and the two arrays that the chart consumes.

Two methods carry the plan lifecycle. One answers a boolean question used by the client to choose between the dashboard and the empty state. The other compares the elapsed day count with the configured duration, transitions the plan to inactive and returns true when the duration is exceeded. Both are called on page load rather than scheduled, which is the behaviour discussed in Section 4.5.4.

6.4 Module 4: AI-Assisted Food Suggestion

Module M4 is the smallest service module and the one with the largest external surface. Its controller exposes a single POST endpoint that accepts four string filters: the meal category, the meal type, the diet type and the cuisine. The service transforms those filters into a prompt, appends the calorie allowance for the selected meal, sends the prompt to the provider through an interface, and parses the reply into a list of suggestion objects.

The prompt is assembled by a private method that formats a single sentence: a request for five healthy dishes of the chosen cuisine, meal type and diet type, for the chosen meal, under the computed calorie figure. The allowance comes from the tracking service, which computes the energy of the selected meal's macro track from the active plan and falls back to five hundred kilocalories when no active plan exists. The fallback is why the suggestion page can be opened by a user whose plan has expired without producing a server error.

The adapter is the only class in the system that performs an outbound request. It injects a REST template declared as a bean in the application class, reads the model identifier and the key from configuration, and posts a chat request to the completions endpoint. The request carries a system message that instructs the model to answer only with a JSON array in a fixed schema and a user message that carries the generated prompt; streaming is disabled. The adapter then extracts the content of the first choice from the response envelope and returns it as a string, leaving the parsing to the service.

Two properties of the module must be stated plainly. First, the nutritional values in the reply are generated: no food-composition database validates them, and the same dish may receive different values on two consecutive calls. Second, the repository contains a method that would persist a list of suggestions, but no controller or service invokes it, so the suggestions collection remains empty in normal operation. Chapter 11 records both properties in the verification matrix. The module is nevertheless complete in the sense that matters for a suggestion feature: it never writes generated output into

a plan.

6.5 Module 5: Cross-Cutting Infrastructure

The infrastructure module holds the classes that every other service module depends on. The exception package defines three runtime exceptions, each with a single message constructor, and a controller advice that maps them to status codes: a not-found failure to 404, a bad-request failure to 400, an unauthorised failure to 401 and a JSON processing failure to 410. The choice of 410 for a parsing failure is unusual and is recorded as an observation rather than a recommendation; 500 or 502 would be the conventional choice for a failure originating in an upstream provider.

The configuration package holds two classes. One declares the mapper bean that the shared mapper uses to convert entities into DTOs. The other extends the abstract database client configuration, fixes the database name and registers two converters between the local date type and the driver date type, both operating in UTC. The converters exist because several documents store a local date, which the driver cannot map without instructions; the UTC choice means that a document created in the early hours of the morning in a positive-offset zone is stored against the previous UTC date, a detail that the single automated test does not exercise.

Two further classes belong here even though they live in other packages. The constants class defines the four meal category strings and a validation method that returns true only for those four values, which is what the switch in the tracking service relies on. And the calculator is a final class with a private constructor and eighteen static methods; it is the only place in the code where the energy equations appear, and each method carries a comment stating the formula, the units and the convention.

6.6 Module 6: Application Shell, Routing and Session

The client is bootstrapped by a single call in the entry module with the configuration object declared in the application configuration file. That object registers zone-based change detection with event coalescing, the route table, and the HTTP client with the two functional interceptors in the order base prefix followed by token attachment. The order matters, because the token interceptor inspects the request URL to decide whether the request is public, and it can only recognise the public paths once the prefix has been applied.

The route table declares thirteen entries. The empty path redirects to the dashboard. Two routes are public, namely the login and signup screens. Nine routes are guarded: the dashboard, the progress page, the suggestion page, the history list and the wizard, together with the empty state and the two history drill-downs, which the table declares without a guard. That omission is recorded as an inconsistency in Chapter 11 rather

than corrected, because the report documents the repository as it stands. The remaining entries are the error page and the wildcard route that redirects to it.

The guard is the enforcement point for client-side route protection. It returns an observable boolean obtained from the authentication service, which first checks the in-memory access token against its expiry claim and, if the token is absent or expired, attempts a refresh. A failed refresh clears the token and returns false, and the guard navigates to the login screen. Because the guard runs before the component is created, a user who reloads a deep link is transparently re-authenticated from the refresh cookie.

The header is the only component that renders differently for an anonymous and an authenticated visitor. It subscribes to router events and records the current URL, so that the login page shows a sign-up button and the signup page a log-in button, while every other page shows the four navigation links and the profile avatar. The avatar letter is derived from the username signal through a computed signal, with a fallback initial so that a slow response does not leave the avatar empty.

6.7 Module 7: Plan and Tracking Screens

The wizard host component holds three pieces of state: the body-metrics request that the first step edits, the three response payloads returned by the service, and the current form status. On initialisation it fetches the form status and dispatches to the appropriate step; each step emits its payload through a typed output, and the host advances the status and stores it. Backward navigation posts the previous status and re-renders from the reply, which means the host never reconstructs an earlier payload from a later one and the client holds no draft of its own.

The tracking page loads in a fixed order. It first asks whether the plan duration has been exceeded and redirects to the empty state if it has. It then loads the active plan and, using its identifier, loads today's document. Both requests treat a 404 as a signal to navigate rather than as an error to display, which is why the page has no error branch for the missing-plan case. The page then renders the daily summary, the four meal cards and the calorie meter, and writes the updated document back into the same field whenever a meal is updated or the weight is changed, so a second update does not read stale values.

The dashboard is smaller but performs the same defensive sequence: a duration check, then the metrics request, with a 404 routed to the empty state. Its template renders the goal percentage through the one custom pipe in the application, which caps the meter at one hundred per cent while displaying a greater-than sign in the label when the user has exceeded the target.

6.8 Module 8: History, Suggestions and Fallback Screens

The history list loads a projection of every plan of the user, ordered by creation date descending, and renders each row with the title, a status badge, the plan type, the weight transition and the duration. Because the projection omits the embedded objects, the list is served by a single efficient query even when the user has many plans, and the same query pattern is reused with a limit of one for the empty state's offer to resume the most recent plan.

Two drill-down screens complete the history feature. The first lists the daily documents of one plan as cards showing the date, the weight, the plan's expenditure and the consumed energy, with the three macro totals beneath; it is again served by a projection. The second shows a single stored document read-only, including its per-meal split, and is the screen a user reaches to review what they actually logged on a given day. Neither offers editing, which is consistent with the principle that history is a record rather than a workspace.

The suggestion page holds four filter groups in a single object, renders each group as a row of pill buttons, and issues a POST when the user asks for suggestions. It maintains two boolean flags that drive the three visual states of the results region: a prompt before the first request, a loading indicator during the request and an error message when the call or the parse fails. The two fallback screens are deliberately minimal: the empty state explains the situation in two sentences and offers a single primary action, and the not-found screen renders a large numeral, a short explanation and a button back to the dashboard.

CHAPTER 7
DATABASE DESIGN AND DATA DICTIONARY

7.1 Design Approach

Macrova uses MongoDB, so the schema design proceeds from access patterns rather than from normalisation. Three questions determined the model. Which data are always read together? The daily document and its four meal tracks, which are read together every time the tracking page or the dashboard loads. Which data change independently? The plan header and its targets, which change only while the wizard is being used. And which data are bulky and rarely read in full? The historical documents, which the history screens read through projections that never load the embedded meal tracks at all.

The answers produced the model of Figure 7.1: a compact plan header document, a daily document that embeds its own value objects, and a projection strategy for the history screens. Table 7.1 records the resulting collections with their approximate document sizes, which were estimated from the number of fields and the size of the embedded objects rather than measured against a corpus, because the corpus available to the project was a single user's history.

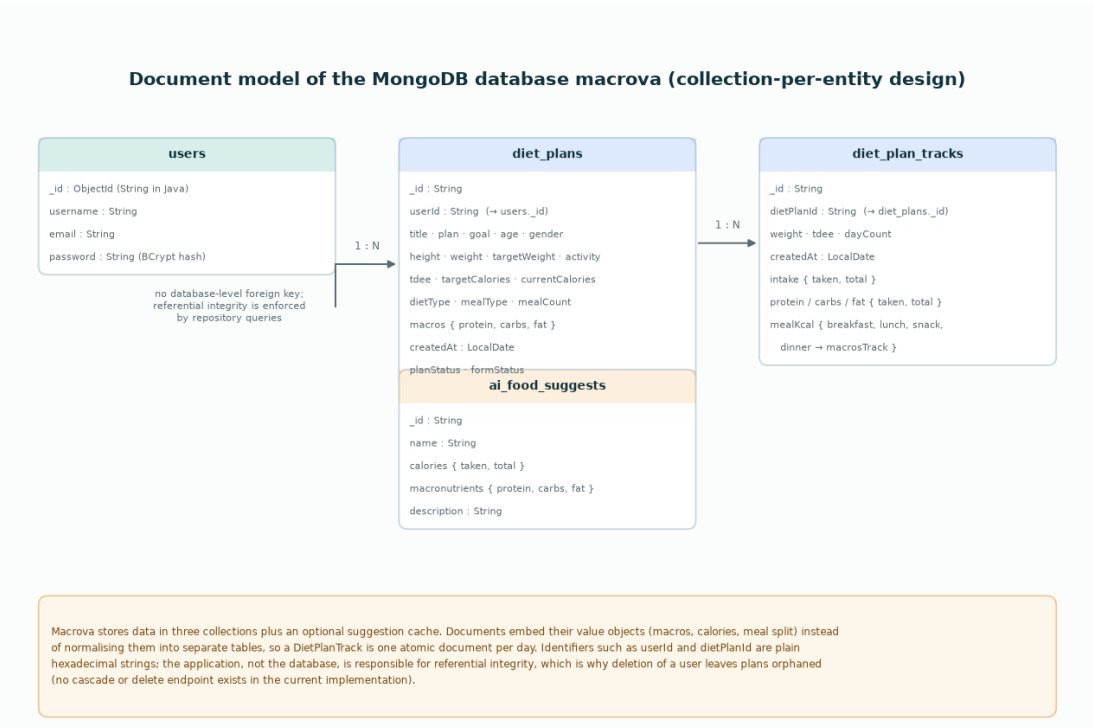


Figure 7.1: Entity-relationship view of the macrova database, showing the fields and references of each collection

Table 7.1: Collection inventory of the macrova database

Collection	Document class	Approximate size	Access pattern
users	User entity	about 250 bytes	Read by username on every request; written once at registration
diet_plans	DietPlan entity	about 700 bytes	Read by user and status on every wizard and dashboard call; written at each wizard step
diet_plan_tracks	DietPlanTrack entity	about 600 bytes	Read by plan and date for tracking and metrics; one document per tracked day
ai_food_suggests	AIFoodSuggest entity	about 300 bytes	The write method exists but is never invoked, so the collection is unused

7.2 The users Collection

The users collection is the smallest and the most security-sensitive. It is declared by the user entity with the collection name given explicitly and annotated with Lombok accessors. The class declares four fields and no behaviour; validation, uniqueness checks and hashing are performed by the user service before the document is written. The details are set out in Table 7.2.

Table 7.2: Data dictionary — users

Field	Type	Constraint or index	Purpose
_id	String	Primary key, generated by the driver when absent	Document identifier, referenced by the owner field of a plan
username	String	Unique in practice, enforced by a check before insert	Login name and the subject of both tokens
email	String	Unique in practice, enforced by a check before insert	Contact address captured at registration
password	String	Never null; the output of the encoder	BCrypt hash, sixty characters including the salt prefix

Two observations follow. The uniqueness of the username and the address is enforced by application checks rather than by a unique index, so two concurrent registrations with the same username could both succeed and a subsequent sign-in would resolve to one of the two documents depending on insertion order; adding the indexes of Table 7.8 would close that race without changing the application logic. Second, the document holds no display name, timestamp or account status, which is consistent with the single-role design but would have to be extended before an administrative interface could be built.

7.3 The diet_plans Collection

The plans collection stores the plan header. It is written at every wizard step, so its fields fall into four groups: those written by the first step, which capture identity and body measurements; those written by the second, which capture the goal and the de-

rived targets; those written by the third, which capture preferences and macros; and the status fields, which every step writes. The corresponding data appear in Table 7.3.

Table 7.3: Data dictionary — diet_plans

Field	Type	Written at	Purpose
_id	String	First save	Identifier, referenced by the plan field of a daily document
title	String	Body metrics	User-supplied plan name shown on the dashboard and in history
plan	Plan enumeration	Body metrics	Weight loss or weight gain; selects a deficit or a surplus
age	int	Body metrics	Years; the third term of the metabolic equation
gender	Gender enumeration	Body metrics	Male or female; selects the constant plus five or minus one hundred and sixty-one
height	double	Body metrics	Centimetres; the second term of the equation
weight	double	Body metrics	Kilograms at plan creation
todayWeight	double	Body metrics, then tracking	Kilograms at the latest update; shown as the current weight
targetWeight	double	Body metrics	Goal weight, used for display and for the duration choice
activity	Activity enumeration	Body metrics	One of five levels carrying multipliers 1.2, 1.375, 1.55, 1.725 and 1.9
tdee	double	Body metrics	Rounded basal rate multiplied by the activity factor, in kilocalories per day
goal	Goal enumeration	Goal selection	Mild, moderate or aggressive, carrying 15, 20 or 25 per cent
duration	int	Goal selection	Planned length in days, chosen by the user
targetCalories	double	Goal selection, replaced at activation	Daily energy target after the deficit or surplus and the rounding step
currentCalories	double	Meal preference	Energy equivalent of the chosen macro grams; copied into the target at activation
dietType	DietType enumeration	Meal preference	Balanced, high protein, low carbohydrate, low fat or custom
mealType	MealType enumeration	Meal preference	Non-vegetarian, vegetarian, vegan or eggetarian
macros	Macros object	Meal preference	Protein, carbohydrate and fat in grams; drives the per-meal split
mealCount	int	Meal preference	Two to five meals per day, recorded for display
eveningSnack	boolean	Meal preference	Whether an evening snack is included
createdAt	LocalDate	Activation	Activation date, from which the day count is computed
userId	String	Body metrics	Owner, resolved from the security context and never from the request
planStatus	PlanStatus enumeration	Every step	Pending at creation, active at activation, inactive when the duration is exceeded
formStatus	FormStatus enumeration	Every step	Body metrics, goal selection, meal preference, preview or completed

The document contains one redundancy that deserves comment. The target is computed twice: once when the goal is selected, from the expenditure and the deficit, and

once when the plan is activated, from the macronutrient grams chosen in the third step. The service overwrites the first value with the second, and the source carries a comment acknowledging that the intermediate field should be removed and the target used directly. The consequence is that a user who accepts the balanced preset sees a target very close to the deficit-derived value, while a user who edits the grams to an unusual split sees a target that reflects the edit rather than the deficit. The behaviour is documented rather than corrected, because it is deliberate in the current design: the macro split is the final authority on the energy target, since it is the last decision the user makes. The complete set of values is given in Table 7.4.

Table 7.4: Example document — diet_plans

Field	Value	Field	Value
_id	6547c1f0b8a1f64a9c2d1e03	title	Karthick's Fat Loss Plan
plan	WEIGHT_LOSS	age and gender	25, MALE
height and weight	175, 95	target and current weight	75, 94.15
activity	LIGHT	tdee	2645
goal and duration	MODERATE, 180 days	target and current calories	2152, 2152
dietType and mealType	BALANCED, NON_VEGETARIAN	macros	protein 161 g, carbs 215 g, fat 72 g
mealCount and snack	3, true	createdAt	2026-11-06
planStatus and formStatus	ACTIVE, COMPLETED	userId	6547c0d3b8a1f64a9c2d1df1

7.4 The diet_plan_tracks Collection

The tracking collection holds one document per tracked day. It is the only collection whose documents are created implicitly, by the read path rather than by an explicit user action: when the tracking or metrics endpoint discovers that no document exists for the current date, it creates one with the plan's current targets and zero logged intake. The same information is presented in tabular form in Table 7.5.

Table 7.5: Data dictionary — diet_plan_tracks

Field	Type	Purpose
_id	String	Identifier returned to the client and used by the single-document endpoint
dietPlanId	String	Owner plan, resolved by query rather than by a database reference
createdAt	LocalDate	The day the document represents, stored as a date at UTC midnight
dayCount	int	One-based index of the day within the plan, computed from the activation date
weight	double	Weight recorded on that day, initially the plan's weight
tdee	double	Snapshot of the plan's expenditure, for later comparison
intake	Calories object	Daily energy: the total from the plan's target and the taken value recomputed from the logged grams

Field	Type	Purpose
protein	Calories object	Daily protein allowance and the grams logged so far
carbs	Calories object	Daily carbohydrate allowance and the grams logged so far
fat	Calories object	Daily fat allowance and the grams logged so far
mealKcal	MealKcal object	Four macro-track objects, one per meal, each holding the per-meal protein, fat and carbohydrate allowance and the values logged against it

The arithmetic explains the structure. Each of the three daily macro objects holds a total, fixed at creation from the plan's macros, and a taken value, recomputed on every update as the sum of the four meal tracks. The intake object follows the same pattern with a conversion: its total is the plan's daily energy target, while its taken value is computed from the three macro taken values with the Atwater factors rather than summed from the meal tracks, so the two representations cannot drift apart. The details are set out in Table 7.6.

Table 7.6: Worked example — meal split and logged values for one day

Category	Share of the daily macros	Protein, total and taken	Carbohydrate, total and taken	Fat, total and taken
Breakfast	30 %	48, 45	65, 70	22, 25
Lunch	30 %	48, 45	65, 70	22, 25
Snack	15 %	24, 20	32, 30	11, 15
Dinner	25 %	40, 0	54, 0	18, 0
Daily document	100 %	161, 110	215, 170	72, 65
Derived energy	Atwater	110 × 4 = 440	170 × 4 = 680	65 × 9 = 585

The final row reproduces the arithmetic that the service performs. The three contributions sum to 1705 kilocalories, which is the value displayed as the consumed intake in the tracking screen of Figure 9.10, and which yields seventy-nine per cent of the 2152 kilocalorie target on the dashboard of Figure 9.9.

7.5 The ai_food_suggests Collection

The fourth collection is declared for completeness of the suggestion feature. Its document class holds an identifier, a name, a calorie object, a macro object and a description, which is exactly the schema the model is instructed to return. The repository is a plain repository with no additional query methods, and the service exposes a single method that saves a list of documents. No controller invokes that method, so the collection is created on first use but otherwise remains empty. The corresponding data appear in Table 7.7.

Table 7.7: Data dictionary — ai_food_suggests

Field	Type	Purpose
_id	String	Identifier, generated on save
name	String	Dish name as returned by the model
calories	Calories object	Model-reported energy value; the taken field is unused
macronutrients	Macros object	Model-reported protein, carbohydrate and fat in grams
description	String	One-paragraph description used as the body of a card

7.6 Indexing, Consistency and Integrity

The repository contains no explicit index definitions. MongoDB creates a unique index on the identifier of every collection automatically, and that is the only index that currently exists. The queries the application issues are therefore collection scans at present: the plan finder by user and status scans the plans collection, and the tracking finder by date and plan scans the tracking collection. At the scale of a single user with a few hundred documents the scans are imperceptible, but the indexes in Table 7.8 would be appropriate before the application is used by a group.

Table 7.8: Recommended indexes and the queries they would accelerate

Collection	Index	Query accelerated
users	username, unique	The username lookup performed on every request
users	email, unique	The existence check performed during registration
diet_plans	user identifier and plan status	The pending and active lookups performed at every wizard step
diet_plans	user identifier and creation date, descending	The history list ordered by creation date
diet_plan_tracks	plan identifier and date, descending	Today's document and the weekly range query
diet_plan_tracks	plan identifier and date, ascending	The day-by-day overview of a plan

Consistency is maintained by the service layer alone. There are no transactions, which is acceptable because the only multi-document operation is activation: it saves the plan and then creates the first document. If the second write failed, the user would hold an active plan with no first-day document, and the read path would repair the inconsistency on the next load because the tracking finder creates the missing document on demand. That self-healing property is why the absence of transactions does not produce a visible failure mode.

Referential integrity is not enforced. The plan identifier inside a daily document is resolved by query, and a dangling reference produces the behaviour the application already exhibits when a plan has been removed outside the application: the tracking endpoints return a not-found failure, which the client routes to the error page. No deletion endpoint exists in the delivered system, so the situation can arise only through direct manipulation of the database.

CHAPTER 8
REST INTERFACE AND CLIENT SERVICE LAYER

8.1 Conventions

The interface follows a small set of conventions consistently. Resource paths are kebab-case, and a noun is pluralised only where a collection is genuinely being addressed. Identifiers appear as path variables rather than as query parameters. Payloads are JSON in both directions. Success responses carry 200 with a body, and error responses carry 400, 401, 404 or 410 with a body that wraps the message in a single-element array. No endpoint uses 201 or 204, because the client does not need to distinguish creation from update and always consumes the returned document.

Authentication is required on every endpoint except registration and refresh, together with the sign-in path, which is mounted by the security filter rather than by a controller. The access token is read from the authorisation header with the bearer scheme. The refresh token is read from a cookie rather than from the header, which is a deliberate asymmetry: the access token is expected to be handled by client code and the refresh token is not.

8.2 Endpoint Catalogue

Table 8.1 lists every endpoint implemented in the repository with its request and response types. The table was produced by enumerating the mapping annotations on the four controller classes and cross-checking each entry against the client method that calls it. The entries with no client caller are marked as such, because a reviewer who counts endpoints will otherwise assume that all of them are in use.

Table 8.1: Complete REST endpoint catalogue

Method	Path	Request body	Response body	Client caller
POST	/user/register	Username, e-mail, password	User DTO	Authentication service, signup
POST	/authenticate	Username, password	Empty 200 with a cookie	Authentication service, login
POST	/user/refresh	None; the cookie is read	Access token DTO	Authentication service, refresh
POST	/user/logout	None	Empty 200 with a cleared cookie	Authentication service, logout
GET	/user/username	None	Username DTO	Authentication service, username

Method	Path	Request body	Response body	Client caller
GET	/diet-plan/form/status	None	Form status envelope	Wizard host, initialisation
POST	/diet-plan/form/status	Current form status	Form status envelope	Wizard host, backward navigation
POST	/diet-plan/form/body-metrics	Body metrics request	Body metrics response	Body metrics step
POST	/diet-plan/form/goal-selection	Goal selection request	Calorie estimation response	Goal selection step
POST	/diet-plan/form/meal-preference	Meal preference request	Plan	Meal preference step
POST	/diet-plan/form/create	None	Plan	Review step
GET	/diet-plan	None	The active plan	Tracking screen
GET	/diet-plan/all	None	List of plans	No client caller
GET	/diet-plan/{dietPlanId}	None	Plan	No client caller
GET	/diet-plan/history	None	List of plan history DTOs	History list
GET	/diet-plan/past-plan	None	Plan history DTO	Empty state
GET	/diet-plan/status	None	Boolean	Dashboard and tracking screen
GET	/diet-plan/active-plans	None	Boolean	No client caller
GET	/diet-plan/metrics	None	Metrics DTO	Dashboard
GET	/diet-plan/overview/{dietPlanId}	None	List of overview DTOs	Plan overview
GET	/diet-plan/meal-kcal/{id}	None	Meal calorie object	No client caller
GET	/diet-plan/track/{trackId}	None	Tracking document	Day overview
GET	/diet-plan/track/plan/{dietPlanId}	None	Tracking document	Tracking screen
POST	/diet-plan/add-weight/{dietPlanId}	A bare JSON number	Tracking document	Weight update
POST	/diet-plan/{category}/{dietPlanId}	Macro track	Tracking document	Meal update
POST	/ai-food-suggest	Four filters	List of suggestions	Suggestion screen

Four entries deserve a note because they are not reachable from the current client. The all-plans endpoint returns every plan without a projection, which is why the history screen uses the projection endpoint instead. The active-plans endpoint duplicates a boolean that the client obtains indirectly by loading the metrics and interpreting a not-found response. The meal-calorie endpoint exposes a computation that the tracking page performs implicitly when it renders the required grams. And the weight endpoint accepts a bare JSON number rather than an object, which binds correctly but is unusual, and is called out in Chapter 11.

8.3 Request and Response Examples

The following examples were produced against the running application and are reproduced with the identifiers and credentials replaced by representative values. They are included because a catalogue of paths does not convey the shape of the payloads, and

the shape is what a future client has to match.

Listing 8.1 — registration, request and response

```
POST /user/register
Content-Type: application/json

{ "username": "karthick", "email": "karthick@example.com", "password": "diet2026plan" }

HTTP/1.1 200 OK

{ "id": "6547c0d3b8a1f64a9c2d1df1", "username": "karthick",
  "email": "karthick@example.com" }
```

Listing 8.2 — sign-in followed by a token refresh

```
POST /authenticate

{ "username": "karthick", "password": "diet2026plan" }

HTTP/1.1 200 OK
Set-Cookie: Refresh-Token=eyJhbGciOiJIUzUxMiJ9...; HttpOnly; SameSite=Lax;
            Path=/; Max-Age=86400

POST /user/refresh
Cookie: Refresh-Token=eyJhbGciOiJIUzUxMiJ9...

HTTP/1.1 200 OK

{ "accessToken": "Bearer eyJhbGciOiJIUzUxMiJ9..." }
```

Listing 8.3 — first wizard step, request and response

```
POST /diet-plan/form/body-metrics
Authorization: Bearer eyJhbGciOiJIUzUxMiJ9...

{ "title": "Karthick's Fat Loss Plan", "plan": "WEIGHT_LOSS",
  "gender": "MALE", "age": 25, "height": 175.0, "weight": 95.0,
  "targetWeight": 75.0, "activity": "LIGHT" }

HTTP/1.1 200 OK

{ "tdee": 2645.0, "weight": 95.0, "targetWeight": 75.0, "plan": "WEIGHT_LOSS" }
```

Listing 8.4 — dashboard metrics, response

```
GET /diet-plan/metrics

HTTP/1.1 200 OK

{ "title": "Karthick's Fat Loss Plan", "taken": 79, "duration": 180,
  "currentDay": 4, "currentWeight": 94.15, "intake": 2152.0,
  "protein": 161.0, "carbs": 215.0, "fat": 72.0,
  "days": ["Fri", "Sat", "Sun", "Mon"],
  "weights": [95.0, 94.85, 94.6, 94.15] }
```

Listing 8.5 — meal update, request and truncated response

```
POST /diet-plan/breakfast/6547c1f0b8a1f64a9c2d1e03

{ "protein": { "taken": 45, "total": 0 },
  "fat":     { "taken": 25, "total": 0 },
  "carbs":   { "taken": 70, "total": 0 } }

HTTP/1.1 200 OK

{ "_id": "6547c9a2b8a1f64a9c2d1e55", "weight": 95.0, "tdee": 2645.0,
  "intake": { "taken": 1705.0, "total": 2152.0 },
  "protein": { "taken": 110.0, "total": 161.0 },
  "carbs": { "taken": 170.0, "total": 215.0 },
  "fat": { "taken": 65.0, "total": 72.0 },
  "mealKcal": { "breakfast": { ... }, "lunch": { ... },
                "snack": { ... }, "dinner": { ... } },
  "dayCount": 4, "createdAt": "2026-11-09",
  "dietPlanId": "6547c1f0b8a1f64a9c2d1e03" }
```

Two details of the last listing are worth pointing out. The request sends totals of zero in each calorie object, because the service ignores the totals and reads only the taken field, so the client could have sent a smaller payload. And the response returns the entire document including all four meal tracks, which is what allows the page to re-render every card from a single response.

8.4 Error Responses

Errors reach the client through two paths. Exceptions raised inside a controller or a service are caught by the controller advice and rendered as a JSON object with a single-element error array. Exceptions raised inside the filter chain, including token verification failures, are caught by the chain's exception filter and rendered as plain text with a 401 status. The client treats both as failures and either displays a toast or navigates, so the inconsistency in media type has no functional consequence at present, but it is recorded as a defect of the contract. The complete set of values is given in Table 8.2.

Table 8.2: Error responses produced by the current implementation

Trigger	Handled by	Status	Body
Unknown plan identifier	Controller advice	404	An error array with a plan message
Unknown tracking identifier	Controller advice	404	An error array with a tracking message
No pending plan when a step is submitted	Controller advice	400	An error array stating that no pending plan was found
Invalid meal category in the path	Controller advice	400	An error array naming the invalid category
Duplicate username or e-mail address	Controller advice	400	An error array with the corresponding message
Malformed JSON from the model provider	Controller advice	410	An error array carrying the parser message
Missing refresh cookie or invalid refresh token	Controller advice	401	An error array with a token message
Expired access token	Chain exception filter	401	Plain text with an expiry message

Trigger	Handled by	Status	Body
Invalid token signature	Chain exception filter	401	Plain text with a verification failure
Wrong password or unknown username	Authentication filter	401	Plain text with a credentials failure
Any other unhandled runtime exception	Chain exception filter	400	Plain text with a bad-request message

8.5 Client Service Layer

The client mirrors the service surface with four injectable services, all provided at the root of the application and therefore singletons for the lifetime of the page. Table 8.3 lists their methods, the endpoint each calls and the component that calls it.

Table 8.3: Client-side service methods

Service	Method	Endpoint	Principal caller
Authentication service	signup	POST /user/register	Signup screen
Authentication service	login	POST /authenticate	Login screen
Authentication service	logout	POST /user/logout	Header
Authentication service	request refresh	POST /user/refresh	Authentication check
Authentication service	is authenticated	POST /user/refresh when needed	Route guard, token interceptor
Authentication service	username	GET /user/username	Header, through a signal
Token service	is logged in	Local only, through jwt-decode	Authentication check
Plan service	form status	GET /diet-plan/form/status	Wizard host
Plan service	form details for a status	POST /diet-plan/form/status	Wizard host, backward navigation
Plan service	body metrics	POST /diet-plan/form/body-metrics	Body metrics step
Plan service	goal selection	POST /diet-plan/form/goal-selection	Goal selection step
Plan service	meal preference	POST /diet-plan/form/meal-preference	Meal preference step
Plan service	activate	POST /diet-plan/form/create	Review step
Plan service	duration reached	GET /diet-plan/status	Dashboard, tracking screen
Plan service	active plan	GET /diet-plan	Tracking screen
Plan service	track of a plan	GET /diet-plan/track/plan/{id}	Tracking screen
Plan service	update macros	POST /diet-plan/{category}/{id}	Meal update
Plan service	add weight	POST /diet-plan/add-weight/{id}	Weight update
Plan service	metrics	GET /diet-plan/metrics	Dashboard
Plan service	history	GET /diet-plan/history	History list

Service	Method	Endpoint	Principal caller
Plan service	past plan	GET /diet-plan/past-plan	Empty state
Plan service	overview of a plan	GET /diet-plan/overview/{id}	Plan overview
Plan service	track by identifier	GET /diet-plan/track/{id}	Day overview
Plan service	update plan	PUT /diet-plan/update	Declared, with no caller and no server mapping
Suggestion service	suggest foods	POST /ai-food-suggest	Suggestion screen

The table contains two inconsistencies that a careful reader will notice. The client declares a plan-update method that issues a PUT to a path the service does not map, so a call would be rejected by the security chain rather than answered with a 404. And the meal-update method lowercases the category before sending it, which is redundant because the service lowercases the value again when it selects the track. Both observations are recorded in the verification matrix rather than silently corrected, because the report documents the repository as it stands.

8.6 Interceptors and Cross-Cutting Client Concerns

The base-prefix interceptor prefixes every request whose URL does not already begin with a scheme with the environment base URL, which is the string /api. The token interceptor is more selective: it returns the request unchanged when the URL contains the sign-in, registration or refresh paths, and otherwise consults the authentication service, which may itself issue a refresh request before the original request proceeds. The use of switchMap ensures that the refresh completes before the original request is dispatched, and the credentials flag ensures that the refresh cookie travels with the request. Because the prefix is applied first, the token interceptor sees the final URL and can recognise the public paths by substring.

The mechanism has one weakness worth stating. Because the authentication check issues a refresh whenever the in-memory token is missing, a burst of parallel requests on a freshly loaded page can issue several refresh calls, and each will succeed because the cookie is not single-use. The consequence is redundant server work rather than incorrect behaviour, but a shared refresh observable would be the conventional remedy and is proposed in Chapter 12.

CHAPTER 9
USER INTERFACE DESIGN AND APPLICATION SCREENS

9.1 Design Objectives

The interface was designed against four objectives that follow from the analysis in Chapter 3. First, every screen must state its purpose in a single line beneath the heading, so that a first-time user does not have to infer what the screen is for. Second, every screen must have exactly one primary action, rendered in the brand colour and at full width where the action concludes a form. Third, numeric feedback must be immediate: an update to a logged value re-renders the calorie meter from the response, and the dashboard recomputes the percentage from stored documents rather than from client-side arithmetic. Fourth, the application must never present an empty layout; a missing plan, an expired plan and an unknown route each resolve to a designed screen.

The screens are presented in the order in which a new user encounters them, which is also the order of the numbered screenshot files captured during the demonstration. Every screenshot in this chapter was taken in a browser window of 1921 by 978 pixels; the browser chrome and the empty canvas around the content have been trimmed from each capture, and every figure preserves the aspect ratio of the original.

9.2 Registration and Authentication Screens

9.2.1 Create Account Screen

The signup screen presents three fields, a username, an e-mail address and a password, inside a single card centred in the viewport, with the brand mark in the header and a log-in button offered for a returning user. The card carries the heading 'Create account' and the supporting line 'Start tracking your nutrition today', which is the pattern of heading plus one-line purpose that recurs across the application. Validation is client-side at this point: the submit handler checks that the username is between three and twenty characters, contains only letters and digits and does not begin with a digit, that the address matches a simple pattern, and that the password is between eight and twenty characters. A failure displays a toast and returns without issuing a request, which is why the screen appears unchanged in the capture. The screen is reproduced in Figure 9.1.

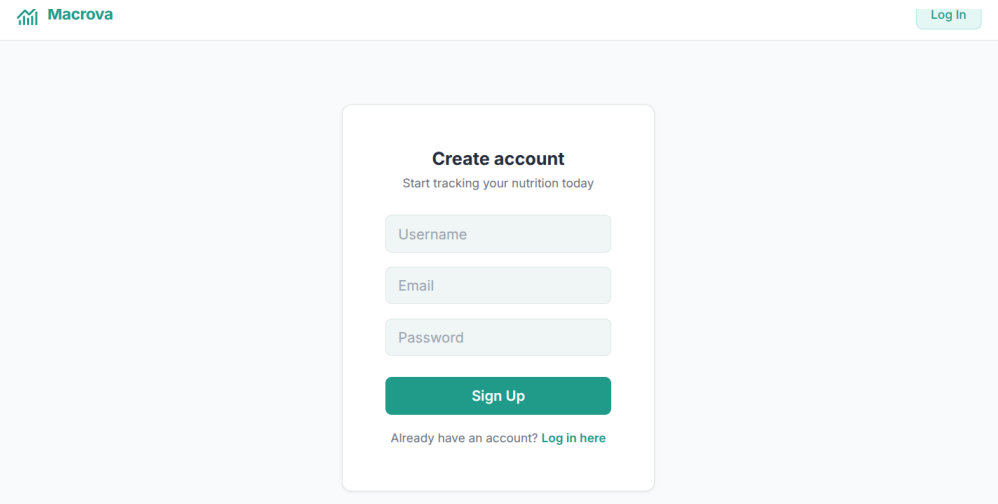


Figure 9.1: Create account screen with the three-field registration form

9.2.2 Log In Screen

The login screen is the mirror image of the registration screen: two fields, a single full-width primary button and a link to registration. On initialisation the component calls the authentication check, which means that a user who already holds a valid session is redirected to the dashboard without ever seeing the form. A failed attempt produces a toast stating that the credentials were rejected, using the same message for an unknown username and a wrong password in order not to disclose which accounts exist. The complete view is captured in Figure 9.2.

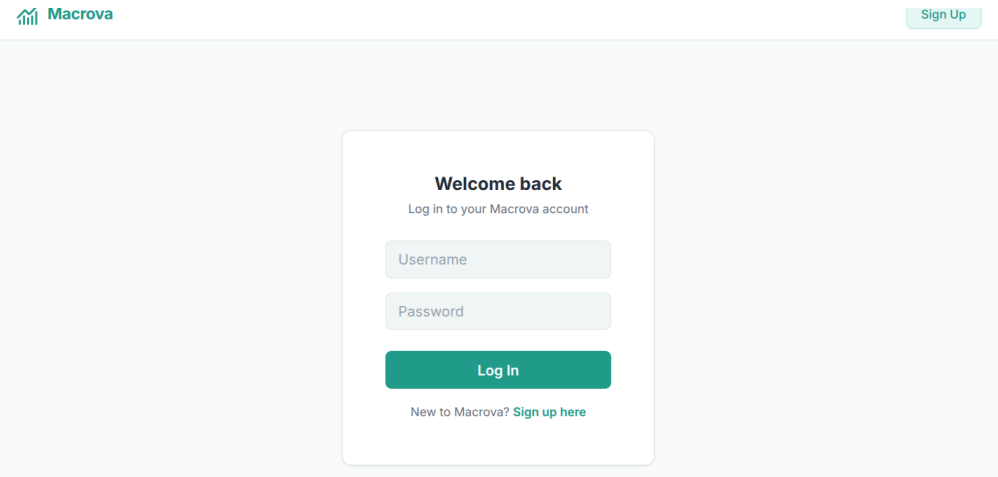


Figure 9.2: Log in screen with the credential form and the registration link

9.3 Empty State and Plan Initiation

9.3.1 No Active Diet Plan Screen

A user who has authenticated but holds no active plan is routed to the no-active-plan screen. The screen consists of a small illustration, the heading 'No Active Diet Plan', two sentences of explanation and a single primary action that opens the wizard. The component also loads the most recent plan, so that a returning user can be offered the

option of reviewing the previous one; the capture shows the state before any plan has ever been created, in which that option is legitimately absent. The rendered screen appears in Figure 9.3.

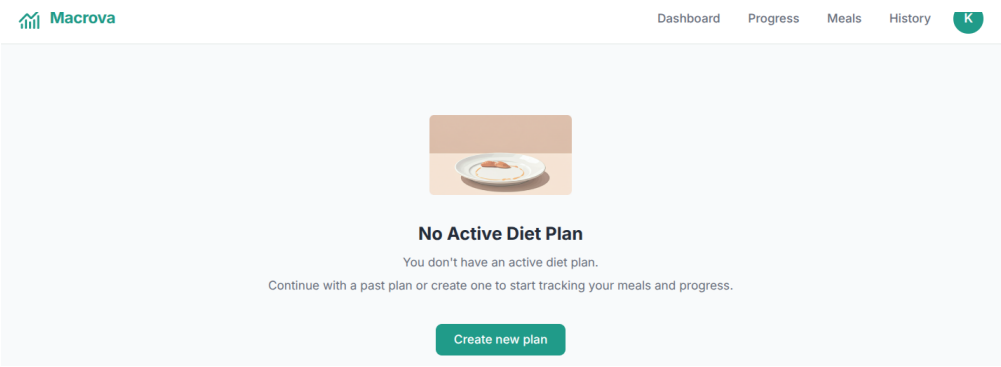


Figure 9.3: Empty state shown when the authenticated user has no active diet plan

9.4 Plan Construction Wizard

The wizard occupies a single route and renders one of four steps beneath a horizontal step indicator. The indicator consists of four numbered circles connected by rules; the current step is filled with the primary colour, completed steps are filled with a tint and subsequent steps are outlined. Because the current step is derived from the server's form status rather than from a client variable, the indicator is always consistent with what is stored, including after a page reload.

9.4.1 Step 1 — Body Metrics

The first step collects the eight values that determine the energy estimate. The plan name is a free-text field; the plan type, the sex and the activity level are pill selectors; age, height, current weight and target weight are numeric inputs whose plausible range is stated inside the label. The range statement is not decoration: the same bounds are enforced in the submit handler, which is why the capture shows ranges for age, height and current weight. The primary action is the only button on the card and is rendered at full width in the brand colour. The screen is reproduced in Figure 9.4.

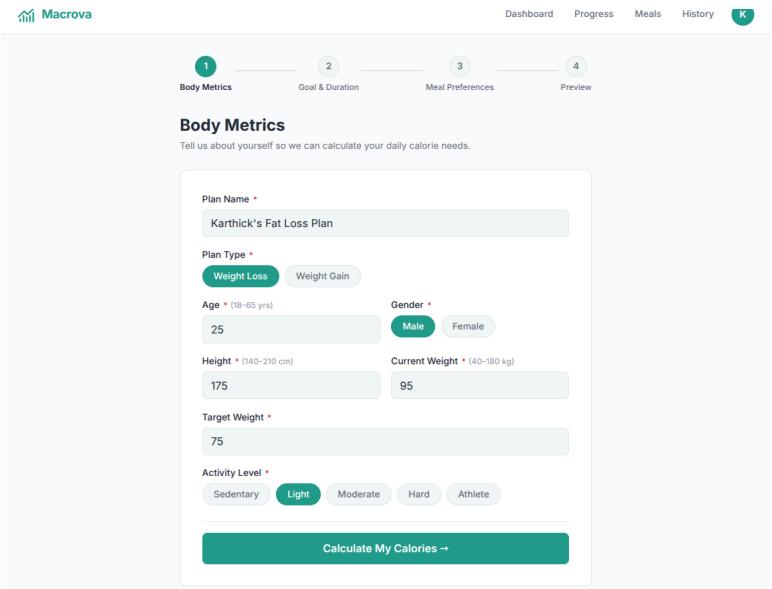


Figure 9.4: Wizard step 1 — body metrics, plan type and activity selection

9.4.2 Step 2 — Goal and Duration

The second step presents the energy estimate that the server has just computed and asks for the intensity of the intervention. A summary strip at the top repeats the daily energy need, the current weight and the target weight, which anchors the three choices that follow. Each intensity is a full-width selectable card with a title, a one-sentence explanation, a badge stating the character of the option and the percentage that it applies. The selected card receives a tinted background and a coloured border. Duration is a second pill group with three values, and the primary action advances to the third step.

One detail of this screen is inaccurate and is recorded rather than corrected. The card for the mild intensity is labelled as a ten per cent deficit, while the enumeration that the service applies carries fifteen per cent. The label is hard-coded presentation text and the arithmetic is correct, so a user who selects the mild option receives a slightly larger deficit than the screen promises. The finding appears again in Section 11.3, and the correction is to render the label from the enumeration value. The complete view is captured in Figure 9.5.

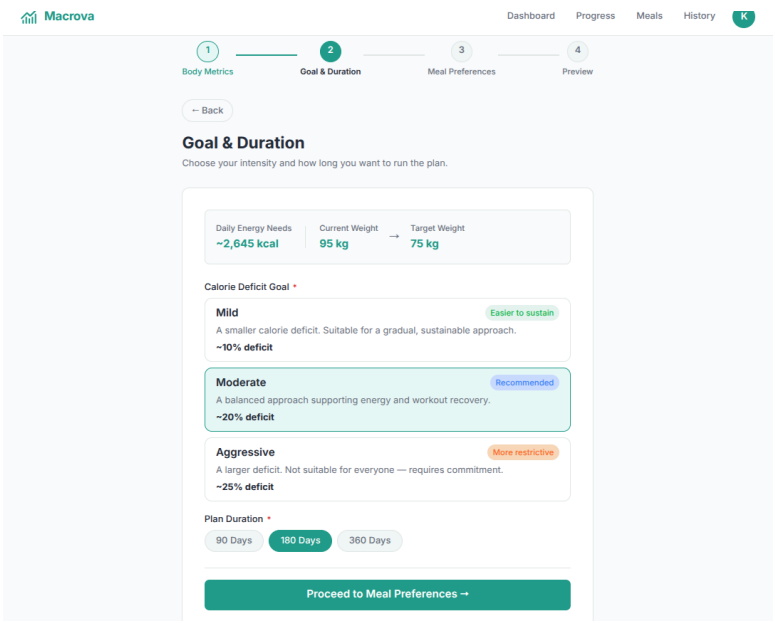


Figure 9.5: Wizard step 2 — goal intensity, energy summary and plan duration

9.4.3 Step 3 — Meal Preferences

The third step converts the energy target into grams. A three-column summary strip shows the current calories, the target calories and the maintenance requirement, which makes the size of the deficit visible at a glance. Below it, a row of five tabs selects a macronutrient preset; the selected preset populates three sliders, one for each macronutrient, whose current values are displayed as grams on the right. Moving a slider switches the preset to custom and recalculates the energy total from the three gram values. The remaining controls are a diet-type group, a meal-count group and a yes-or-no group for the evening snack.

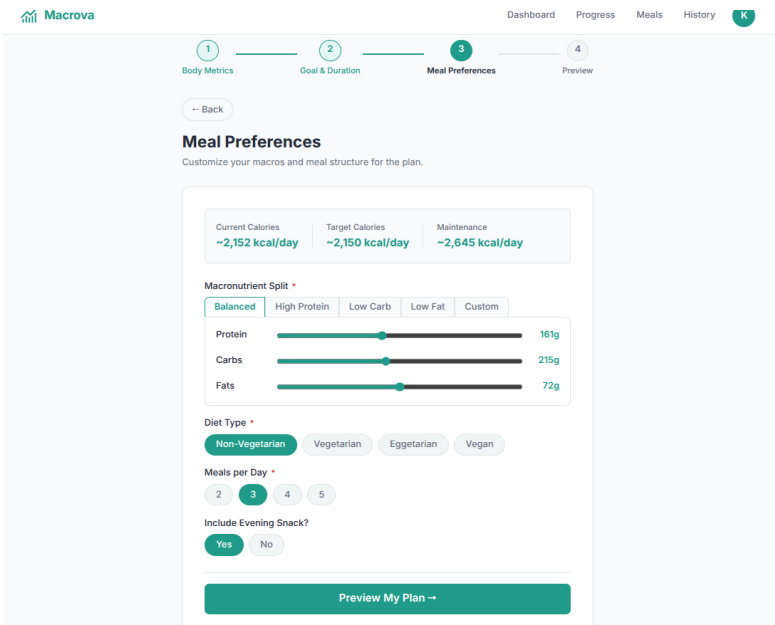


Figure 9.6: Wizard step 3 — macronutrient split, diet type and meal structure

9.4.4 Step 4 — Review and Confirmation

The final step renders the complete plan as a read-only summary organised in three labelled sections, one for the body metrics, one for the goal and calories and one for the meal preferences, with a plan-type badge in the corner and a row of three macro chips at the bottom. The screen exists because it is the last opportunity to detect an input error before activation and because the document it displays is the document that will be stored. The single primary action posts to the activation endpoint, and the host component navigates to the dashboard on success. The rendered screen appears in Figure 9.7.

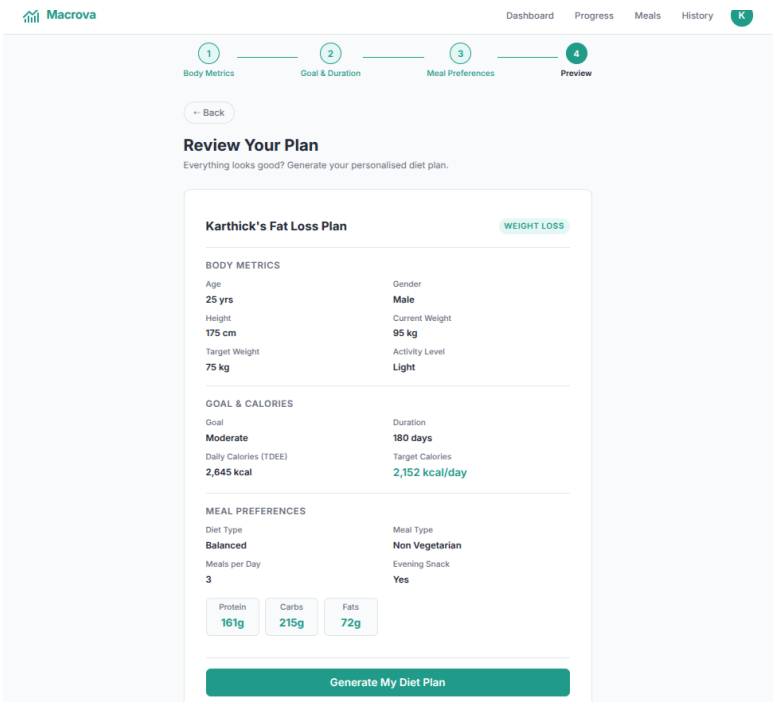


Figure 9.7: Wizard step 4 — review of body metrics, goal, calories and macro split

9.5 Dashboard

The dashboard is the landing page after authentication and the screen the user sees most often. Its layout is a vertical stack of three regions. The goal region shows the percentage of the daily target achieved and a horizontal meter whose fill is capped at one hundred per cent. Two rows of three metric cards follow: the first row shows the streak as the current day over the total duration, the current weight and the daily energy target; the second row shows the protein, carbohydrate and fat targets. The final region is the weight trend chart, which plots the recorded weights against the abbreviated weekday names of the sampled days.

Figures 9.8 and 9.9 show the same screen at two moments. In the first, taken immediately after the plan was activated, the streak stands at one of one hundred and eighty days, the chart holds a single observation, and the goal meter is empty because no in-

take has yet been logged. In the second, taken after four days of logging, the streak reads four, the current weight has fallen from ninety-five to ninety-four point one five kilograms, the goal meter is at seventy-nine per cent and the chart shows a monotonic decline across four observations. The pair is included because it demonstrates the feedback loop described in Section 2.2.5 rather better than either capture alone. The screen is reproduced in Figure 9.8.

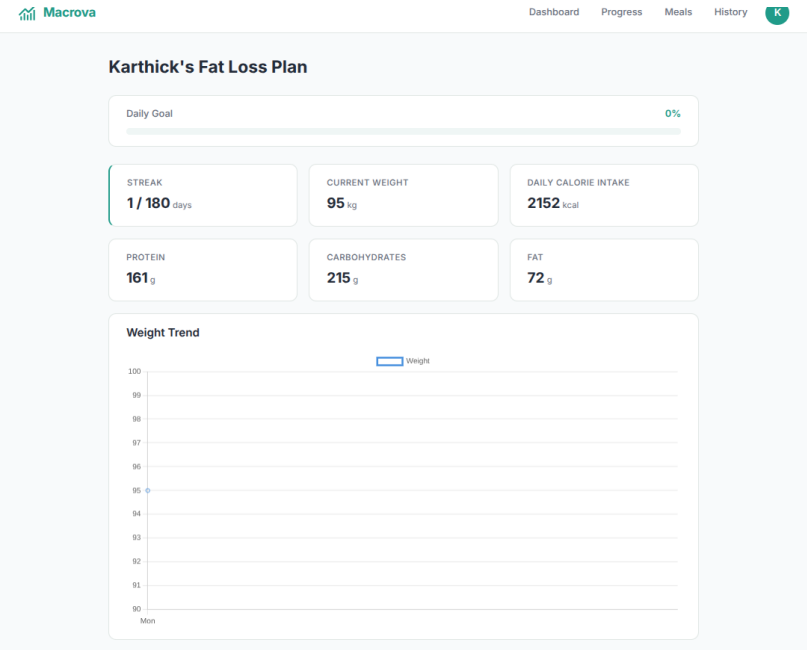


Figure 9.8: Dashboard on the first day of an activated plan — initial state

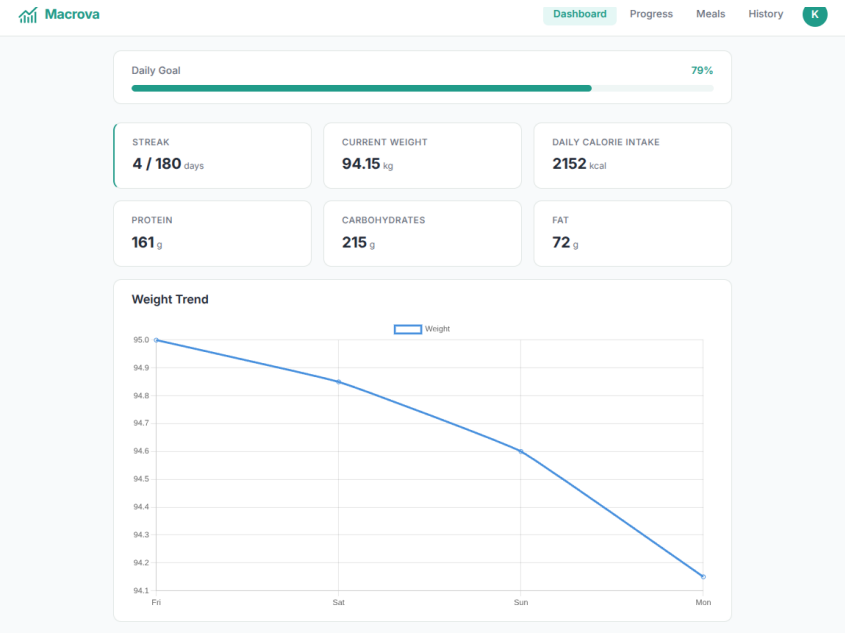


Figure 9.9: Dashboard after four days of logging — goal progress and weight trend

9.6 Daily Tracking Screen

The tracking screen, reached through the progress link in the header, opens with the daily summary card. That card contains a read-only day number, an editable current weight and a button that commits the weight, followed by the calorie meter, which displays the consumed energy against the daily target. Below the summary, four meal cards are rendered by a single template instantiated once per category. Each card carries the category name, three numeric inputs whose labels state the required grams for that meal, and a button that commits the three values for that category only.

The capture shows partial adherence, which is the ordinary case: breakfast and lunch have been logged at seventy grams of carbohydrate, forty-five of protein and twenty-five of fat, the snack at thirty, twenty and fifteen, and dinner has not yet been logged. The meter reflects exactly that, and the server's response to the last update is what produced the numbers on the screen. The three required figures printed in the input labels are the per-meal split of the plan's macros, which is the property that makes the screen a plan rather than a diary.

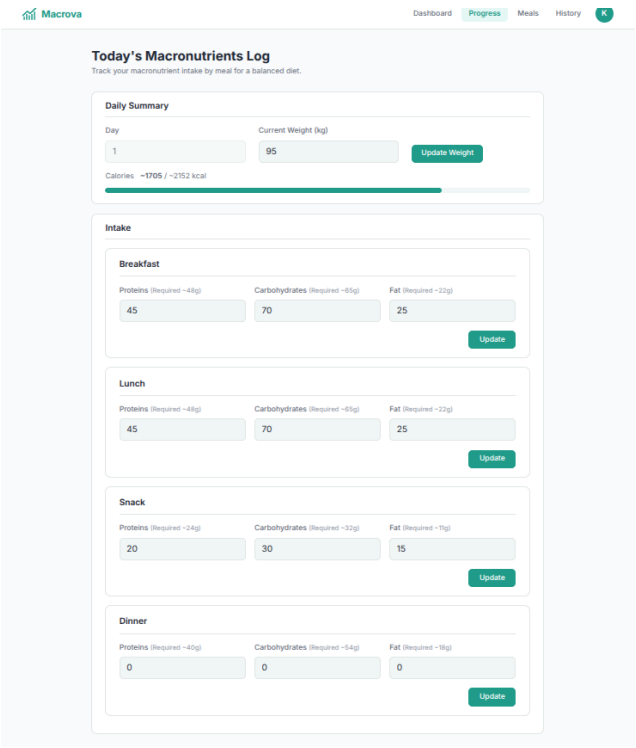


Figure 9.10: Daily tracking screen with the summary card and the four editable meal cards

9.7 AI Food Suggestion Screen

The suggestion screen presents four filter rows, one each for the meal, the meal type, the diet and the cuisine, rendered as rows of pill buttons with a single active option, followed by a primary action that requests the suggestions. Results are rendered as an ordered list of cards, each carrying a rank, a dish name, a descriptive paragraph and a footer with the reported energy and the three macronutrient values. The screen has

three states beyond the initial prompt: a loading indicator, the results list and an error message, all driven by two boolean flags in the component.

The capture shows five Indian vegetarian dinner suggestions for a balanced diet, each under the allowance that the server computed for the dinner category of the active plan. A caveat applies to every number in this figure: the values are generated by the model and are not verified against a food-composition database, as documented in Sections 6.4 and 11.4. The complete view is captured in Figure 9.11.

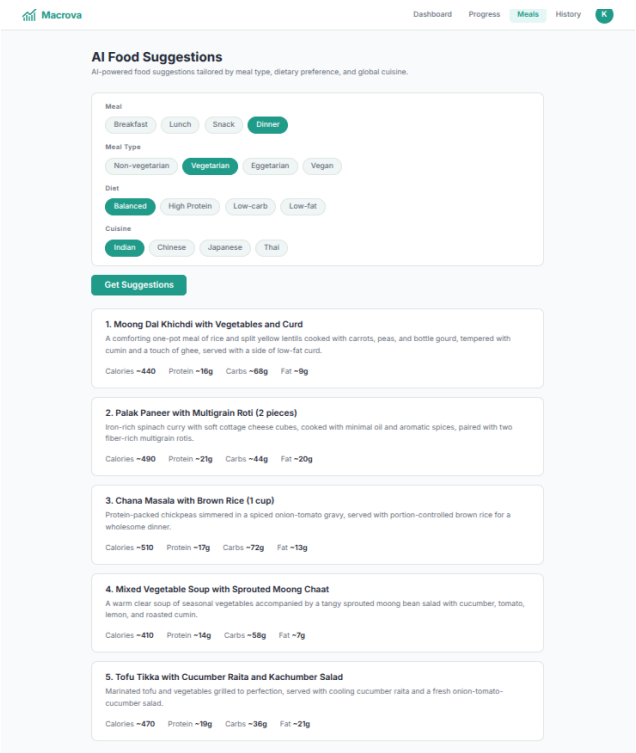


Figure 9.11: AI food suggestion screen with four filters and five generated suggestion cards

9.8 Plan History and Day-by-Day Review

9.8.1 Diet Plans History

The history list is a vertical sequence of rows, each of which carries the plan title, a status badge that distinguishes an active plan from an inactive one, a metadata line with the plan type, the weight transition and the duration, and a chevron that opens the day-by-day view. The capture shows three plans created in sequence, of which only the most recent is active, which illustrates the lifecycle described in Section 4.5.4: the earlier plans were completed or superseded and are retained with the inactive status rather than deleted. The rendered screen appears in Figure 9.12.

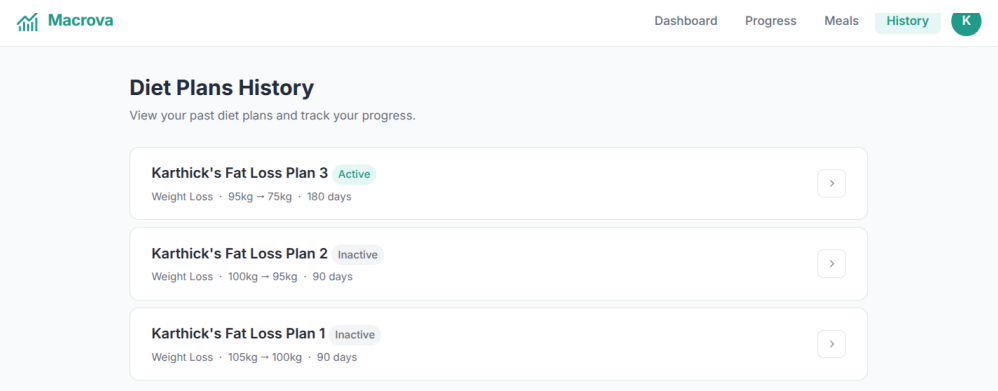


Figure 9.12: Diet plans history with status badges and weight transitions

9.8.2 Past Tracking Entries

Selecting a plan opens the list of its daily documents, ordered by date descending. Each entry shows the date and a metadata line with the weight recorded on that day, the plan's expenditure and the energy consumed, followed by a second line with the three macronutrient totals. The card therefore reads as a comparison between what the plan allowed and what the user actually logged, placed beside the weight that was current when the entry was recorded. In the capture the expenditure is 2645 and the consumed energy is 1705 on every one of the four days, because the demonstration logged the same values on each day; the weight column remains at the initial 95 kilograms because the weight update was applied to the current day only, which is why the dashboard of Figure 9.9 reports 94.15 kilograms as the current weight while the stored days still carry the earlier figure. The screen is reproduced in Figure 9.13.

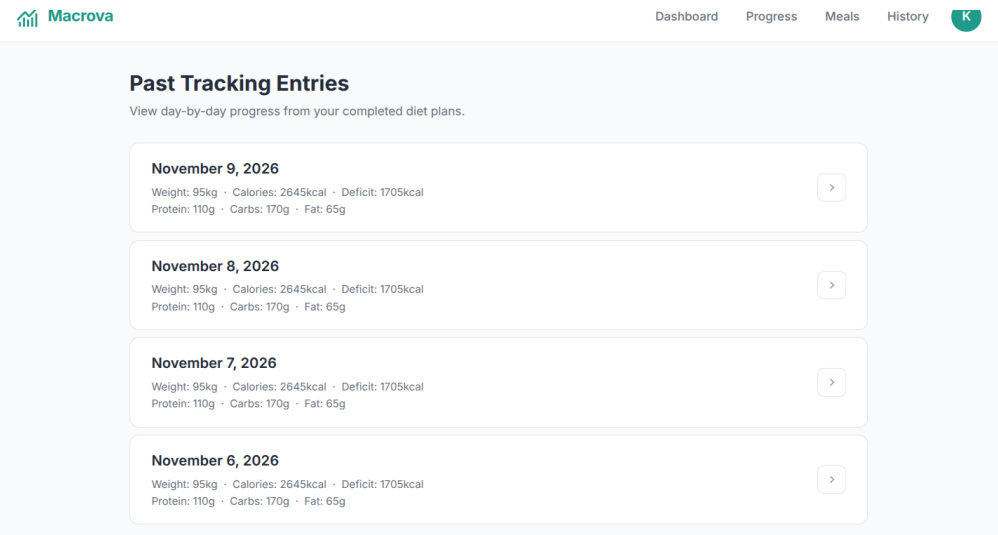


Figure 9.13: Day-by-day tracking entries of a single plan

9.8.3 Read-Only Review of a Stored Day

The final history screen shows one stored document exactly as it was recorded. It reproduces the daily summary and the four meal cards, but the inputs are disabled and no update button is rendered, which enforces the principle that history is a record

rather than a workspace. The capture shows the values stored for the ninth of November 2026, which are the values that the tracking screen of Figure 9.10 submitted, presented without the editing affordances. The heading is built from the stored date, which is why it reads as a possessive construction rather than as a fixed title; that is a presentation detail a future revision should reformat. The complete view is captured in Figure 9.14.

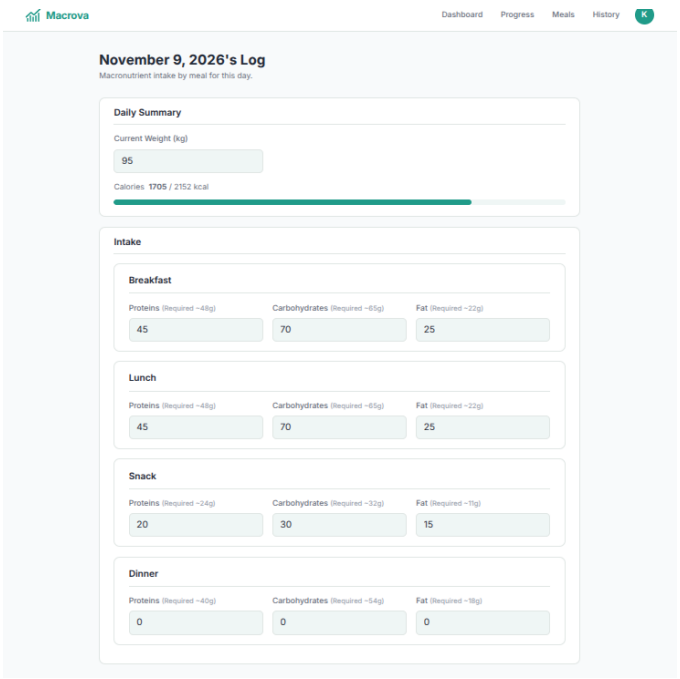


Figure 9.14: Read-only review of the macronutrient log stored for a single day

9.9 Error and Fallback Screens

The not-found screen is rendered for any route that does not match the route table. It presents a large numeral in a tint of the brand colour, the heading 'Page not found', a one-line explanation and a single primary action that returns the user to the dashboard. The header remains visible and functional, so the screen is a recoverable state rather than a dead end. The same component is reached programmatically when a request resolves to a not-found response, which is why the numeral is large: it doubles as an error signal in that case. The rendered screen appears in Figure 9.15.

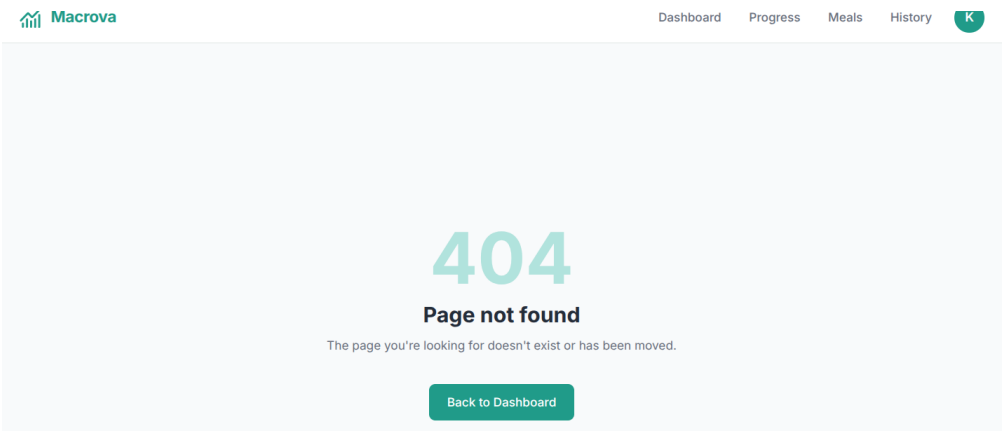


Figure 9.15: Not-found screen rendered for an unmatched route

The two fallback screens are the smallest in the application and the ones most often omitted from a student project, which is why they are reproduced here. Their value is not aesthetic. A user who has completed a plan and returns a month later must be told why the dashboard is no longer available rather than shown an empty shell, and a user who mistypes a route must be offered a way back rather than left with a blank page. Both states are reachable in the delivered build: the empty state is what a reviewer sees before any plan has been created, and the not-found screen is what appears when the address is edited by hand.

9.10 Consistency Observations

Reviewing the fifteen screens together reveals a consistent visual language and three inconsistencies that a future revision should correct. The consistent elements are the card surface with an eight-pixel radius and a one-pixel border, the pill selector, the primary button, the heading-plus-subtitle pattern, and the metric card with its upper-case label and inline unit. The inconsistencies are as follows. The authentication screens use a single centred column while the wizard uses a wider two-column grid, so the card widths differ between the two contexts. The dashboard uses the plan title as the page heading, whereas every other screen uses a fixed heading and displays the plan title as data. And the history screens use a chevron affordance that appears nowhere else in the application. The same information is presented in tabular form in Table 9.1.

Table 9.1: Screen inventory: route, data source and primary action of each screen

Figure	Screen	Route	Data source	Primary action
9.1	Create account	/signup	None	Sign up
9.2	Log in	/login	None	Log in
9.3	No active plan	/no-active-plan	Past plan projection	Create new plan
9.4	Body metrics	/diet-plan-form	Form status	Calculate my calories
9.5	Goal and duration	/diet-plan-form	Goal selection	Proceed
9.6	Meal preferences	/diet-plan-form	Meal preference	Preview my plan
9.7	Review	/diet-plan-form	Plan activation	Generate my diet plan
9.8	Dashboard, day 1	/dashboard	Metrics	None; navigation only
9.9	Dashboard, day 4	/dashboard	Metrics	None; navigation only
9.10	Daily tracking	/progress	Tracking document	Update, per meal
9.11	AI suggestions	/meals	Suggestion endpoint	Get suggestions
9.12	Plan history	/history	History projection	Open a plan
9.13	Past entries	/history/:id	Overview projection	Open a day
9.14	Stored day	/history/track/:id	Tracking document	None; read-only
9.15	Not found	/not-found	None	Back to dashboard

CHAPTER 10
ALGORITHMS AND IMPLEMENTATION DETAILS

10.1 Overview

This chapter documents the six algorithms on which the correctness of the application depends. For each one the mathematical definition is stated first, the implementation is then reproduced from the repository and explained, and the observable consequences are recorded against the worked example that runs through the report: a twenty-five-year-old man, 175 centimetres tall and 95 kilograms at the start of the plan, with a target weight of 75 kilograms, a light activity level and a moderate weight-loss goal over 180 days.

10.2 Basal Metabolic Rate — Mifflin–St Jeor

Basal metabolic rate is estimated with the Mifflin–St Jeor equation, which expresses resting energy expenditure as a linear function of mass, height and age with a sex-specific constant. Writing w for weight in kilograms, h for height in centimetres and a for age in years, the equation is ten w plus six point two five h minus five a plus five for men, and the same expression minus one hundred and sixty-one for women. The two constants differ by one hundred and sixty-six kilocalories, which is the equation's model of the mean difference in body composition between the sexes.

Listing 10.1 — the basal rate method

```
public static double findBMR(
  int age, double weight, double height, Gender gender
) {
  return (
    10 * weight + 6.25 * height - 5 * age
      + (gender == Gender.MALE ? 5 : -161)
  );
}
```

The implementation is a direct transcription and introduces no rounding, which is deliberate: the value is carried forward as a double and rounded once, when the expenditure is written into the plan. For the worked example the basal rate is 950 plus 1093.75 minus 125 plus 5, which is 1923.75 kilocalories per day.

10.3 Total Daily Energy Expenditure

Total daily energy expenditure multiplies the basal rate by a physical-activity factor. Macrova stores the five factors as the constructor argument of an enumeration, so that each multiplier is a property of a named constant rather than a literal inside the arith-

metic. The details are set out in Table 10.1.

Listing 10.2 — the activity enumeration and the expenditure method

```
public enum Activity {
  SEDENTARY(1.2), LIGHT(1.375), MODERATE(1.55), HARD(1.725), ATHLETE(1.9);
  private final double value;
  Activity(double value) { this.value = value; }
  public double getValue() { return value; }
}

public static double findTDEE(
  int age, double weight, double height, Gender gender, Activity activity
) {
  return findBMR(age, weight, height, gender) * activity.getValue();
}
```

Table 10.1: Activity factors and the expenditure they produce for the worked example

Activity level	Multiplier	Expenditure at a basal rate of 1923.75	Rounded value stored
Sedentary	1.2	2308.50	2309
Light	1.375	2645.16	2645
Moderate	1.55	2981.81	2982
Hard	1.725	3318.47	3318
Athlete	1.9	3655.13	3655

The rounding is performed by the caller: the first wizard step stores the rounded expenditure, which is why the final column of the table shows whole numbers. The rounding is to the nearest kilocalorie and is applied once, so every later stage works from the rounded value and a recomputation from the stored value reproduces the stored target.

10.4 Target Calories — Deficit and Surplus

The daily target is obtained by applying an intensity to the expenditure. For a weight-loss plan the intensity is subtracted; for a weight-gain plan an effective surplus is added; and in both cases the result is rounded to a value ending in fifty or a hundred so that it reads as a prescription rather than as a measurement. Writing T for the expenditure and p for the intensity in per cent, the two rules are T minus pT over one hundred for a deficit and T plus (p minus ten) T over one hundred for a surplus.

The ten-point subtraction in the surplus rule is not a defect but a deliberate moderation. A gross surplus of twenty-five per cent would add a quarter of the expenditure on top of an already generous maintenance figure; subtracting ten points makes the effective surplus fifteen per cent, which is the upper bound that the sports-nutrition literature generally recommends for a lean gain. The rule is stated in the comment above the method and reproduced in the listing below, together with the rounding helper it depends on. The corresponding data appear in Table 10.2.

Listing 10.3 — percentage, rounding, deficit and surplus

```
public static long percentageOf(double value, int percent) {
  return Math.round((value * percent) / 100);
}

public static long roundCalories(long kcal) {
  long round = (kcal / 100) * 100;
  long difference = kcal - round;
  return round + (difference &gt; 50 ? 100 : (difference &gt; 0 ? 50 : 0));
}

public static long findDeficit(double tdee, int percent) {
  return roundCalories(Math.round(tdee - percentageOf(tdee, percent)));
}

public static long findSurplus(double tdee, int percent) {
  return roundCalories(Math.round(tdee) + percentageOf(tdee, percent - 10));
}
```

Table 10.2: Target calories for the worked example at each intensity

Goal	Intensity	Weight-loss target	Weight-gain target
Mild	15 %	2645 − 397 = 2248, rounded to 2250	2645 + 132 = 2777, rounded to 2800
Moderate	20 %	2645 − 529 = 2116, rounded to 2150	2645 + 265 = 2910, rounded to 2900
Aggressive	25 %	2645 − 661 = 1984, rounded to 2000	2645 + 397 = 3042, rounded to 3050

The moderate weight-loss row reproduces the value shown in the second wizard step, which displays 2150 kilocalories per day after the rounding step. The plan document, however, stores 2152, because activation replaces the deficit-derived target with the energy equivalent of the chosen macro grams, as discussed in Section 7.3. The two figures differ by two kilocalories here because the balanced preset is close to the deficit split; a custom split would diverge much further, and that divergence is the intended behaviour.

10.5 Macronutrient Presets and Bands

The energy target is translated into grams with the four presets recorded in Table 10.3. Each preset allocates a percentage of the target to each macronutrient and divides by the energy density of that macronutrient, four kilocalories per gram for protein and carbohydrate and nine for fat. The same three densities are used in the reverse direction when logged intake is converted back into energy, which guarantees that a preset and its energy equivalent agree.

Table 10.3: Macronutrient presets and their gram values for a 2150 kilocalorie target

Preset	Protein %	Carbs %	Fat %	Protein g	Carbs g	Fat g
Balanced	30	40	30	161	215	72
High protein	40	35	25	215	188	60
Low carbohydrate	35	25	40	188	134	96
Low fat	30	50	20	161	269	48

The presets are accompanied by three minimum and maximum bands, one per macronutrient, which the interface uses to initialise the custom sliders. The protein band runs from fifteen to fifty per cent of the target, the carbohydrate band from twenty to sixty-five and the fat band from twenty to forty. The bands are advisory: the service accepts any gram value the client submits, so a user who drags a slider beyond a band is not prevented from doing so, which is consistent with the principle that the macro split is the user's decision.

Listing 10.4 — preset construction

```
public static double findProtein(double deficit, int percent) {
  return Double.valueOf(Math.round(percentageOf(deficit, percent) / 4.0));
}

public static Macros findBalancedMacros(double deficit) {
  return new Macros(
    findProtein(deficit, 30), findCarbs(deficit, 40), findFat(deficit, 30)
  );
}

public static MacrosQuantity findProteinQuantity(double deficit) {
  return new MacrosQuantity(
    findProtein(deficit, 15), findProtein(deficit, 50)
  );
}
```

10.6 Meal Split and Daily Aggregation

The daily allowance is split across four meals with the fixed percentages thirty, thirty, fifteen and twenty-five for breakfast, lunch, snack and dinner. The split is applied to the three macro totals independently, and the resulting twelve numbers are stored in the tracking document as four macro tracks. Because the percentages sum to one hundred, the sum of the four meal budgets equals the daily budget for each macronutrient, which is the property that makes the aggregation on the next update exact.

Listing 10.5 — the meal split and the aggregation performed on update

```
private MealKcal splitForMealKcal(DietPlanTrack track) {
  MacrosTrack macros = new MacrosTrack(
    track.getProtein(), track.getFat(), track.getCarbs()
  );
  return new MealKcal(
    NutrientsCalculator.macrosPercentage(macros, 30),   // breakfast
    NutrientsCalculator.macrosPercentage(macros, 30),   // lunch
    NutrientsCalculator.macrosPercentage(macros, 15),   // snack
    NutrientsCalculator.macrosPercentage(macros, 25)    // dinner
  );
}

public DietPlanTrack updateMacros(
  String dietPlanId, String category, MacrosTrack macros
) {
  DietPlanTrack track = findDietPlanTrackByDietPlanId(dietPlanId);
  MacrosTrack target = getMacrosByCategory(track.getMealKcal(), category);
  target.getProtein().setTaken(macros.getProtein().getTaken());
  target.getFat().setTaken(macros.getFat().getTaken());
  target.getCarbs().setTaken(macros.getCarbs().getTaken());
  adjustMacros(track);
  track.getIntake().setTaken(calcDeficitTaken(track));
  return dietPlanTrackRepository.save(track);
}
```

Two properties of that method are worth emphasising. The category is resolved through a switch over the four names defined by the constants class, and an unknown name raises a bad-request failure rather than silently doing nothing. And the daily intake is recomputed from the macro totals rather than accumulated incrementally, which makes the operation idempotent: submitting the same values twice produces the same document rather than doubling the logged intake.

10.7 Energy Conversion of Logged Intake

The conversion from grams to kilocalories uses the Atwater factors four, nine and four. The two methods that perform it are instructive because they apply the same convention in two directions: one converts the logged values of a meal track, the other converts the allowance of a meal track.

Listing 10.6 — energy conversion of taken and total values

```
public static double calcDeficitTaken(MacrosTrack macros) {
  return (
    macros.getProtein().getTaken() * 4
      + macros.getFat().getTaken() * 9
      + macros.getCarbs().getTaken() * 4
  );
}

public static long calcTotalDeficit(MacrosTrack macros) {
  return Math.round(
    macros.getProtein().getTotal() * 4
      + macros.getFat().getTotal() * 9
      + macros.getCarbs().getTotal() * 4
  );
}
```

The second method is what makes the suggestion feature coherent. When the user asks for dinner suggestions, the tracking service selects the dinner macro track, passes it to the second method and thereby obtains the energy that the dinner budget allows. The prompt therefore carries a number derived from the plan by the same arithmetic that the tracking page uses, rather than by a separate estimate.

10.8 Progress Percentage and Weekly Metrics

The dashboard percentage is the rounded ratio of the logged intake to the plan target, clamped only in the interface. The weekly metrics are assembled by selecting the tracking documents whose date falls between the start of the previous week and the end of today in the Asia/Kolkata zone, projecting each into a weekday abbreviation and a weight, and returning the two arrays beside the scalar metrics.

Listing 10.7 — assembling the dashboard metrics

```
public MetricsDTO getMetricsByDateRange() {
  List&lt;String&gt; days = new ArrayList&lt;&gt;();
  List&lt;Double&gt; weights = new ArrayList&lt;&gt;();
  Date[] range = getOneWeekDateRange();
  DietPlan plan = findDietPlanByUserId();
  DietPlanTrack track = findDietPlanTrackByDietPlanId(plan.getId());
  dietPlanTrackRepository
    .findByDateRangeAndDietPlanId(range[0], range[1], plan.getId())
    .forEach(t -&gt; {
      days.add(getDayName(t.getCreatedAt()));
      weights.add(t.getWeight());
    });
  return buildMetrics(track, plan.getTitle(), plan.getDuration(), days, weights);
}
```

The zone is fixed rather than taken from the server default, which is the correct choice for an application whose users are in one country; it does mean that the weekly window is defined in Indian Standard Time regardless of where the service runs. The range begins one week before today and is inclusive of today, so the chart shows at most eight points even though the interface describes it as a weekly trend.

10.9 Draft Construction and Resumption

The wizard is a stateful conversation implemented over a stateless protocol, and the algorithm that makes it work deserves to be stated as such. Each of the three creation methods follows the same four steps: resolve the user from the security context, locate the pending plan, creating it if the method is the first step and it does not yet exist, copy the submitted values onto the plan, and advance the form status. The pending plan is identified by the pair of user identifier and pending status, which is why a user can hold at most one draft at a time.

Listing 10.8 — the common shape of a wizard step

```
public CaloriesEstimationResponse createDietPlanGoalSelection(
  GoalSelectionRequest request
) {
  DietPlan plan = getPendingDietPlan();   // bad request when absent
  plan.setGoal(request.getGoal());
  plan.setDuration(request.getDuration());
  plan.setFormStatus(FormStatus.MEAL_PREFERENCE);
  if (Plan.WEIGHT_LOSS.equals(plan.getPlan())) {
    plan.setTargetCalories(
      NutrientsCalculator.findDeficit(plan.getTdee(), plan.getGoal().getValue())
    );
  } else {
    plan.setTargetCalories(
      NutrientsCalculator.findSurplus(plan.getTdee(), plan.getGoal().getValue())
    );
  }
  return toCaloriesEstimationResponse(dietPlanRepository.save(plan));
}
```

The interesting property of this shape is that it is idempotent with respect to the draft: submitting the second step twice produces the same document, because the method copies values rather than appending them. The property it does not have is transactional isolation; if two tabs submit the same step concurrently, the last write wins and the earlier submission is lost silently. That is acceptable for a single-user wizard and is recorded here because it is the one concurrency hazard in the construction path.

Resumption depends on the read counterpart. The status method loads any existing pending plan and returns an envelope whose form status tells the client which step to render, together with the payload for that step. A user who closes the browser after the second step is therefore returned to the third step on the next visit, and a user who presses back is served the payload of the earlier step by the POST variant of the status endpoint, which reconstructs it from the stored document rather than from a client-side cache.

10.10 AI Suggestion Pipeline

The suggestion pipeline has five stages, each of which is a single method. The first resolves the calorie allowance for the requested meal. The second formats the prompt from the four filters and the allowance. The third builds the chat model with a system message that fixes the output schema. The fourth posts the model to the provider and extracts the content of the first choice. The fifth parses that content into typed objects. Listing 10.9 reproduces the second stage.

Listing 10.9 — prompt construction

```
private String buildPrompt(FilterDTO filter, long calories) {
  return String.format(
    "Suggest me 5 healthy %s cuisine %s %s foods for %s under %d calories.",
    filter.getCuisineFilter(), filter.getMealTypeFilter(),
    filter.getDietTypeFilter(), filter.getMealFilter(), calories
  );
}
```

Three implementation details are consequences of the specific provider rather than of the algorithm. The system message ends with a token that suppresses a reasoning preamble on models that otherwise emit one. The maximum token count is set to minus one, which the provider interprets as an instruction not to impose an explicit limit rather than as an error. And the response is read by traversal of a JSON tree rather than through a typed DTO, so a change in the provider's envelope does not break compilation; the consequence is that a missing content field yields an empty string and a parsing failure at the fifth stage rather than a clear diagnostic.

10.11 Pseudocode Summary

The following pseudocode states the complete plan-and-track cycle in the notation used for algorithmic description, so that the arithmetic can be reviewed independently of the Java syntax and of the framework that hosts it.

Pseudocode 10.1 — plan construction, activation and daily logging

```
PROCEDURE EstimateEnergy(age, weight, height, gender, activity):
  bmr  &lt;- 10*weight + 6.25*height - 5*age + (5 IF gender = MALE ELSE -161)
  tdee &lt;- bmr * activity.multiplier
  RETURN round(tdee)

PROCEDURE DeriveTarget(tdee, plan, intensity):
  IF plan = WEIGHT_LOSS THEN raw &lt;- tdee - tdee*intensity/100
  ELSE                       raw &lt;- tdee + tdee*(intensity-10)/100
  base &lt;- floor(raw/100)*100
  diff &lt;- raw - base
  RETURN base + (100 IF diff &gt; 50 ELSE (50 IF diff &gt; 0 ELSE 0))

PROCEDURE BuildMacros(targetKcal, preset):
  FOR EACH macro IN {protein, carbs, fat}:
    kcal  &lt;- targetKcal * preset.percent[macro] / 100
    grams &lt;- round(kcal / density[macro])      // 4, 4 and 9
  RETURN grams

PROCEDURE Activate(plan):
  plan.targetCalories &lt;- energy(plan.macros)
  plan.todayWeight    &lt;- plan.weight
  plan.createdAt      &lt;- today
  plan.planStatus     &lt;- ACTIVE
  plan.formStatus     &lt;- COMPLETED
  SAVE plan
  CREATE track(weight = plan.todayWeight, dayCount = 1,
               intake = plan.targetCalories,
               mealKcal = Split(plan.macros, [30, 30, 15, 25]))

PROCEDURE LogMeal(planId, category, grams):
  track &lt;- FindOrCreateTodayTrack(planId)
  meal  &lt;- track.mealKcal[category]
  meal.taken &lt;- grams
  FOR EACH macro IN {protein, carbs, fat}:
    track[macro].taken &lt;- SUM(track.mealKcal[m][macro].taken
                               FOR m IN {breakfast, lunch, snack, dinner})
  track.intake.taken &lt;- 4*protein.taken + 4*carbs.taken + 9*fat.taken
  SAVE track
  RETURN track
```

10.12 Code Organisation and Naming Conventions

The service follows a package-by-feature organisation rather than a package-by-layer organisation. Each of the four features, namely the identity feature, the diet-plan feature, the food-suggestion feature and the chat service, contains its own controller, service, repository, entity and DTO packages, so a change to the plan feature never requires opening a file in another feature's directory. The cross-cutting code lives in six sibling packages: configuration, exception, security, shared, utility and the shared model that holds the value objects used by more than one feature.

Naming is consistent and slightly verbose, which was preferred over abbreviation. Entities are singular nouns. Request and response types end in Request or Response, and the types that exist to shape a response end in DTO. Services end in Service, and they are concrete classes rather than interface-and-implementation pairs, with one exception: the language-model adapter is declared behind an interface, because it is the one collaborator whose implementation might legitimately vary between a live provider and a stub.

The client mirrors the naming of the service. Services end in Service and are file-named with the suffix, with one exception in the plan feature whose file predates the convention and uses a hyphenated form; that exception is recorded in the verification matrix. Components follow the CLI convention of a name, a component suffix and a sibling template and stylesheet, and the single custom pipe and the model interfaces follow the same pattern.

CHAPTER 11
TESTING AND VERIFICATION

11.1 Testing Strategy

The testing strategy has two parts, and they were pursued to different depths. The first part is automated testing inside the repository. The service contains one test class with a single method that asserts the application context loads; this is the smoke test that the project initialiser generates, and it is the only automated test the delivered repository contains. The client contains no test files at all, although the Karma and Jasmine dependencies are declared and the CLI is therefore configured to run them.

The second part is the verification exercise that produced this report. Because the automated suite is inadequate to substantiate the claims made in Chapters 5 to 10, every such claim was checked against the source by reading it, and the results were recorded in the matrix of Appendix B. The matrix uses three states: verified, meaning that the behaviour was traced through the code and, where possible, observed in the running application; inferred, meaning that the behaviour follows from the code but was not exercised; and not implemented, meaning that the repository does not contain the artefact. This chapter reports the outcome of that exercise and is deliberately candid about the limits of the evidence.

11.2 Test Levels and Proposed Test Cases

The architecture supports a conventional test pyramid, and Table 11.1 specifies the cases that the next revision should implement. The table is presented as a specification rather than as a report of executed tests, because only the context-load test currently exists. The expected values are those the implementation produces, derived by hand from the equations of Chapter 10, so the table can serve directly as a set of assertions.

Table 11.1: Test case specification for the Macrova application

ID	Level	Unit under test	Input or condition	Expected result
TC-01	Unit	Basal rate method	Male, 25 years, 175 cm, 95 kg	1923.75
TC-02	Unit	Basal rate method	Female, 30 years, 160 cm, 60 kg	1289.0
TC-03	Unit	Expenditure method	Basal 1923.75, light activity	2645.16
TC-04	Unit	Calorie rounding	2116	2150
TC-05	Unit	Calorie rounding	2000, 2050 and 2051	2000, 2050 and 2100
TC-06	Unit	Deficit method	Expenditure 2645, 20 per cent	2150
TC-07	Unit	Surplus method	Expenditure 2645, 25 per cent	3050
TC-08	Unit	Balanced preset	2150 kilocalories	161, 215 and 72 grams

ID	Level	Unit under test	Input or condition	Expected result
TC-09	Unit	Meal split	Totals 161, 72 and 215 at 30 per cent	48, 22 and 65
TC-10	Unit	Taken-energy conversion	Taken 110, 65 and 170	1705.0
TC-11	Unit	Category validation	breakfast and brunch	true and false
TC-12	Service	Registration	Duplicate username	Bad-request failure
TC-13	Service	Registration	Valid payload	DTO without a password field
TC-14	Service	Authentication manager	Wrong password	Bad-credentials failure
TC-15	Service	Authentication manager	Unknown username	Bad-credentials failure
TC-16	Service	First wizard step	No pending plan	Plan created
TC-17	Service	Pending plan lookup	No pending plan	Bad-request failure
TC-18	Service	Second wizard step	Loss plan, moderate	Target 2150
TC-19	Service	Meal update	Category brunch	Bad-request failure
TC-20	Service	Plan by identifier	Identifier of another user	Not-found failure
TC-21	Service	Duration check	Day count 181, duration 180	True, with the plan inactive
TC-22	Service	Meal allowance	No active plan	500
TC-23	Web	Registration	Missing password	400 with an error array
TC-24	Web	Metrics	No authorisation header	401
TC-25	Web	Metrics	Expired access token	401
TC-26	Web	Plan by identifier	Valid token, foreign identifier	404
TC-27	Web	Meal update	Invalid category in the path	400
TC-28	Web	Token refresh	No cookie	401
TC-29	Integration	Wizard end to end	Four sequential posts	Active plan with a first-day document
TC-30	Integration	Tracking round trip	Log all four meals	Intake equals four protein plus four carbohydrate plus nine fat
TC-31	End to end	Sign-in and dashboard	Valid credentials in a browser	Metrics rendered
TC-32	End to end	Guard redirection	Deep link without a token	Redirect to the login screen

11.3 Executed Verification

The verification exercise examined every class in the repository and every endpoint in the catalogue. The results are summarised in Table 11.2 and set out in full in Appendix B. Of the twenty functional requirements, nineteen are implemented and verified by reading the code, one is inferred because it depends on a third party, and none is missing. Of the ten non-functional requirements, eight are satisfied directly, one is satisfied only in the development profile and one is not satisfied.

Table 11.2: Summary of the verification results

Area	Artefacts examined	Verified	Partially verified	Not implemented
Identity and session	4 classes, 3 filters, 5 endpoints	The four classes and five endpoints	Sign-out clears the cookie but does not revoke the token	Token revocation
Plan construction	1 controller, 1 service, 8 DTOs, 6 endpoints	Six endpoints and both classes	None	Server-side range validation
Daily tracking	1 controller, 1 service, 2 entities, 13 endpoints	Thirteen endpoints	The metrics window is fixed to eight days	None
AI suggestions	1 controller, 1 service, 2 provider classes	The endpoint and the adapter	The prompt and the parse were verified by reading	Persistence of suggestions
History and review	3 components, 3 endpoints	All three of each	None	None
Security	1 configuration, 3 filters, 1 manager, 1 token service	The authentication and authorisation paths	The cross-origin policy is pinned to the development origin	Throttling, transport security and the secure cookie flag
Client navigation	13 routes, 1 guard, 2 interceptors	Eleven guarded routes	Two history routes are unguarded	Unit tests
Automated tests	1 test class	One context-load test	None	All service, web and end-to-end tests

The exercise produced ten findings that a reviewer should know about. They are listed in order of severity, with the artefact concerned and the recommended correction for each.

1. Two history routes are declared without the authentication guard, so a visitor who knows or guesses a plan identifier can load the component. The subsequent request still fails with 401, because the interceptor cannot attach a token, so no data is exposed; the correction is nevertheless to add the guard to both entries.
2. Ownership is enforced by query construction rather than by a central authorisation check. That is effective for every endpoint that queries by identifier and user, but a future endpoint that queried by identifier alone would silently bypass the rule. A method-level check would make the rule structural.
3. Refresh tokens are neither rotated nor revocable. Sign-out clears the cookie, but a copy of the token taken before sign-out remains valid until its expiry. Rotation on every refresh, together with a server-side token family, would close the window.
4. The refresh cookie is issued without the secure attribute while the development server runs over plain HTTP. That is appropriate for a local demonstration and unsuitable for any other deployment; the correction is a profile-specific property rather than a change to the code.
5. There is no throttling or lockout on failed authentication. The manager returns a bad-credentials failure without counting attempts, so an online dictionary attack against the login endpoint is not impeded by the application.

6. The controller advice maps a JSON parsing failure to 410, which is not the conventional choice for an upstream provider failure. A 502 or 500 would be more accurate and would not confuse a client that treats 410 as a permanent condition.
7. The interface exposes three endpoints that no client calls, and the client declares one method that the service does not map. These are dead or inconsistent surfaces that should either be removed or given a caller before the next release.
8. The suggestions collection and the method that would write to it are unused, so the suggestion feature is stateless with respect to the database. That is a deliberate outcome of the decision not to persist generated output, but a repository for data that is never written is a maintenance hazard.
9. The mild intensity is labelled as a ten per cent deficit in the goal-selection template while the enumeration that the service uses carries fifteen per cent. The label is hard-coded presentation text and the arithmetic is correct, so a user who selects the mild option receives a larger deficit than the screen promises; the correction is to render the label from the enumeration value.
10. The automated suite is a single context-load test. Every statement in Chapters 5 to 10 about arithmetic, persistence and security is therefore substantiated by reading the code rather than by executed assertions, which is why Table 11.1 specifies the cases that the next revision must implement.

11.4 Analysis of the AI Feature

The suggestion feature deserves a separate analysis because it is the only part of the system whose correctness cannot be established by reading the code. Three risks were identified. First, the nutritional values are unverified: the model produces plausible numbers that may differ from any published composition table and may differ between calls for the same dish. Second, the schema instruction in the system prompt is a request rather than a constraint, so a compliant response is not guaranteed; the parser will fail on a prose reply and the user will see an error rather than a suggestion. Third, the prompt incorporates the meal allowance but incorporates no allergy, medical condition or dietary restriction beyond the four filters, so a suggestion could be unsuitable for a user with an intolerance.

The mitigations that are implemented are correspondingly modest, and they are stated here so that the report does not overstate the feature. The feature is optional and can be left unused, although the service must still be configured with a key in order to start. Suggestions are never written into a plan, so a model error cannot corrupt the deterministic part of the system. The output is labelled as a suggestion in the interface copy, and this chapter records the caveat beside the screenshot. The mitigations that would

be required before the feature could be recommended to a real user are listed in Chapter 12.

11.5 Performance Observations

No formal performance testing was conducted, and the following observations are qualitative. On the development machine a dashboard load issues two requests and completes in well under a second; the tracking page issues two and completes in a comparable time. The heaviest operation is the weekly metrics query, which reads at most eight documents but does so by scanning the collection in the absence of an index; with an index on the plan identifier and date the query would become a bounded range scan. The suggestion call dominates the latency of that screen, because it waits for a remote inference; the interface shows a loading state during the wait, which is the correct behaviour for an operation whose duration the application does not control.

The client bundle is not code-split. The production build reports a single initial bundle within the default budget of one megabyte, and the chart library is imported wholesale by the weight chart component rather than limited to the modules that the chart uses. At the current size the choice is defensible; splitting the wizard and the chart would reduce the time to first paint on the login screen, which is the page a new visitor sees first.

11.6 Limitations Recorded

- The automated test suite is a single smoke test; no unit, web-slice or end-to-end test is present.
- Input ranges are validated in the browser only; the service accepts any numeric value that binds successfully to the request type.
- Uniqueness of the username and the address is enforced by application checks rather than by a unique index, so concurrent registrations could create duplicates.
- There is no password reset, no address verification and no account deletion.
- Refresh tokens are neither rotated nor revocable, and there is no throttling of failed sign-in attempts.
- The cross-origin allow-list, the insecure cookie flag and the absence of transport security confine the deployment to a local development setting.
- The suggestion feature returns unverified nutrition data, considers neither allergies nor medical conditions, and no index is declared on any collection.
- Two history routes lack the authentication guard.
- The mild intensity is mislabelled in the goal-selection template.

CHAPTER 12
CONCLUSION AND FUTURE ENHANCEMENTS

12.1 Summary of the Work

This project designed, implemented and documented Macrova, a web application that turns a published energy-balance equation into a working planning and tracking tool. The delivered system comprises 71 Java source files totalling 2,779 lines, 60 TypeScript files totalling 1,915 lines, 20 component templates totalling 1,238 lines and 19 stylesheets totalling 2,381 lines, organised into eight functional modules over a three-tier architecture. It exposes 26 REST endpoints, persists four collections of which three are written in normal operation, and renders fifteen distinct screens.

Functionally, the system allows a user to register, authenticate through a stateless token exchange, construct a plan through a resumable four-step wizard, receive a calorie and macronutrient budget derived from the Mifflin–St Jeor equation, log intake per meal and weight per day, observe progress on a dashboard with a weight trend chart, review the daily records of any past plan, and request meal suggestions constrained by the allowance of a specific meal. The arithmetic behind every figure is isolated in a single documented utility class, and the report reproduces both the equations and the code that implements them.

12.2 Conclusion

The project demonstrates a point that motivated it in the first place: the difficult part of a diet application is not the formula but the consistency with which the formula is applied. The metabolic equation occupies one line of Java, and the value of the system lies in the discipline that surrounds it. There is a single utility class rather than repeated expressions. A stored plan records the assumptions that produced its target. A tracking document recomputes rather than accumulates, so a repeated submission cannot inflate the day. And a wizard whose state lives on the server resumes where it stopped. Each of those decisions is small, and together they produce an application whose numbers can be audited by hand, which is the property the design set out to achieve.

The project also demonstrates the value of an honest verification exercise. The repository delivered here does not have the test suite that its architecture would support, its deployment profile is a development profile, and its suggestion feature returns data that is not verified. Those facts are recorded in Chapter 11 rather than omitted, because a report that claimed otherwise would fail the first review that attempted to re-

produce its claims. The distinction maintained throughout the document between behaviour that was verified, behaviour that was inferred from the code and behaviour that does not exist is, in the author's view, the most transferable result of the project.

12.3 Learning Outcomes

The work produced five outcomes that were not anticipated at the outset. The first is a practical understanding of the difference between stateless and stateful authentication, in particular the consequence that a stateless token cannot be revoked without a server-side list, which is why the sign-out in this application is a cookie deletion. The second is an appreciation of document-database modelling: the decision to embed four meal tracks inside a daily document was taken because the tracking page reads them together, and that decision eliminated the joins a relational design would have required. The third is the difficulty of verifying a generative feature, which has no oracle and whose output differs on every call. The fourth is the value of typed component contracts, because the typed inputs and outputs of the wizard turned four potential runtime errors into compile errors. The fifth is that documentation is a design activity: writing the data dictionary exposed the duplicated target computation described in Section 7.3, which had not been noticed while the code was being written.

12.4 Future Enhancements

The following enhancements are proposed in the order in which they would deliver value. None is implemented in the delivered system, and each is stated with the artefact it would change.

1. Automated test suite. Implement the thirty-two cases of Table 11.1 as unit tests for the calculator and the services, web-slice tests for the controllers and component tests for the pipe and the guard. This is first because every later change depends on it.
2. Indexes. Add the unique indexes on the username and the address and the compound indexes of Table 7.8, so that the current collection scans become bounded range scans.
3. Refreshed-token hardening. Rotate the refresh token on every use, store a token family and a revocation timestamp, add a failed-sign-in counter with a temporary lockout, and set the cookie attributes from configuration rather than from literals.
4. Server-side validation. Move the range checks from the Angular submit handlers into validation annotations on the request types, so that the rules hold for any client.
5. A verified food catalogue. Replace free-text macro entry with a curated food table, so that a meal is recorded as one or more food items with portions; the cur-

rent entry would remain as a fallback for composite dishes.

6. Password reset and address verification. Add a single-use token collection, a mail sender and the two flows, which are the minimum required before the application can be used by anyone other than its author.
7. Deletion and export. Provide a cascade deletion of a user and their plans, and an export of a plan's daily records, so that the user's data is portable.
8. Code splitting. Lazy-load the wizard and the chart and reduce the chart import surface, in order to shorten the time to first paint on the login screen.
9. Restriction-aware suggestions. Add an allergy and restriction profile to the prompt, cross-check every returned dish name against a small curated list of unsafe items, and label the results as unverified in the interface.
10. A second client. Expose the same interface to a mobile shell, since the service is already stateless and therefore ready for one.

REFERENCES

1. Books

- C. S. Walls, Spring in Action, 6th ed. Shelter Island, NY, USA: Manning Publications, 2022.
- J. Bloch, Effective Java, 3rd ed. Boston, MA, USA: Addison-Wesley Professional, 2018.
- M. Kleppmann, Designing Data-Intensive Applications. Sebastopol, CA, USA: O'Reilly Media, 2017.
- M. Fowler, Patterns of Enterprise Application Architecture. Boston, MA, USA: Addison-Wesley, 2002.
- P. J. Sadalage and M. Fowler, NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence. Boston, MA, USA: Addison-Wesley, 2012.
- R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach, 9th ed. New York, NY, USA: McGraw-Hill Education, 2020.
- I. Sommerville, Software Engineering, 10th ed. Harlow, UK: Pearson Education, 2016.
- A. Freeman, Pro Angular. Berkeley, CA, USA: Apress, 2024.
- E. Evans, Domain-Driven Design: Tackling Complexity in the Heart of Software. Boston, MA, USA: Addison-Wesley, 2004.
- N. Ferguson, B. Schneier and T. Kohno, Cryptography Engineering: Design Principles and Practical Applications. Indianapolis, IN, USA: Wiley, 2010.
- J. Nielsen, Usability Engineering. San Francisco, CA, USA: Morgan Kaufmann, 1994.

2. Research Papers, Standards and Reports

- M. D. Mifflin, S. T. St Jeor, L. A. Hill, B. J. Scott, S. A. Daugherty and Y. O. Koh, “A new predictive equation for resting energy expenditure in healthy indi-

viduals," American Journal of Clinical Nutrition, vol. 51, no. 2, pp. 241–247, Feb. 1990.

- J. W. Frankenfield, L. Roth-Yousey and C. Compher, "Comparison of predictive equations for resting metabolic rate in healthy nonobese and obese adults: a systematic review," Journal of the American Dietetic Association, vol. 105, no. 5, pp. 775–789, May 2005.
- Food and Agriculture Organization, World Health Organization and United Nations University, Human Energy Requirements: Report of a Joint FAO/WHO/UNU Expert Consultation. Rome, Italy: FAO Food and Nutrition Technical Report Series 1, 2004.
- World Health Organization, Obesity: Preventing and Managing the Global Epidemic. Geneva, Switzerland: WHO Technical Report Series 894, 2000.
- L. E. Burke, J. Wang and M. A. Sevick, "Self-monitoring in weight loss: a systematic review of the literature," Journal of the American Dietetic Association, vol. 111, no. 1, pp. 92–102, Jan. 2011.
- S. Michie, C. Abraham, C. Whittington, J. McAteer and S. Gupta, "Effective techniques in healthy eating and physical activity interventions: a meta-regression," Health Psychology, vol. 28, no. 6, pp. 690–701, Nov. 2009.
- R. S. Sandhu, E. J. Coyne, H. L. Feinstein and C. E. Youman, "Role-based access control models," IEEE Computer, vol. 29, no. 2, pp. 38–47, Feb. 1996.
- N. Provos and D. Mazières, "A future-adaptable password scheme," in Proc. USENIX Annual Technical Conference, Monterey, CA, USA, 1999, pp. 81–91.
- M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015.
- M. Jones, J. Bradley and N. Sakimura, "JSON Web Signature (JWS)," IETF RFC 7515, May 2015.
- R. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, University of California, Irvine, CA, USA, 2000.
- R. Bommasani et al., "On the opportunities and risks of foundation models," arXiv:2108.07258, Aug. 2021.
- OWASP Foundation, "OWASP Top 10:2021 — The Ten Most Critical Web Application Security Risks," Open Worldwide Application Project, 2021.
- M. Fowler, "The practical test pyramid," martinfowler.com, Feb. 2018.

3. Official Documentation and Technical Sources

- Broadcom and VMware Tanzu, “Spring Boot Reference Documentation, version 3.4,” https://docs.spring.io/spring-boot/ (accessed November 2026).
- Spring Security Team, “Spring Security Reference — Servlet Applications,” https://docs.spring.io/spring-security/reference/ (accessed November 2026).
- Spring Data Team, “Spring Data MongoDB Reference Documentation,” https://docs.spring.io/spring-data/mongodb/reference/ (accessed November 2026).
- Google Angular Team, “Angular Documentation — Standalone Components, Signals and Functional Interceptors,” https://angular.dev/ (accessed November 2026).
- Google Angular Team, “Angular CLI Builders and Proxy Configuration,” https://angular.dev/tools/cli/ (accessed November 2026).
- Auth0, “java-jwt Library Documentation, version 4.5.0,” https://github.com/auth0/java-jwt (accessed November 2026).
- MongoDB Inc., “MongoDB Manual — Data Modelling and Indexes,” https://www.mongodb.com/docs/manual/ (accessed November 2026).
- Chart.js Contributors, “Chart.js Documentation, version 4.4,” https://www.chartjs.org/docs/ (accessed November 2026).
- OpenRouter, “API Reference — Chat Completions,” https://openrouter.ai/docs (accessed November 2026).
- Oracle Corporation, “Java Platform, Standard Edition 21 API Specification,” https://docs.oracle.com/en/java/javase/21/docs/api/ (accessed November 2026).
- Project Lombok, “Lombok Reference Documentation,” https://projectlombok.org/features/ (accessed November 2026).
- ModelMapper, “ModelMapper User Manual, version 3.2,” https://modelmapper.org/ (accessed November 2026).
- JUnit Team, “JUnit 5 User Guide,” https://junit.org/junit5/docs/current/user-guide/ (accessed November 2026).
- MDN Web Docs, “HTTP Cookies, Cross-Origin Resource Sharing and HTTP Caching,” https://developer.mozilla.org/ (accessed November 2026).
- Git SCM, “Pro Git, 2nd ed.,” https://git-scm.com/book/en/v2 (accessed November 2026).

APPENDICES

Appendix A Endpoint Reference Summary

This appendix reproduces the interface in a form that a client developer can use without the narrative of Chapter 8. Table A.1 groups the twenty-six endpoints by resource and states the authentication requirement of each group.

Table A.1: Endpoint groups with authentication requirements

Group	Base path	Endpoints	Authentication
Identity	/user and /authenticate	Five	Registration and refresh are public; the others require a bearer token
Plan construction	/diet-plan/form	Six	Bearer token
Plan retrieval	/diet-plan	Eight	Bearer token
Tracking	/diet-plan/track and /diet-plan/{category}	Four	Bearer token
Reporting	/diet-plan/metrics, /status and /overview	Three	Bearer token
Suggestions	/ai-food-suggest	One	Bearer token

The request and response shapes are declared by the DTO classes of the service, which are summarised in Table 8.1 and illustrated by the listings of Chapter 8. A client that implements the five identity endpoints, the six construction endpoints and the tracking endpoints can reproduce the behaviour of the delivered client.

Appendix B Verification Matrix

Table B.2 applies the same treatment to the ten non-functional requirements. The matrix records, for every functional and non-functional requirement, the artefact that implements it and the result of the verification exercise. The state column uses the three values defined in Section 11.1. The complete set of values is given in Table B.1.

Table B.1: Verification matrix for the functional requirements

ID	Implementing artefact	State	Note
FR-01	User controller, user service	Verified	Client validation precedes the call
FR-02	User service existence checks	Verified	Application level rather than an index

ID	Implementing artefact	State	Note
FR-03	Authentication filter, custom manager	Verified	Uniform failure message
FR-04	Token service, authentication filter	Verified	Cookie is HttpOnly and SameSite Lax
FR-05	Refresh and logout methods	Verified	Cookie holds the refresh token
FR-06	Body-metrics step	Verified	Expenditure rounded once
FR-07	Goal-selection step	Verified	Deficit or surplus by plan type
FR-08	Calculator presets, meal-preference step	Verified	Custom values accepted
FR-09	Meal-preference step	Verified	None
FR-10	Activation method	Verified	Creates the first daily document
FR-11	Status endpoint and envelope	Verified	Backward navigation uses the POST variant
FR-12	Tracking service split	Verified	Split stored with the document
FR-13	Meal update method	Verified	Totals recomputed from four categories
FR-14	Taken-energy conversion	Verified	Atwater factors
FR-15	Weight update method	Verified	Updates both documents
FR-16	Metrics assembly	Verified	Eight-day window
FR-17	Duration check	Verified	Evaluated on page load, not scheduled
FR-18	Projection queries in both repositories	Verified	Descending by date
FR-19	Day overview component	Verified	Inputs disabled
FR-20	Suggestion controller, service and adapter	Inferred	Values unverified; the parse depends on model compliance

Table B.2: Verification matrix for the non-functional requirements

ID	Requirement	State	Evidence
NFR-01	Passwords stored only as adaptive hashes	Verified	BCrypt encoder; the hash never leaves the service
NFR-02	Every operation scoped to the authenticated user	Verified	The session component is called in every plan and tracking query
NFR-03	Stateless sessions	Verified	Session policy is stateless; the bearer token is verified per request
NFR-04	A first-time user reaches a plan unaided	Verified	Four-step wizard with inline guidance
NFR-05	Every screen offers a recoverable action	Verified	Empty state, error page and persistent header
NFR-06	Dashboard load under a second	Inferred	Observed qualitatively in Section 11.5
NFR-07	Missing daily document created on demand	Verified	Find-or-create path in the tracking service
NFR-08	Energy arithmetic in exactly one class	Verified	The calculator class
NFR-09	Buildable from a clean checkout	Verified	Maven wrapper and Angular CLI are both committed or declared
NFR-10	Tier interface documented to the field level	Verified	Chapter 8 and Appendix A

Appendix C Selected Source Listings

The three listings in this appendix are reproduced from the repository and are included because they are the artefacts most likely to be consulted during an evaluation: the security configuration, which defines the whole request policy; the method that performs the daily aggregation; and the enumeration and constants that constrain the domain.

Listing C.1 — the security filter chain (abridged)

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
  AuthenticationFilter authenticationFilter =
    new AuthenticationFilter(customAuthenticationManager, jwtTokenService);
  authenticationFilter.setFilterProcessesUrl("/authenticate");
  http
    .csrf(csrf -&gt; csrf.disable())
    .cors(cors -&gt; cors.configurationSource(corsConfigurationSource()))
    .authorizeHttpRequests(auth -&gt; auth
      .requestMatchers(HttpMethod.POST,
        SecurityConstants.REGISTER_PATH,
        SecurityConstants.REFRESH_PATH).permitAll()
      .anyRequest().authenticated())
    .addFilterBefore(new ExceptionHandlerFilter(), AuthenticationFilter.class)
    .addFilter(authenticationFilter)
    .addFilterAfter(new JWTAuthenticationFilter(jwtTokenService),
                    AuthenticationFilter.class)
    .sessionManagement(s -&gt;
      s.sessionCreationPolicy(SessionCreationPolicy.STATELESS));
  return http.build();
}
```

Listing C.2 — the aggregation of a day's totals

```
private void adjustMacros(DietPlanTrack dietPlanTrack) {
  MealKcal mealKcal = dietPlanTrack.getMealKcal();
  dietPlanTrack.getProtein().setTaken(
    mealKcal.getBreakfast().getProtein().getTaken()
      + mealKcal.getLunch().getProtein().getTaken()
      + mealKcal.getDinner().getProtein().getTaken()
      + mealKcal.getSnack().getProtein().getTaken());
  dietPlanTrack.getCarbs().setTaken(
    mealKcal.getBreakfast().getCarbs().getTaken()
      + mealKcal.getLunch().getCarbs().getTaken()
      + mealKcal.getDinner().getCarbs().getTaken()
      + mealKcal.getSnack().getCarbs().getTaken());
  dietPlanTrack.getFat().setTaken(
    mealKcal.getBreakfast().getFat().getTaken()
      + mealKcal.getLunch().getFat().getTaken()
      + mealKcal.getDinner().getFat().getTaken()
      + mealKcal.getSnack().getFat().getTaken());
}
```

Listing C.3 — the meal category constants and their validation

```
public final class MacrosConstants {
  public static final String BREAKFAST = "breakfast";
  public static final String LUNCH = "lunch";
  public static final String SNACK = "snack";
  public static final String DINNER = "dinner";

  private MacrosConstants() {}

  public static boolean validateMacro(String macro) {
    return switch (macro.toLowerCase()) {
      case BREAKFAST, LUNCH, SNACK, DINNER -&gt; true;
      default -&gt; false;
    };
  }
}
```

Appendix D User Guide

This appendix describes how to run the delivered system and how to carry out the sequence of operations that the screenshots in Chapter 9 record. It assumes a machine with a Java 21 runtime, Node.js and a running database instance.

1. Start the database. Confirm that the daemon is listening on the default port and that the application can create a database named macrova.
2. Supply the three environment variables. The service requires a token secret, a key for the model provider and a model identifier; without all three it refuses to start, which is the configuration decision recorded in Section 5.5.
3. Start the service from the backend directory with the Maven wrapper and wait until the log reports that the embedded container has started.
4. Start the client from the frontend directory with npm start and open the development server address in a browser. The proxy forwards requests under the /api prefix to the service.
5. Register an account, sign in, and confirm that the application routes to the empty state because no plan exists.
6. Create a plan. Enter the body metrics, choose an intensity and a duration, adjust the macro split if required, review the summary and confirm. The dashboard then shows the first day of the plan with an empty goal meter.
7. Log a day. Open the progress page, enter the macronutrient grams for each meal and update the weight. The meter and the summary change with each submission.
8. Review the history. Open the history list, select the plan and open one of its days to see the read-only view.
9. Request suggestions. Open the suggestions page, choose the four filters and request suggestions; a failure here indicates an unreachable provider or a malformed

reply rather than a defect in the rest of the system.

Two operational notes complete the guide. The application stores dates at UTC midnight, so a document created shortly after local midnight in a positive-offset zone may appear under the previous date in a direct database inspection. And the development profile issues the refresh cookie without the secure attribute, so the system should not be exposed beyond the developer's machine without the configuration changes listed in Chapter 12.